

Fabrication of Wheeled Inverted Pendulum

Practical Exercise

V1.0 20260317

Contents

Class Calendar	4
Course Schedule and Weekly Tasks	5
Grading Criteria	6
1 Purpose of the Exercise	7
2 Overview of Inverted Pendulum Control	7
2.1 Model of the Inverted Pendulum	8
2.2 Control of Pendulum Angle	9
2.3 PD Control	9
2.4 Is this enough to invert?	10
2.5 Motor Torque	11
3 Week 1: Assembly & Bring-up	12
3.1 Week 1 Overview & Objectives	12
3.2 Electronic Components Explanation	12
3.2.1 Resistor and Capacitor Markings (Value & Polarity)	12
3.2.2 Motor Driver (DRV8832)	13
3.2.3 Photo-interrupter & Inclination Sensor	13
3.2.4 Three-terminal Regulator	15
3.3 Parallel Assembly Tasks	15
3.3.1 Task A: Mechanical Assembly (Student A)	15
3.3.2 Task B: Microcontroller Setup (Student B)	16
3.3.3 Task C: Breadboard Wiring Design (Student C)	21
3.4 Circuit Assembly (Team Collaboration)	22
3.4.1 Safety precautions	23
3.4.2 Assembly strategy (3 stages)	23
3.4.3 Stage 1: Motor Driver & Motor Testing	23
3.4.4 Stage 2: Angle Sensor Testing	25
3.4.5 Stage 3: Complete Circuit Assembly and Integration	26
3.5 Week 1 Submission	28
4 Week 2: Encoder & Position Control	30
4.1 Week 2 Overview & Objectives	30
4.2 Parallel Tasks	30
4.3 Task A: Encoder Fabrication and Installation (Student A)	30
4.3.1 Rotary Encoder	30
4.3.2 Fabrication of Rotary Encoder	31
4.3.3 Soldering Photo-interrupter	33
4.3.4 Adjustment and Reading of Photo-interrupter	33

4.4	Task B: Add Encoder Feedback to Code and Debug (Student B)	33
4.4.1	Using the Complete Control Program	33
4.4.2	Preparation for Communication	34
4.4.3	Initial Settings	34
4.4.4	Trying Inversion (Proportional Control)	35
4.4.5	Calculation of Angular Velocity	36
4.4.6	Inversion adding Derivative Control	38
4.4.7	Summary so far	38
4.4.8	Position Control	38
4.4.9	Calculation of Velocity and Addition of Derivative Gain	39
4.4.10	Let's Run	40
4.4.11	The rest depends on ingenuity	40
4.5	Task C: MATLAB Simulation with Encoder Feedback (Student C)	40
4.5.1	Role of Simulation	40
4.5.2	Setting Up MATLAB Environment	41
4.5.3	Building the Simulink Model	41
4.5.4	How the Plant is Built	41
4.5.5	Controller Internal Setup	43
4.5.6	Running the Simulation	44
4.5.7	PD Controller Design and Parameter Tuning	44
4.5.8	Analyzing Simulation Results	45
4.5.9	Optional Extensions	45
4.5.10	From Simulation to Real Hardware	45
4.5.11	Assignment: Add Position and Velocity Loops with Encoder-like Feedback	46
4.6	Supplements: Troubleshooting & Speed	46
4.7	Week 2 Submission	48
5	Week 3: Advanced Modeling & Load Testing	49
5.1	Week 3 Overview & Objectives	49
5.2	Parallel Tasks	49
5.3	Task A: Load Attachment and Hardware Tests (Student A)	49
5.3.1	A1. Choose and attach the load	49
5.3.2	A2. Hardware test protocol (baseline → load)	49
5.3.3	A3. What to record (minimum evidence)	50
5.4	Task B: Extend Simulation (Wheel Dynamics + Ground Friction) (Student B)	50
5.4.1	B1. Add wheel dynamics	50
5.4.2	B2. Add a friction model	50
5.4.3	B3. Run and document enhanced simulation	50
5.5	Task C: PID Tuning and Controller Comparison (Student C)	50
5.5.1	C1. Tune PID (or PD) under load in MATLAB	50
5.5.2	C2. Try at least one advanced controller (recommended: LQR)	51
5.5.3	C3. Simulation vs hardware comparison	51
5.6	Week 3 Submission	51
6	Week 4: Final Presentation	52
6.1	Week 4 Overview & Objectives	52
6.2	Week 4 Requirements	52
6.3	Week 4 Submission	52
•	List of Appendices	53
A	Parts List	54
B	RE-280RA datasheet (Mabuchi Motor)	58
C	Processing Oscilloscope Manual	59

D	Circuit example	61
E	DRV8832 datasheet (Texas Instruments)	62
F	TPR-105F datasheet	64
G	LD1117V33 datasheet	69

Class Calendar

Sessions: **Mondays, 13:00–18:30 (JST, Asia/Tokyo)**; The course runs in **three cohorts (4 weeks each)**. **Week 0** marks preparation; please complete required setup *before* Week 1 begins (Week 0 is not shown).

Legend: Cohort 1 Cohort 2 Cohort 3 OFF **Badge:** C#-W# = cohort/week, OFF = no class.

Apr 2026

Mon	Tue	Wed	Thu	Fri	Sat	Sun
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27 C1-W1	28	29	30			

May 2026

Mon	Tue	Wed	Thu	Fri	Sat	Sun
				1	2	3
4 OFF	5	6	7 C1-W2	8	9	10
11 C1-W3	12	13	14	15	16	17
18 C1-W4	19	20	21	22	23	24
25 C2-W1	26	27	28	29	30	31

Jun 2026

Mon	Tue	Wed	Thu	Fri	Sat	Sun
1 OFF	2	3	4	5	6	7
8 C2-W2	9	10	11	12	13	14
15 C2-W3	16	17	18	19	20	21
22 C2-W4	23	24	25	26	27	28
29 C3-W1	30					

Jul 2026

Mon	Tue	Wed	Thu	Fri	Sat	Sun
		1	2	3	4	5
6 C3-W2	7	8	9	10	11	12
13 C3-W3	14	15	16 C3-W4	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		

Course Schedule and Weekly Tasks

Week	Tasks	Individual Responsibilities	Submissions
Week 0	Preparation & Course Study - Study lecture notes and theory - Prepare computer and software environment - Install required software / set up accounts (MATLAB, Keil Studio Cloud, Processing, Fritzing)	All students: - Read through the entire lecture text - Prepare your PC environment Student A: check the hardware kit contents (see Appendix A) Student B: set up Keil Studio Cloud account + install Processing (see Section 3.3.2 ; see Appendix C) Student C: install Fritzing + MATLAB (see Section 3.3.3 ; see Section 4.5.2)	None
Week 1	Assembly & Bring-up - Mechanical assembly - Microcontroller bring-up and debugging - Breadboard circuit design (Fritzing) - Integration and debugging	Step 1 (individual): - Student A: mechanical assembly (see Section 3.3.1) - Student B: microcontroller bring-up/debug (see Section 3.3.2) - Student C: Fritzing breadboard design (see Section 3.3.3) Step 2 (team): integrate and debug successfully (see Section 3.4)	- Stand demo video
Week 2	Encoder & Position Control - Build encoder + validate signals - Close the loop: position control (compare with simulation)	Student A: encoder hardware (see Section 4.3) Student B: encoder readout + control on mbed (see Section 4.4) Student C: simulation + comparison (see Section 4.5)	Group report: - Position-control video - Encoder A/B plots - Sim vs. hardware comparison
Week 3	Advanced Modeling & Load Testing - Add wheel dynamics and ground friction to simulation - Add load to the cart/pendulum and test on hardware - Tune PID gains in MATLAB and try other controllers (LQR/MPC, etc.) and compare performance	Student A: - Attach load (e.g., weights) to the inverted pendulum cart - Run hardware tests under load, record video Student B: - Extend the MATLAB/Simscape model: wheel dynamics + ground friction - Run and document the enhanced simulation Student C: - Tune PID gains in MATLAB with added load (e.g., PID Tuner; for Simulink-model-based tuning, Simulink Control Design is needed) - Then try other controllers (LQR, MPC, etc.) and compare simulation vs real results	Group report: - Enhanced simulation results - Load test video - PID tuning + controller comparison documentation
Week 4	Final Presentation - Prepare presentation slides - Demonstrate working system - Q&A and discussion	All students: - Prepare 10-minute presentation covering: theory, design, simulation, hardware results, challenges - Demonstrate live inverted pendulum control - Answer technical questions	Final submission: - Slides - Final demo video - Presentation & Q&A

Grading Criteria

Week	Deliverables	Criterion	Score tiers
W0 (0 pts)	Preparation check	Completion only	No submission required.
W1 (30 pts)	Stand demo video — upright stand	Standing (max 20)	20 — upright ≥ 15 s, full system in frame 12 — achieved but < 15 s or marginal 0 — not achieved
		Safety & assembly (max 5)	5 — full assembly visible 0 — not visible
		Video clarity (max 5)	5 — all key parts unobscured throughout 3 — key behavior still identifiable 0 — unusable
W2 (20 pts)	Group report — position-control video — encoder A/B plots — sim vs. hardware	Position control (max 8)	8 — inversion ≥ 10 s within ± 0.5 m of set-point 5 — inversion held but position drift large 0 — inversion lost within 5 s
		Encoder (max 7)	7 — A/B waveforms labeled <i>and</i> direction reversal verified 5 — waveforms shown, direction not verified 2 — pulse count only, no phase analysis 0 — not submitted
		Simulation (max 5)	5 — $x(t)$, $\dot{x}(t)$ plots sim vs. hardware <i>and</i> discrepancy explained 3 — plots included, no explanation 1 — qualitative description only
W3 (20 pts)	Group report — enhanced sim — load test video — controller comparison	Enhanced model (max 6)	6 — wheel inertia + friction + numerical params 4 — two of three elements 2 — one of three elements 0 — no enhancement over W2
		Load test (max 7)	7 — stable ≥ 5 s <i>and</i> load mass/location identified 5 — stable but load unquantified 2 — attempted with documentation 0 — not submitted
		Controller comparison (max 7)	7 — PID metrics (rise time, overshoot, settling) + structurally different controller (LQR / state feedback / feedforward) with same metrics 4 — PID only, or same-family alternative 2 — qualitative discussion only
W4 (30 pts)	Final submission — slides — final demo video — presentation & Q&A	Slides (max 10)	10 — all four elements with data: theory, design, validation, lessons 7 — three of four elements 3 — ≤ 2 elements or text-only
		Demo video (max 10)	10 — stable ≥ 10 s <i>and</i> enable/disable sequence clear 6 — stable but enable/disable sequence unclear 2 — demo fails but honestly documented 0 — not submitted
		Q&A (max 10)	5 — ≥ 2 technical questions answered correctly 5 — answers consistent with submitted materials <i>Partial credit applied within each sub-criterion.</i>

!!! Please Read Before the Exercise !!!

- **Pre-study of this text:** Before starting the exercise, read this text to understand the overview of the work and the theoretical background. Starting work without reading the text may lead to fatal mistakes that make it difficult to continue the exercise.
- **Course Materials:** All supplementary materials (debugging tools, example programs, appendix PDFs, MATLAB models) are provided in the `course_materials/` folder¹. The `debug_tools/` sub-folder contains three essential tools with complete documentation (README files in both Markdown and PDF formats). The `six_channel_oscilloscope` tool includes the complete inverted pendulum control program that you will use in Section 4.4.
- **PC Preparation:** For this practical exercise, we will use the PCs in the design exercise room, so you do not need to prepare your own PC. However, it is possible to use your own PC. In that case, Windows or Macintosh is required. Also, a USB port (Type-A) is required to connect to the control microcontroller. If you only have a Type-C port, please prepare a hub that can connect to Type-A.
- **Work on the day:** Installation of the programming environment and assembly of the mechanism will be explained on the day of the exercise.

1 Purpose of the Exercise

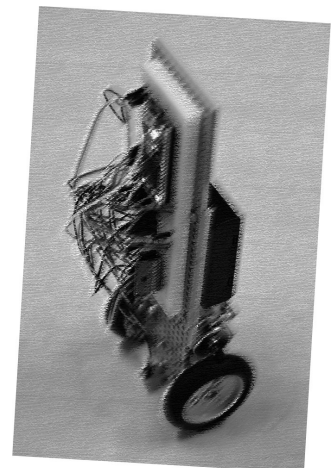
In this exercise, by building a wheeled inverted pendulum (almost) from scratch, we aim to master:

- Basic usage of embedded microcontrollers
- Principle and structure of encoders
- Basics of sensor signal processing
- Basics of feedback control

An inverted pendulum is literally a pendulum that is inverted. An inverted pendulum is unstable, but it can be maintained in a stable inverted state by moving the base part according to the inclination of the pendulum. The operation of moving the base according to the inclination is a typical example of "feedback control," and the inverted pendulum is widely used as an exercise material for control and mechatronics due to its clarity.

There are two types of inverted pendulums: one with a pendulum attached to a linear stage, and one where the pendulum itself travels. The latter corresponds to the wheeled inverted pendulum in this exercise. The principle of the wheeled inverted pendulum is used in mobility devices represented by Segway and wheeled robots.

Although wheeled inverted pendulum kits are commercially available, we will not use such kits in this exercise. Instead, we aim to *understand the configuration and mechanism more deeply by building the inverted pendulum ourselves using only basic parts*. Also, since sensors are very important in feedback control, we would like you to *experience the difficulty and importance of sensors* by making your own sensors (using more primitive sensor elements).



2 Overview of Inverted Pendulum Control

¹Course materials repository: <https://github.com/UTokyo2026/UTokyo-Control-Practice-2026>

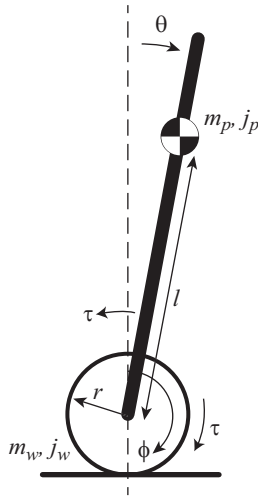


Figure 1: Simplified model

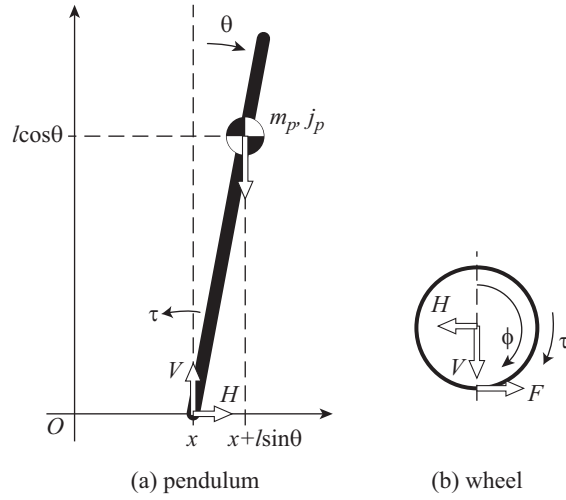


Figure 2: Forces and torques

2.1 Model of the Inverted Pendulum

First, let's consider the overview of the control of the wheeled inverted pendulum. Figure 1 is a simplified model of the inverted pendulum. The pendulum (= body of the inverted pendulum, mass m_p , moment of inertia j_p) stands from the axis of the tires (mass m_w , moment of inertia j_w). There is a motor between the tire and the pendulum, and when the motor is driven, opposite torques ($\pm\tau$) are generated on both. Let the horizontal center position of the tire be x . For simplicity, assume there is no friction or loss other than the friction acting on the tire from the ground. We want to control the angle of the inverted pendulum θ and the tire position x (= body position).

Consider the equations of motion for the pendulum and the tire separately as shown in Figure 2. First, the equation of motion for the pendulum can be written as:

$$\text{Horizontal} \quad m_p \frac{d^2}{dt^2}(x + l \sin \theta) = m_p(\ddot{x} + l\ddot{\theta} \cos \theta - l\dot{\theta}^2 \sin \theta) = H \quad (1)$$

$$\text{Vertical} \quad m_p \frac{d^2}{dt^2}(l \cos \theta) = -m_p(l\ddot{\theta} \sin \theta + l\dot{\theta}^2 \cos \theta) = V - m_p g \quad (2)$$

$$\text{Rotational} \quad j_p \ddot{\theta} = lV \sin \theta - lH \cos \theta - \tau \quad (3)$$

Here, H and V are the horizontal and vertical components of the force acting between the tire and the pendulum. On the other hand, the equation of motion for the tire is:

$$\text{Horizontal} \quad m_w \ddot{x} = F - H \quad (4)$$

$$\text{Rotational} \quad j_w \ddot{\phi} = \tau - rF \quad (5)$$

(The vertical direction is omitted because it balances with the normal force from the ground). Here, F is the friction force received from the ground (which drives the tire), and we assume there is no slip between the tire and the ground ($x = r\phi$). From this series of equations, we eliminate the internal forces H , V and the friction force F to find equations expressed only by the pendulum angle θ and position x .

First, substituting H , V obtained from equations (1) and (2) into equation (3), we get:

$$j_p \ddot{\theta} = m_p g l \sin \theta - m_p l \ddot{x} \cos \theta - m_p l^2 \ddot{\theta} - \tau \quad (6)$$

Next, eliminating F from equations (4) and (5) to find H :

$$H = \frac{\tau}{r} - \left(m_w + \frac{j_w}{r^2} \right) \ddot{x} \quad (7)$$

Substituting this into equation (1), we get:

$$\frac{\tau}{r} = \left((m_p + m_w) + \frac{j_w}{r^2} \right) \ddot{x} + m_p l \cos \theta \ddot{\theta} - l \sin \theta \dot{\theta}^2 \quad (8)$$

Equations (6) and (8) are the basic equations of the inverted pendulum. Since these equations contain nonlinear terms (such as $\sin$ and $\dot{\theta}^2$) and are difficult to handle as they are, we linearly approximate them assuming that the inclination angle θ is minute ($\sin \theta \simeq \theta$, $\cos \theta \simeq 1$, $\dot{\theta}^2 \simeq 0$). Then, the following pair of linear differential equations is obtained. This is the linear approximation model of the inverted pendulum.

$$(j_p + m_p l^2) \ddot{\theta} = m_p g l \theta - m_p l \ddot{x} - \tau \quad (9)$$

$$\frac{\tau}{r} = \left((m_p + m_w) + \frac{j_w}{r^2} \right) \ddot{x} + m_p l \ddot{\theta} \quad (10)$$

2.2 Control of Pendulum Angle

Ultimately, we want to control both the pendulum rotation angle θ and the body position x simultaneously. However, first, we focus only on the pendulum rotation angle θ and consider controlling it so that $\theta = 0$, that is, simply inverting the pendulum. To do this, find $\ddot{x}$ from equation (10), substitute it into equation (9), and obtain a differential equation for θ only.

$$\left(j_p + m_p l^2 \left(1 - \frac{m_p}{M} \right) \right) \ddot{\theta} - m_p g l \theta = - \left(1 + \frac{m_p l}{M r} \right) \tau \quad (11)$$

where $M = m_p + m_w + \frac{j_w}{r^2}$. Since the coefficients are complex, we replace them simply as:

$$I \ddot{\theta} - m_p g l \theta = -A \tau \quad (12)$$

This equation represents the same situation as a pendulum with mass m_p and moment of inertia I around the axis fixed on a shaft, receiving torque $A\tau$ from a motor, as shown in Figure 3.

Let's consider the behavior of the pendulum in Figure 3. When there is no torque from the motor (= no control), equation (12) becomes $I \ddot{\theta} = m_p g l \theta$. Formally, this is the same as the spring equation, so let's compare it with the spring-mass system in Figure 4. The equation of motion for the spring-mass system in Figure 4 is $m \ddot{x} = -kx$. Comparing this with the previous equation, we can see that **the inverted pendulum is equivalent to a spring-mass system with a spring constant of $-m_p g l$** . However, since the spring constant ($-m_p g l$) is negative, it performs an unstable operation exactly opposite to a spring (if it moves slightly away from the equilibrium point, a force acts to move it further away).

Thus, the inverted pendulum can be interpreted as unstable because it has a negative spring constant. Therefore, let's consider attaching a spring between it and the vertical axis as shown in Figure 5 so that the total combined spring constant (= original negative spring constant + newly attached positive spring constant) becomes positive. If the combined spring constant becomes positive, the pendulum should perform simple harmonic motion near the vertical axis. Furthermore, if a damper (= dashpot: produces a reaction force proportional to velocity) is also attached, the oscillation can be damped, and the inverted pendulum can be made to stand along the vertical axis. However, since we cannot attach a real spring and damper, let's realize the function equivalent to a spring and damper by feedback control.

2.3 PD Control

We consider a method to virtually realize a spring and damper by feedback control. Since the total force of the spring and damper can be expressed as $K_p \theta + K_d \dot{\theta}$, if the pendulum angle θ (and its derivative $\dot{\theta}$) is known, we can make the motor generate torque such that:

$$A \tau = K_p \theta + K_d \dot{\theta} \quad (13)$$

This allows the motor to play the role of a spring + damper.

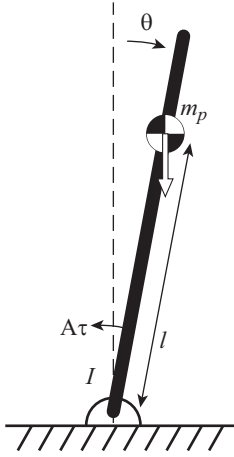


Figure 3: Equivalent Pendulum

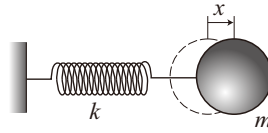


Figure 4: Spring-Mass System

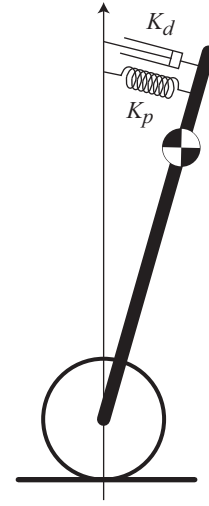


Figure 5: Stabilization

As in this example, a control method that feeds back an amount *proportional* to the position or angle (error from the target) and an amount *proportional to the derivative* of the position or angle is called **“PD control”**. P stands for *Proportional* and D stands for *Derivative*. When only one of them is given, they are called P control (proportional control) and D control (derivative control), respectively. As understood from the above, P control acts equivalently to a spring, and D control acts equivalently to a damper. Also, the respective coefficients (K_p , K_d) are called P gain (proportional gain) and D gain (derivative gain).

The equation of motion of the PD-controlled pendulum can be expressed from equations (12) and (13) as:

$$I\ddot{\theta} = -(K_p - m_p g l)\theta - K_d \dot{\theta} = -K\theta - K_d \dot{\theta} \quad (14)$$

Here, $K = K_p - m_p g l$ is the spring constant of the combined spring of “negative spring due to gravity” and “positive spring due to control”. From this equation, it can be seen that the PD-controlled inverted pendulum moves in the same way as a spring-mass-damper system with spring constant K and damping coefficient K_d . That is, if an angle deviated from the equilibrium point ($\theta = 0$) is given as an initial value, it converges to the equilibrium point $\theta = 0$ while performing damped oscillation. Since it is the same as a normal spring-mass-damper system, increasing the spring constant ($\approx$ proportional gain) increases the oscillation frequency, and increasing the derivative gain increases damping, causing the oscillation to decay quickly.

2.4 Is this enough to invert?

If you are lucky, controlling the motor torque based on equation (13) alone will keep the inverted pendulum stable. However, in reality, there are many cases where this alone cannot invert it (it inverts for a short time but does not last). The causes are offsets in the inclination sensor that measures the inclination angle θ , and disturbances (forces applied from outside the system, such as wind or desk vibration).

For example, if there is even a slight offset (= deviation in output value) in the inclination sensor, even if it looks like $\theta = 0$, the inclination corresponding to the offset actually remains, so the inverted pendulum falls due to gravity. Then, trying to pull up the inverted pendulum, the control system increases the torque of the motor (causing the inverted pendulum to start running). To prevent it from falling, it must continue to run at a constant acceleration, but the motor speed has a limit. When the motor reaches its speed limit, it can no longer accelerate, and the inverted pendulum falls.

To prevent this, it is necessary not only to control by looking at the angle but also to detect the position of the inverted pendulum ($\propto$ amount of tire rotation) and add PD control for the positional deviation. We will discuss this point again later.

2.5 Motor Torque

In order to perform control based on equation (13), there is one more thing to consider. It is about the motor torque. When we drive a motor, we do so by applying a voltage across the motor terminals. If this voltage is changed, the generated torque of the motor also changes, but what is the relationship between voltage and generated torque?

Figure 6 is an equivalent circuit model of a DC motor. Inside the motor, there is a winding for generating a magnetic field, which has resistance R and inductance L . Also, since the motor has the same action as a generator, when the motor rotates, a voltage proportional to the rotation speed called *counter electromotive force* (counter EMF) is generated in the opposite direction to the applied voltage.

Since the motor torque is proportional to the strength of the magnetic field generated by the winding, and the strength of the magnetic field is proportional to the current flowing through the winding, the generated torque is proportional to the current flowing through the winding ($\tau = \kappa i$). This proportionality coefficient κ is called the "torque constant" of the motor.

In other words, while what we determine and apply to the motor is **voltage**, what is proportional to torque is **current**. If there is a proportional relationship between the two, the story is simple, but in reality, due to the influence of back EMF and coil impedance, the two are not simply proportional. So, let's find out the relationship between the two.

From the equivalent circuit in Figure 6, deriving the relationship equation between current (Laplace transformed as $I(s)$) and applied voltage (similarly $V(s)$), we get:

$$I(s) = \frac{V(s) - \kappa \Omega_m(s)}{R + sL} \quad (15)$$

Here, κ is the torque constant (= back EMF constant)¹, $\Omega_m(s)$ is the Laplace transform of the motor angular velocity ω_m , and sL is the impedance of inductance L (expressed by Laplace transform).

In the above equation, if sL is sufficiently small compared to R , the inductance can be ignored. In fact, since L can be considered sufficiently small in the DC motor used this time, we will ignore the inductance. In this case, the motor torque can be simply expressed as (without Laplace transform since transient response does not need to be considered):

$$\tau = \kappa \frac{v - \kappa \omega_m}{R} \quad (16)$$

As can be seen from this equation, to generate torque correctly, it is necessary to find the angular velocity of the motor (ω_m) and add the back EMF component ($\kappa \omega_m$) to the applied voltage. This requires a sensor to measure the rotation speed, but until we can actually measure the rotation speed, we will proceed by ignoring the existence of back EMF. If the existence of back EMF is ignored, the applied voltage and current, and further, the motor torque will be proportional. Therefore, if the motor torque is calculated according to equation (13) and a voltage proportional to it is applied to the motor, the inverted pendulum should stand (at least for a short time).

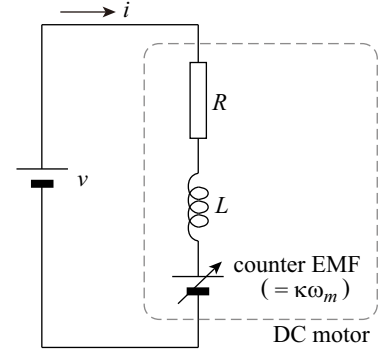


Figure 6: DC Motor Model

¹ The proportionality coefficient between back EMF and speed is called "back EMF constant," but when units are unified to the SI system, the back EMF constant and torque constant become the same value. This can be confirmed by calculating considering energy conservation.

3 Week 1: Assembly & Bring-up

3.1 Week 1 Overview & Objectives

Week 1 focuses on building and bringing up the hardware platform for the inverted pendulum. The work proceeds in two parts: (1) three **parallel tasks** that can be done simultaneously by different students, and (2) a **team bring-up** process where the circuit is assembled and verified step-by-step.

Objectives:

- Complete the mechanical assembly (sensor, motor, breadboard, battery box)
- Set up the microcontroller environment and verify toolchain / serial debugging
- Design the full breadboard wiring in Fritzing before touching the real breadboard
- Assemble and verify the circuit in three stages (motor → angle sensor → full integration)
- Record evidence (photos, Fritzing files, and bring-up results) for submission

Parallel tasks (recommended roles): Student A (mechanical), Student B (microcontroller), Student C (Fritzing/wiring design). Roles can be rotated, but all students should participate in the stage-by-stage bring-up and verification.

3.2 Electronic Components Explanation

Next, we will assemble the electronic circuit, but before that, we will explain the parts to be used. The specific circuit assembly work is described later in this Week 1 section, so please read the explanation first.

IMPORTANT: Caution when handling electronic circuits

- Semiconductor parts (ICs like microcontrollers, LSIs) are easily destroyed by static electricity (especially in dry winter). Be careful not to touch the pins with bare hands.
- *Your body is charged!* Before assembling the electronic circuit, touch a large conductor (such as a door or metal part of a desk) to discharge your body.
- Once discharged, be careful not to get charged during work. Moving your hips on a chair or taking off a sweater generates static electricity, so try to stay still as much as possible.
- Even if discharged, do not touch the IC/LSI terminals directly with your fingers just in case.

3.2.1 Resistor and Capacitor Markings (Value & Polarity)

■ Resistors: how to identify the resistance value

You can confirm a resistor value either by measuring it with a multimeter (set to the Ω range), or by reading the color bands. For accurate measurement, it is best to measure the resistor *out of circuit* (or at least lift one lead), because in-circuit measurements can appear lower due to parallel paths.

4-band color code (most common): 1st band = 1st digit, 2nd band = 2nd digit, 3rd band = multiplier, 4th band = tolerance.

Digit colors: Black=0, Brown=1, Red=2, Orange=3, Yellow=4, Green=5, Blue=6, Violet=7, Gray=8, White=9.

- **Example (100 Ω , 4-band):** Brown–Black–Brown with a tolerance band typically Gold ($\pm 5\%$) or Brown ($\pm 1\%$). Here, Brown=1 and Black=0, so the first two bands give “10”; the third band Brown is the multiplier $\times 10^1$, so $10 \times 10 = 100 \Omega$.

Some precision resistors use a 5-band code; if you see five bands, use the same idea (more significant digits + multiplier + tolerance).

Numeric markings (often on SMD resistors): resistors may also be printed with digits, which is similar in spirit to the capacitor 3-digit code, but the **unit is Ω** . A common 3-digit resistor code is: first two digits = significant figures, third digit = number of zeros ($\times 10^n$) in Ω . For example, **101** means $10 \times 10^1 = 100 \Omega$, and **102** means $10 \times 10^2 = 1 \text{ k}\Omega$.

■ Capacitors: how to read the capacitance value and polarity

Many small bypass capacitors are non-polarized (e.g., ceramic/film) and can be inserted either way, while electrolytic/tantalum capacitors are polarized and must be oriented correctly.

- **Typical markings:** some capacitors print the value directly (e.g., 0.1 μ , 100n), while many small ceramic capacitors use a 3-digit code in pF (first two digits are significant figures, the third digit is the number of zeros). For example, **104** means $10 \times 10^4 \text{ pF} = 100 \text{ nF} = 0.1 \mu\text{F}$.
- **Polarized vs. non-polarized:** ceramic/film capacitors have **no polarity**; aluminum electrolytic and tantalum capacitors are **polarized**. For typical aluminum electrolytics, the stripe on the body indicates the **negative** side (and the longer lead is often the **positive** side). PCB symbols may also mark +.

3.2.2 Motor Driver (DRV8832)

In the above example, we expect to drive the motor with the voltage output to pin p18 on line 7, but unfortunately, the output from mbed can only flow a small current, so the motor cannot be driven by connecting it directly to the pin. Therefore, the motor is driven via a motor driver IC capable of flowing a large current.

The driver used this time is Texas Instruments' DRV8832. Since it is a small IC and difficult to use, we use one mounted on a board (AE-DRV8832: sold at Akizuki Denshi in Akihabara). The board has 3 input pins, 2 of which determine the motor drive direction (sign of voltage) (IN1, IN2), and the remaining 1 determines the applied voltage (absolute value) (VSET). We connect 2 digital signal outputs of mbed directly to the pins determining the drive direction. On the other hand, we connect the analog signal output of mbed to the pin instructing the applied voltage. There is also a pin for current limiting (ISENSE, pin 2), but since we do not limit the current this time, connect it directly to GND (0V, reference voltage of the circuit. Minus side of the battery in this case).

The two wires coming out of the motor will be connected to pins 1 and 3 of the driver, but the connection between the motor and the driver will be frequently attached and detached during future experiments. However, since the motor wires are soft and difficult to insert into and remove from the breadboard, do not connect the motor wires directly to the driver, but connect them via separate jumper wires. You can disconnect the motor and driver by inserting/removing jumper wires (leave the motor wires that are difficult to insert/remove plugged into the breadboard).

For other details regarding the motor driver, refer to the circuit example¹ and the datasheet².

3.2.3 Photo-interrupter & Inclination Sensor

■ Photo-interrupter

In the inverted pendulum of this exercise, all sensors (= inclination sensor, rotary encoder. However, the rotary encoder will be described later) are realized with photo-interrupters. Photo-interrupters can be obtained very cheaply, and the ones used this time are sold for about 40 yen each in Akihabara. For the convenience of the exercise, an inclination sensor using photo-interrupters is prepared in advance, but the rotary encoder described later will be made by each of you.

A photo-interrupter is an optical element combining an LED and a phototransistor. In many cases, wavelengths in the infrared region are used, so the light cannot be seen directly. As shown in Figure 7, there are reflective and transmissive types of photo-interrupters, which detect the presence of an object by the presence or absence of reflected light or transmitted light, respectively. Especially in the case of the reflective type, since the amount of reflected light changes depending on the reflectance ($\approx$ color) of the object surface and the distance to the object (see Fig.7 of the datasheet), it can also be used as a color sensor (discriminating gray scale) or a distance sensor. This time, we apply this function as a distance sensor to realize an inclination sensor. (In the rotary encoder described later, the photo-interrupter reads the black and white pattern on the scale surface.)

The basic circuit of a photo-interrupter is shown in Figure 8. First, the LED must be lit. Looking at the datasheet³, the forward voltage V_F (1.2V) and the typical current value for lighting (20mA) are displayed. The

¹ Appendix D: Circuit example.

² Appendix E: DRV8832 datasheet (Texas Instruments).

³ Appendix F: TPR-105F datasheet.

forward voltage is the voltage drop across the LED during operation. This time, since the 3.3V of the three-terminal regulator described later is used as the power supply voltage (VCC), if the forward voltage is 1.2V, the difference of 2.1V must be dropped by a resistor connected in series. To drop 2.1V at a current value of 20mA, Ohm's law tells us that $R = 2.1V/20mA = 105\Omega$ is good, but since a 105 Ω resistor is not sold, we use a close value of 100 Ω . See Section 3.2.1 for how to identify resistor values by color code or multimeter.

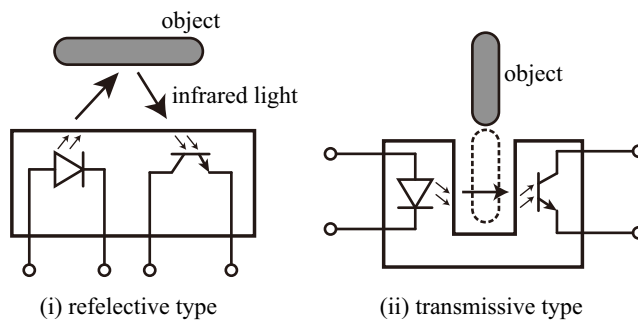


Figure 7: Photo-interrupter types

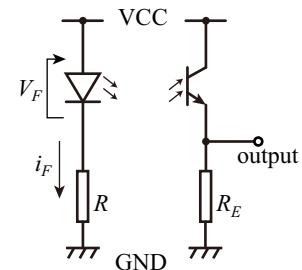


Figure 8: Circuit Diagram

Note that if a current exceeding the absolute maximum rating (50mA this time) flows, the LED will burn out in an instant. Be careful never to connect directly to the power supply without a resistor (The distributed inclination sensor has a 100 Ω resistor soldered to the tip of the wiring from the beginning so that this does not happen).

On the side of the phototransistor receiving light, connect a resistor to the emitter to read the voltage (a resistor can be attached to the collector side, but the output voltage is inverted). When light enters the phototransistor, current flows between the collector and emitter. This current is converted to voltage by flowing through the external resistor. The obtained voltage is roughly proportional to the amount of incident light. The value of the external resistor is determined while looking at the datasheet and actual situation. For the same amount of incident light, a larger resistance value yields a larger voltage, but since output cannot exceed the power supply voltage, set an appropriate resistance value so that the output does not saturate.

Since the LED of the photo-interrupter is infrared light and cannot be seen visually, you may feel anxious about whether it is really working. In such a case, let's look at the LED with a smartphone or computer camera. You should see the LED glowing purple. However, some smartphones have an infrared cut filter in the camera part, so the light of the infrared LED cannot be seen. In that case, try the front camera as well. Front cameras often do not cut infrared rays. If you want to know if your smartphone camera has an infrared cut filter, look at the light emitting part of a TV remote control with the camera. If no filter is inserted, you should see a fierce brightness when pressing the remote button (compared to that, the photo-interrupter used this time is much dimmer).

■ Inclination Sensor

Gyro sensors are often used to measure the inclination angle of inverted pendulums, but gyro sensors measure angular velocity and need to be integrated to find the angle (this involves drift problems). This time, we use photo-interrupters to measure the angle more directly.

An inclination sensor should already be attached to the bottom of the inverted pendulum. This inclination sensor consists of two photo-interrupters. The photo-interrupters at the front and rear of the machine measure the distance (reflected light amount) to the floor surface, respectively, and output the difference in this distance as an angle.

It measures the angle with respect to the floor surface, not the angle with respect to the vertical plane. Therefore, if the inclination of the floor changes, an error occurs in the sensor output (if considering the vertical plane as the reference).

Also, if the reflectance of the floor surface changes, the sensitivity of the sensor changes. On a floor surface with high reflectance (bright), the inclination will likely be output as large. However, brightness here is brightness in the infrared region, so it does not necessarily match the visual (visible light region) brightness.

3.2.4 Three-terminal Regulator

This time, batteries are used as the power source for the entire inverted pendulum, but if a large current is drawn from the battery, the generated voltage drops due to internal resistance. In this circuit, a large current flows along with the motor drive, so the output voltage of the battery fluctuates violently according to the motor operation. If the photo-interrupter is driven using this fluctuating voltage, large noise will be superimposed on the sensor output, so the voltage for driving the photo-interrupter is stabilized using a three-terminal regulator.

A three-terminal regulator is an IC that takes a fluctuating voltage as input and outputs a constant voltage (lower than that). Various model numbers of three-terminal regulators are sold, but basically, the last two digits of the model number represent the output voltage. This time, we use a regulator with model number LD33 (precisely LD1117V33)¹. The output voltage is 3.3V. All photo-interrupter related voltages should use the output of this three-terminal regulator. On the other hand, since we want to separate the power supply for the motor and the photo-interrupter, *take the power supply for the motor driver directly from the battery, not from the three-terminal regulator.*

Actually, mbed's board is also equipped with a 3.3V three-terminal regulator (you can see a chip called LD33), and stabilized 3.3V is output from the VOUT pin on the upper right (see Figure 13). You can use this voltage, but due to the size of the breadboard, this pin cannot be accessed, so we will install a regulator ourselves this time.

3.3 Parallel Assembly Tasks

The following three tasks can be performed in parallel. Each task produces outputs that will be used in the team circuit assembly and bring-up.

3.3.1 Task A: Mechanical Assembly (Student A)

Now that you understand the outline of control, let's look at the assembly of the inverted pendulum (**Actual assembly will be done on the day of the exercise, but please read through it in advance**).² The parts used here are as follows³ (electrical parts are excluded as they are explained separately in this Week 1 section).

1. Universal board (Tamiya, partially processed) + Wheels (Tamiya)
2. Universal board (Tamiya) + Breadboard for circuits + mbed LPC1768
3. DC Motor (Mabuchi Motor RE-280RA)⁴ (Rubber roller attached to the tip)
4. Motor bracket
5. Battery box (Rechargeable AA batteries x 4)
6. Inclination sensor (Unique part of this exercise)
7. L-bracket

The universal board with wheels (No. 1) will be the center of assembly. Below, parts will be attached to this board.

■ Attaching the Inclination Sensor

First, attach the inclination sensor (No. 6) to the bottom of the universal board with wheels (No. 1). Use the L-bracket (No. 7) to attach it as shown in Figure 9.

■ Attaching the DC Motor

Next, attach the DC motor (No. 3). Use the motor bracket (No. 4) to attach it to the universal board (No. 1) (attach it to the same surface as the axle. See Figure 10). The screws for attaching the bracket to the board are

¹ Appendix G: LD1117V33 datasheet.

² A video demonstration of the mechanical assembly process is available in the course materials repository: `course_materials/video/1_mechanical_assembly.mp4`

³ For a complete parts list with detailed specifications and quantities, see Appendix A.

⁴ Appendix B: RE-280RA datasheet (Mabuchi Motor).

slightly smaller M2 screws. The screw heads are small, so it might be difficult to turn them with the screwdriver included in the kit, but try to secure them.

When fixing the DC motor, adjust it so that the rubber roller of the DC motor touches the tire "lightly". **The point is to adjust it so that it is *touching lightly*, as driving will not be possible if pressed too strongly.** However, since the tire is not a perfect circle, if adjusted too lightly, the roller may slip partially when the tire is rotated. Rotate the tire once to confirm that the roller is always in contact.

■ Attaching the Breadboard

Screw the No. 2 universal board with the breadboard onto the No. 1 universal board. Bolts should be protruding from this No. 2 board. Do not remove these bolts as it is difficult to reattach them (of course, remove/attach the "nuts" during installation). The No. 1 board should have screws for motor attachment, so make sure they do not interfere (overlap) (if forced to screw while overlapping, the board will crack). Also, since the battery box will be attached in the next part, decide the position of the board while considering the mounting position of the battery box.

The microcontroller should already be attached to the breadboard. Attach the universal board so that the microcontroller is on the upper side (USB port facing up). ***Do not remove the microcontroller on the breadboard, as the pins may break.***

■ Attaching the Battery Box

Finally, attach the battery box (No. 5) to the universal board (No. 1) with double-sided tape. Since the battery box is the heaviest part, the characteristics of the inverted pendulum change depending on the mounting position. You can stick it anywhere there is space, but be careful about the front and back of the battery box (if you put double-sided tape on the lid side, the lid will not open). Consider how to stick it so that you can access the switch on the back of the battery box.

Note that double-sided tape is surprisingly strong. If you stick a large amount, you won't be able to peel it off. Two strips of about 5 cm length on the top and bottom are sufficient.

Up to this point, the basic mechanical structure is complete. A completed example is shown in Figure 11.

3.3.2 Task B: Microcontroller Setup (Student B)

The microcontroller used this time is a series called mbed¹. The feature of mbed is that the development environment using C/C++ language is prepared on the cloud (online), and you can start development immediately by signing up. Also, writing a program to the microcontroller is as simple as connecting the mbed via USB (recognized as a USB drive by the computer) and copying the compiled program, making it easier to handle compared to other microcontrollers. There are various microcontroller boards in the mbed series, but this time

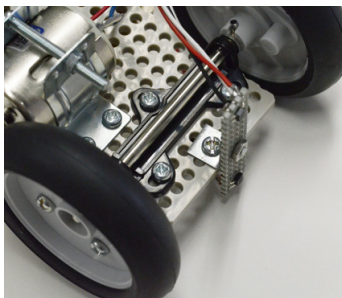


Figure 9: Attaching sensor



Figure 10: Attaching motor

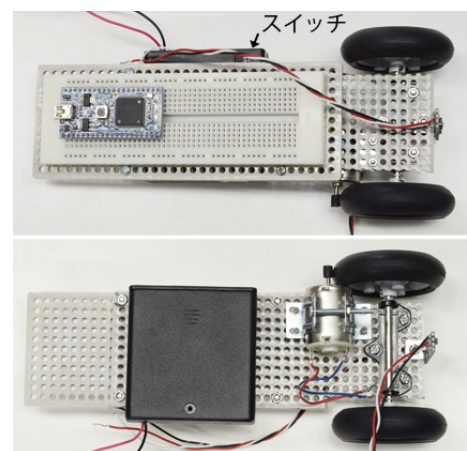


Figure 11: Completion image

¹ There are various other microcontroller series such as Arduino, Raspberry PI, PIC, etc.

we will use the most standard LPC1768.

■ Keil Studio Cloud signup

First, access the Keil Studio Cloud¹ and create an account. After signing up, connect the microcontroller to your computer with the included USB cable. It will be recognized as a USB drive. In the upper left of the compiler screen, "mbed LPC1768" should be selected as the microcontroller board (Build Target) to be used.

■ Compile a sample program

In the compiler screen Project menu, select "File" → "New Project" and create a new program using mbed_blinky as a template (see Figure 12). Specifically, select "Mbed2-example-blinky" at the bottom of Example Project, and set the Project Name as you like. A new program with code to blink an LED will be created, so "Compile" it as is. Compilation can be done by clicking the icon that looks like a hammer. When compilation is finished, an executable file (extension .bin) is downloaded, so write this to the USB drive (mbed). Just drag and drop it perfectly.

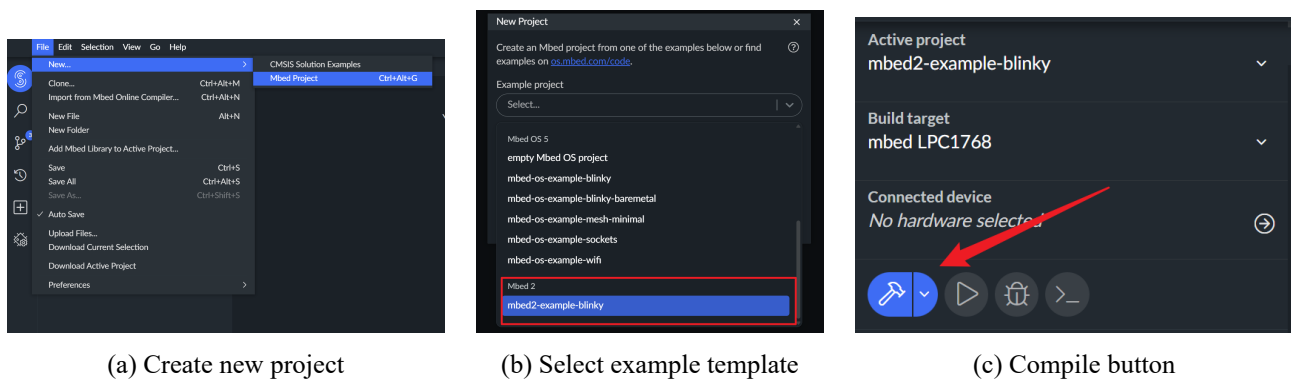


Figure 12: Keil Studio Cloud interface: creating and compiling a project

After writing the program, press the button in the center of the microcontroller board (this is the reset switch). Then, the newest file in the USB drive (= the file written last) is executed.

In the sample program, if the left LED among the 4 LEDs lined up at the bottom of the mbed board blinks, it is successful. If successful, try rewriting the program (main.cpp file) a little, such as changing the LED blinking time, and practice the flow of Compile → Run.

* Mac users may not work well with the above. In that case, refer to the file "Write Error and Countermeasures on macOS" and execute.

■ Pin usage explanation

The microcontroller board used this time has various input/output functions. For example, 26 channels of digital signal I/O, 1 channel of analog voltage output by built-in DA converter, and 6 channels of voltage input by AD converter are available. Besides this, there are 6 channels of PWM (Pulse Width Modulation) signal output, etc., but assigning all these functions to individual pins would require a huge number of pins and the board would become huge.

Generally, these functions are not all used at the same time, so in this board (and microcontrollers in general), multiple functions are assigned to one pin, and the function of the pin is selected from the program as needed. This function assignment is shown in Figure 13.

The pins available for user signal I/O are the 26 pins labeled "pXX" (XX is a number) in the outermost columns on the left and right. All these pins can be used for digital signal input or output. Also, by specifying from the program, functions written next to the pin name can be used. For example, looking at pin p18 at the bottom left, you can see that it has AnalogIn and AnalogOut functions. That is, a total of 4 functions: digital signal input, digital signal output, analog signal input, and analog signal output are assigned to this pin. Among these 4 functions, if you want to use the analog signal output function, in the program:

¹ Keil Studio Cloud: <https://studio.keil.arm.com/>

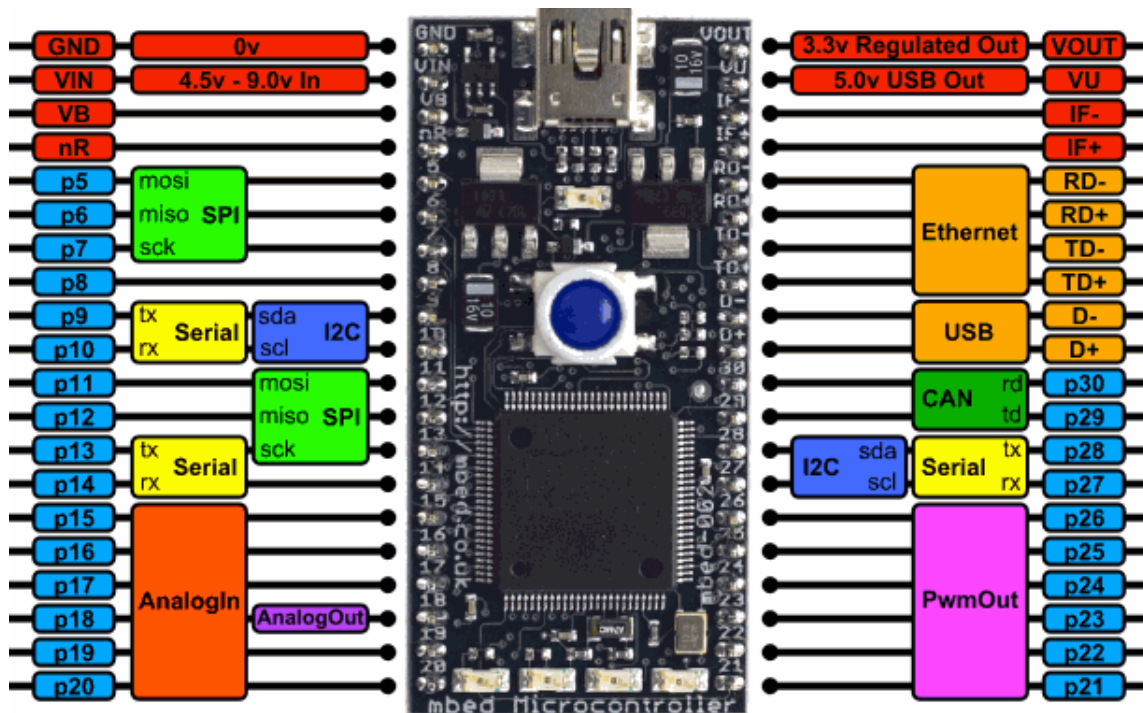


Figure 13: Pinout of mbed LPC1768 (<https://os.mbed.com/platforms/mbed-LPC1768/>)

```
AnalogOut aaa(p18);
```

By declaring this, AnalogOut is selected as the function of p18. The declared object (aaa above) can be written to (or read from in the case of AnalogIn) like a variable. For example,

```
aaa = 0.5;
```

If you do this, an analog voltage of 0.5 times the full scale (3.3V) (i.e., 1.65V) is output to p18. Conversely, when setting this pin as analog input,

```
AnalogIn aaa(p18);
... (omitted)
b = aaa;
```

Like this, using the declared object `aaa`, you can read the voltage value input to p18 (ratio to full scale = 3.3V) (in this example, the value enters variable `b`).

For details, refer to the reference manual on the following page.

<https://os.mbed.com/handbook/Homepage>

■ Control program basics

In a feedback control program, the following processes are repeated.

- Reading sensor signals
- Control calculation
- Output of calculation result

Although 'for' loops and 'while' loops can be used for repetition, the repetition time is not known accurately, and (if there are conditional branches such as 'if' inside the loop) the repetition cycle changes depending on the processing content.

In feedback control and digital signal processing, it is important that the repetition time (sampling time) is fixed and known. Therefore, generally, a timer is used to manage time. As specific programming methods

using a timer, there are methods using timer interrupts and methods using threads, but this time we use timer interrupts. A simple program example is shown below (numbers on the left are line numbers for explanation).

```
-----
1:  AnalogIn ad(p20);
2:  AnalogOut da(p18);
3:
4:  void int0() {
5:      theta = ad;           // Reading sensor signal
6:      command = theta * Kp; // Control calculation
7:      da = command;         // Output
8:  }
9:
10: int main() {
11:     t_int.attach(&int0, 0.001); // Start timer interrupt
12:     while(1);                  // Infinite loop as nothing else to do
13: }
```

Here, taking the case of P control as an example, the main part of the program is shown. The C language program is executed from the main function on line 10, but the main function is done after setting the timer interrupt on line 11 (setting here calls function ‘int0()’ every 0.001 seconds). When the main function ends, the execution of the entire program also ends, so it enters an infinite loop so as not to end (line 12).

On the other hand, the function ‘int0()’ from line 4 specified for the timer interrupt processing is the main body of the control program. In this example, line 5 reads the sensor signal input from outside the microcontroller, line 6 calculates P control, and line 7 outputs voltage outside the microcontroller. Every time a signal comes from the timer, this function is executed, and as a result (in this example) repetitive processing with a 1ms period is realized.

The interrupt processing function must definitely finish within the interrupt period (1ms in the above example). If processing does not finish within the interrupt period, the next interrupt enters, and processing breaks down. Therefore, it is an iron rule to avoid time-consuming tasks within the interrupt processing function. Especially requiring caution is function calls.

Function calls can be written in one line from the perspective of writing a program, so we tend to imagine that the execution time is short, but some functions have unexpectedly long processing times. Easy calling of such functions from within an interrupt processing function causes the interrupt processing to break down. Specifically, string processing functions such as ‘sprintf()’, memory processing functions such as ‘malloc()’, and mathematical functions such as ‘sin()’ have long execution times and tend to cause trouble.

Also, *since function calls themselves have large processing overhead, it is safer not to define and call functions yourself excessively*. Although there is an idea to group multiple processes into a function to improve program readability, in such cases, it is better to use inline functions or macro definitions if possible (these have no overhead at the time of calling).

■ Serial communication & debugging

Serial communication (UART) is one of the most important debugging tools when developing microcontroller programs. The mbed LPC1768 has built-in USB-to-Serial functionality, which allows you to send debug messages from your program to your computer. This is particularly useful when debugging control programs, as you can monitor variable values and program behavior in real time without interrupting the control loop.

UART Communication Principle: UART (Universal Asynchronous Receiver-Transmitter) is a hardware communication protocol that transmits data one bit at a time over a single wire. Unlike synchronous protocols that require a shared clock signal, UART is asynchronous—both the transmitter and receiver must agree on the communication speed (baud rate) beforehand. Each byte of data is transmitted as a frame consisting of: (1)

a start bit (logic low), (2) 8 data bits (LSB first), (3) an optional parity bit for error checking, and (4) one or more stop bits (logic high). The mbed LPC1768 includes a built-in USB-to-Serial converter chip that bridges the UART interface (USBTX/USBRX pins) to USB, allowing seamless communication with your computer without external hardware.

To use serial communication in your program, you need to create a Serial object using pins USBTX and USBRX, which are connected to the USB interface. A simple example would be:

```
Serial pc(USBTX, USBRX);

int main() {
    pc.baud(9600); // Set baud rate to 9600 bps
    pc.printf("Hello, World!\n");

    while(1) {
        pc.printf("Counter: %d\n", counter);
        wait(1.0);
    }
}
```

The baud rate determines the communication speed and must match the setting in your terminal software. Common values are 9600 bps (recommended for beginners due to its reliability) or 115200 bps (faster, commonly used for debugging). Other standard rates include 4800, 19200, 38400, and 57600 bps.

When the mbed is connected via USB, it appears as a serial port on your computer (typically COM3 or similar on Windows, `/dev/tty.usbmodem*` on macOS, `/dev/ttyACM*` on Linux). There are several ways to view these serial messages. The most convenient option is to use the built-in serial monitor in Keil Studio Cloud, which allows you to view serial output directly in the web browser without any additional software. Alternatively, you can use traditional terminal software such as Tera Term or PuTTY on Windows, or the screen command and CoolTerm on macOS/Linux. The Arduino IDE Serial Monitor also works well on all platforms.

Custom Debugging Tools: We provide three specialized debugging tools in the `debug_tools/` directory¹, each consisting of three files:

- **.cpp file:** mbed microcontroller program (C++ source code). You can read and modify this code to understand how the program works or customize it for your needs.
- **.bin file:** Pre-compiled binary executable ready to flash directly to mbed LPC1768. Simply drag-and-drop this file onto the mbed USB drive to program the microcontroller—no compilation needed.
- **.pde file:** Processing² visualization program that runs on your computer. This provides a graphical interface to display real-time data received from mbed via serial communication.

To use these tools: (1) Flash the `.bin` file to mbed by copying it to the mbed USB drive, (2) Press the reset button on mbed, (3) Run the corresponding `.pde` file in Processing. If you want to modify the mbed program, you can edit the `.cpp` file in Keil Studio Cloud, compile it to generate a new `.bin` file, and flash that to mbed. The tools use frame-based serial protocols (with header bytes 0xAA 0x55) to reliably transmit multiple data values per frame, ensuring proper synchronization between mbed and the PC visualization. See the `README.md` file in the `debug_tools/` directory for detailed documentation on each tool, pin assignments, communication protocols, and troubleshooting tips.

Serial communication is invaluable for debugging control programs. You can print variable values to verify calculations, monitor control loop timing by printing timestamps, check sensor readings by displaying raw data, and verify program flow by adding messages at key execution points. For example:

```
pc.printf("Angle: %f, Motor: %f\n", theta, motor_output);
```

¹ Access the course materials repository at <https://github.com/UTokyo2026/UTokyo-Control-Practice-2026>

² Processing is a free, open-source software for visual programming. You can download it from <https://processing.org/>. As of 2026, the currently available major release is Processing 4. For more instructions, see Appendix C.

However, be cautious when using `printf` inside interrupt service routines, as it can take significant time and may disrupt timing-critical control loops. For time-critical debugging, consider toggling a digital output pin and observing it with an oscilloscope instead. For more details on Serial communication, refer to the mbed Serial API documentation.

3.3.3 Task C: Breadboard Wiring Design (Student C)

■ How to use Breadboard

A breadboard is a board often used for prototyping electrical circuits, allowing simple circuit construction without soldering. Figure 14 shows the outline of a breadboard. In a breadboard, 5 holes lined up horizontally are electrically connected, and wiring is done utilizing this. The 5 holes on the right and left sides of the board are independent of each other.

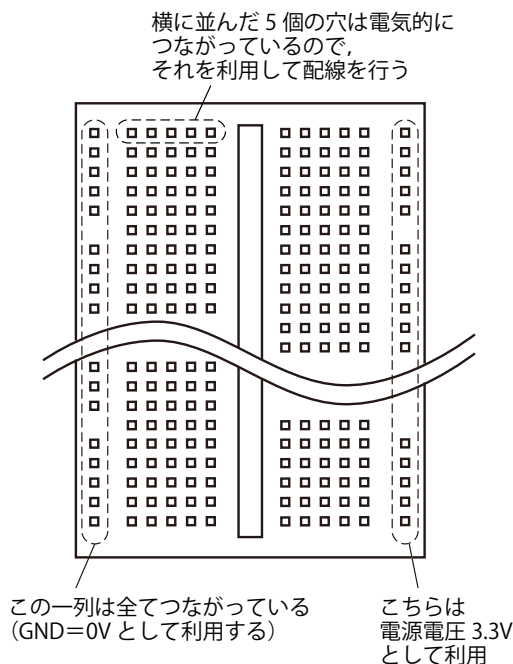


Figure 14: Breadboard

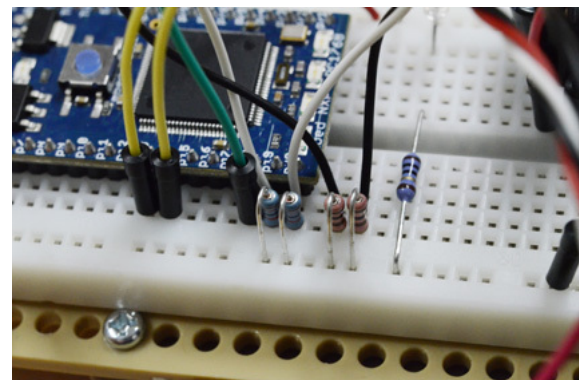


Figure 15: Wiring Example

For example, mbed should already be inserted on the breadboard. If you want to connect something to a pin of mbed, just insert a wire or leg of a resistor into the hole immediately next to that pin (see Figure 15).

Also, lines for power supply voltage are prepared on the breadboard. The holes lined up vertically at both ends of the board are electrically connected in one vertical column respectively. There is one column on the right and left, so use them as GND and 3.3V respectively. For this purpose, connect the output pin (3.3V) of the three-terminal regulator to some hole in the right column (any is fine) with a wire. Similarly, connect the left column to GND (black wire coming from the battery).

■ Using Fritzing for Circuit Design

Because breadboard wiring can become visually complex and error-prone, we recommend drafting your circuit with Fritzing¹ before building it on a real breadboard. Fritzing is a free and open-source electronic design automation (EDA) tool designed for breadboard-based prototyping, and it provides synchronized **Breadboard**, **Schematic**, and **PCB** views.

For this experiment, we provide custom Fritzing parts for the *Microcontroller*, *Motor Driver*, and *Photo Reflector* modules. These parts must be **imported into Fritzing first** and then placed from the parts library.

Figure 16 shows example circuit views for this experiment. Before placing parts and routing wires, open the

¹Fritzing installers (shared Google Drive): <https://drive.google.com/drive/folders/1CwJ8srD090W6h0eP39BXLUZ0VY883Kn2?usp=sharing>

<https://drive.google.com/drive/folders/1CwJ8srD090W6h0eP39BXLUZ0VY883Kn2?usp=sharing>

View menu in Fritzing and uncheck *Align to Grid*; grid alignment is not convenient for the component placement and wiring in this setup. We also provide an example design where the **motor driver section is already wired** (Figures 16a and 16b). Use it as a reference, and see the Circuit Assembly subsection in Week 1 for the detailed parts list and step-by-step procedure. A full sample circuit is also given in the appendix¹ for reference whether you use Fritzing or not.

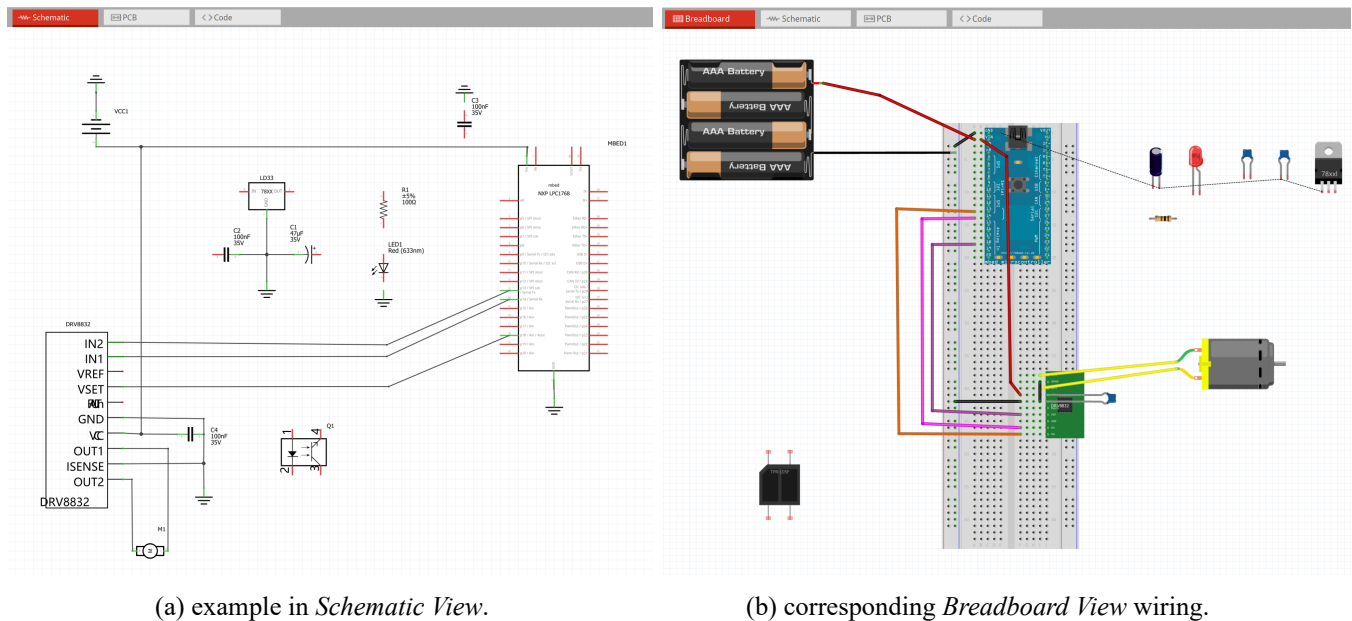


Figure 16: Fritzing example circuit views for this experiment.

When using Fritzing, start by importing all provided custom parts, then create a new sketch and place **all modules that will be mounted on your panel** (microcontroller, motor driver, photo reflector, regulator, battery connector, etc.).

Next, use Schematic View to confirm **which pins must be connected** and to catch missing connections, and then implement the same connectivity in Breadboard View. The physical placement can be different, but the electrical connections must match. When routing wires, prioritize readability: keep them as short as possible, avoid unnecessary crossings, and group wires by function (power, motor, sensors, encoder). This will make the real breadboard assembly faster and reduces debugging time.

Before building on the real breadboard, perform a final check on your Fritzing design by verifying every net related to **power (3.3V/GND)**, **motor driver inputs/outputs**, and **sensor signal lines**, and confirming that each signal is connected to the intended microcontroller pin. In particular, **complete the full panel circuit design in Fritzing first** (all modules placed and all connections routed and checked) **before you start assembling on the breadboard**.

Sync Point (All Students)

Before starting the team circuit assembly, confirm that:

- The mechanical platform is complete and the motor roller contact is correctly adjusted
- The microcontroller can be programmed and serial communication works
- The Fritzing design includes **all modules** and all required nets are routed and checked

3.4 Circuit Assembly (Team Collaboration)

This part is performed as a team. To minimize the risk of errors and facilitate debugging, we assemble and test the circuit in three stages.

¹ Appendix D: Circuit example.

3.4.1 Safety precautions

CAUTION: If you don't follow this, it will break

- Before turning on the power switch, check carefully that there are no mistakes in the wiring around the power supply at least!
- If there is a mistake in the wiring around the power supply (connecting plus and minus in reverse, short circuit, etc.), *the circuit will be destroyed at the same time as switch on.*
- It is dangerous to turn on the switch and see what happens, so check well in advance instead of taking a chance. Especially, pay attention to *power connection to microcontroller/motor driver* and *polarity of electrolytic capacitor.*
- Disconnect power before rewiring. Never move wires on a powered breadboard.
- Keep the motor power (battery) and sensor power (3.3V regulator) separated as instructed.

Soldering and continuity checks. During circuit assembly, you will use a soldering iron to solder resistors and DuPont wires (or leads) to make robust connections. Before powering the circuit, you can use a multimeter in continuity mode (buzzer) to verify.

CAUTION: Soldering iron safety

- Unplug the soldering iron after use, and let it cool down before storing it or leaving the bench.
- Never leave a powered soldering iron unattended.

3.4.2 Assembly strategy (3 stages)

To minimize the risk of errors and facilitate debugging, we recommend assembling and testing the circuit in three stages¹:

1. **Motor Driver and Motor Testing** - Verify motor control (forward/reverse rotation and speed control)
2. **Angle Sensor Testing** - Verify sensor signal acquisition and processing
3. **Complete Circuit Assembly** - Integrate all components for final control system

This staged approach allows you to identify and fix problems early, before the circuit becomes too complex. Each stage has a dedicated example program and debugging tool to help you verify correct operation.

3.4.3 Stage 1: Motor Driver & Motor Testing

■ Objective

In this first stage, you will assemble the power supply circuit, motor driver (DRV8832), and motor. The goal is to verify that you can control the motor's direction (forward and reverse) and speed using PWM signals from the mbed.

■ Circuit Components for Stage 1

- mbed LPC1768 microcontroller
- Battery box (4 AA batteries, 6V)
- Three-terminal regulator (LD1117V33) for 3.3V supply
- Motor driver IC (DRV8832)
- DC motor (RE-280RA)
- Bypass capacitors (ceramic capacitors for power stabilization)
- Electrolytic capacitor for regulator

¹ A video demonstration of the circuit assembly process is available in the course materials repository: [course_materials/video/2_circuit_assembly.mp4](#)

- Jumper wires

Note: For detailed part numbers, specifications, and quantities, refer to Appendix A.

■ Wiring Instructions

Wire the circuit according to the motor driver section of the sample circuit diagram¹. Key points to remember:

- In the sample circuit diagram, capacitors are inserted between the power pins of mbed and motor driver and GND. This is called a bypass capacitor (bypass cap), which has the role of softening power supply voltage fluctuations due to voltage drops in wiring (which also has resistance) and stabilizing IC operation. *It is an iron rule to install this bypass capacitor right next to the power pin (adjacent hole).* Even if it is the same on the circuit diagram, installing it far away makes the bypass capacitor meaningless.
- Similarly, *install the capacitor connected to the three-terminal regulator in the immediate hole.*
- Connect the three-terminal regulator output (3.3V) to one vertical power rail on the breadboard
- Connect GND (battery black wire) to the other vertical power rail
- Pay careful attention to *electrolytic capacitor polarity* (the stripe indicates negative side)
- See Section 3.2.1 for capacitor markings (e.g., 0.1u/104) and which capacitors are polarized vs. non-polarized.
- Double-check all power connections before turning on the power switch

■ Example Program and Debugging Tool

We provide an example program (`motor_test.cpp`) that demonstrates motor control. This program allows you to test:

- Motor forward rotation at various speeds
- Motor reverse rotation at various speeds
- Motor stopping and braking

To help you visualize the control signals and verify correct operation, we provide **Debugging Tool 1: Motor Controller** (located in `debug_tools/1_motor_controller/`). This tool consists of:

- **1_motor_controller.cpp**: mbed program that receives text commands via serial (115200 baud). Commands: F xx (forward at xx% duty), B xx (backward at xx%), S (stop). Uses pins p13 (motor forward), p14 (motor reverse), and p18 (PWM voltage).
- **1_motor_controller.bin**: Pre-compiled binary—drag and drop to mbed USB drive to program the microcontroller.
- **1_motor_controller.pde**: Processing GUI with sliders and buttons to interactively control motor speed and direction without modifying code.

The Processing GUI displays real-time motor status including PWM duty cycle, direction (forward/reverse), and motor current (if current sensing is implemented). This tool allows you to test the motor driver circuit thoroughly before integrating it with control algorithms.

■ Verification Checklist

Before proceeding to Stage 2, verify:

- ☐ Motor rotates forward when commanded
- ☐ Motor rotates backward when commanded
- ☐ Motor speed changes smoothly with PWM duty cycle
- ☐ Motor stops completely when commanded
- ☐ No abnormal heating of motor driver IC
- ☐ Power supply voltage remains stable (measure with multimeter)

¹ Appendix D: Circuit example.

3.4.4 Stage 2: Angle Sensor Testing

■ Objective

In this stage, you will add the angle sensor (photo-interrupter or rotary encoder) to your circuit and verify that you can correctly read the pendulum angle. This is the most critical sensor for inverted pendulum control.

■ Additional Components for Stage 2

- Photo-interrupter sensors (TPR-105F) $\times$ 2
- Pull-up resistors for sensor outputs
- Additional jumper wires for sensor connections

■ Wiring Instructions

Add the sensor circuit to your existing motor driver circuit according to the sensor section of the sample circuit diagram¹. Key points:

- Connect sensor power to 3.3V rail (not 5V, as mbed inputs are 3.3V logic)
- Install appropriate pull-up resistors on sensor output lines
- Route sensor signal wires away from motor power lines to minimize noise
- Ensure sensors are mechanically aligned with the encoder disk or slit pattern

■ Example Program and Debugging Tool

We provide an example program (`sensor_test.cpp`) that reads and processes sensor signals. This program demonstrates:

- Reading digital sensor outputs
- Calculating angle from sensor readings
- Computing angular velocity by differentiation
- Applying filtering to reduce noise

Debugging Tool 2: Analog Input Visualizer (located in `debug_tools/2_analog_input_visualizer/`) provides real-time visualization of 4 analog input channels:

- **2_analog_input_visualizer.cpp**: mbed program that reads 4 analog inputs at approximately 200 Hz and transmits data via serial (115200 baud) using a frame-based protocol (header: 0xAA 0x55, followed by 4 data bytes). Pins: p16 (CH1), p17 (CH2), p19 (CH3, typically photo-interrupter 1), p20 (CH4, typically photo-interrupter 2). The 4 channels can be used for any analog sensors: angle sensors, encoder signals, potentiometers, etc.
- **2_analog_input_visualizer.bin**: Pre-compiled binary ready to flash to mbed.
- **2_analog_input_visualizer.pde**: Processing program that displays 4-channel real-time waveforms. Can calculate pendulum angle and angular velocity when using photo-interrupters, or display encoder signals when using rotary encoders on CH1/CH2.

This versatile tool is essential for verifying sensor operation, detecting mechanical issues, and confirming signal quality. For angle sensors, you can manually move the pendulum and observe the sensor readings in real time. For encoders, you can rotate the wheel and observe the quadrature signals. The frame-based protocol ensures reliable data transmission by using header bytes to synchronize the data stream between mbed and PC visualization.

■ Verification Checklist

Before proceeding to Stage 3, verify:

- ☐ Sensor outputs change correctly as pendulum moves
- ☐ Angle calculation corresponds to actual pendulum position
- ☐ Angular velocity shows reasonable values (not excessively noisy)

¹Appendix D: Circuit example.

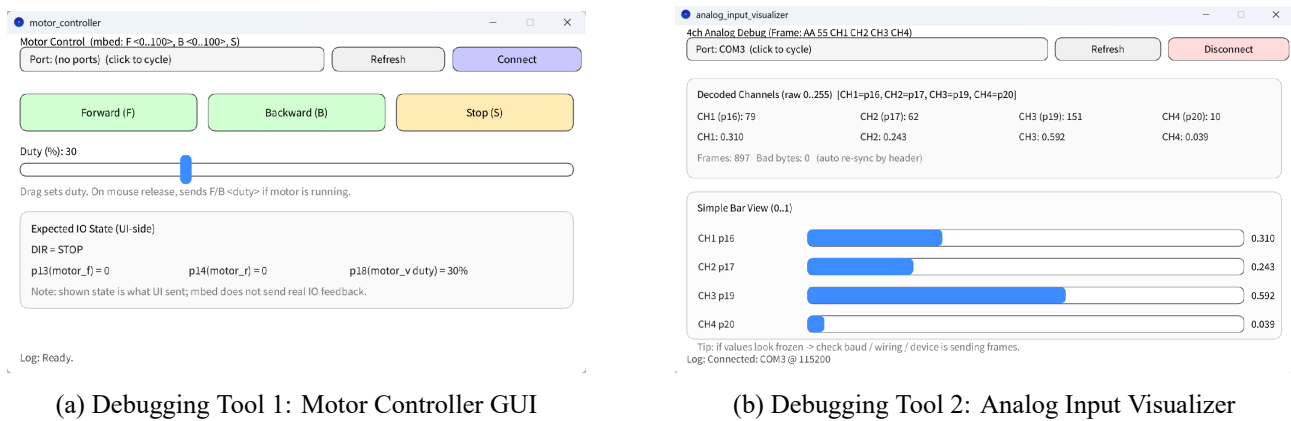


Figure 17: Processing GUI interfaces for Debugging Tools 1 and 2

- ☐ Sensors detect full range of pendulum motion
- ☐ No sensor signal interference from motor operation

3.4.5 Stage 3: Complete Circuit Assembly and Integration

■ Objective

In this final stage, you will integrate the motor control and sensor systems into a complete feedback control system capable of balancing the inverted pendulum.

■ Final Assembly

Your circuit should now include all components according to the complete sample circuit diagram¹. Review the entire circuit and verify:

- All power connections are correct and secure
- All bypass capacitors are installed adjacent to IC power pins
- Wire routing is neat and organized (see Figure 18)
- No wires are at risk of short circuits
- All component orientations are correct (ICs, electrolytic capacitors)

■ General Notes on Circuit Assembly

- *The simpler the circuit looks, the better it works.* Electronic circuits, especially analog circuits, change characteristics with slight wiring methods. Circuits that look beautiful = simple circuits tend to have better characteristics.
- *Since breadboards have large contact resistance, fewer wires allow more stable operation.* Avoid daisy-chaining multiple jumper wires when a single wire would suffice.
- Again, try not to touch the pins of IC/LSI (Resistors, capacitors, LEDs, three-terminal regulators are fine to touch).
- The sample circuit diagram is just one wiring example, so you can use different pins for mbed input/output (However, AnalogOut is only available on p18, so it cannot be changed. Also, pins on the right side of mbed cannot be used due to space limitations). Rewrite the program according to the pins used.
- Cut resistor legs to appropriate length to avoid short circuits (However, do not let the cut legs fly away).
- Follow color conventions: red for positive power, black for GND (0V).

An implementation example of the complete circuit is shown in Figure 18.

¹Appendix D: Circuit example.

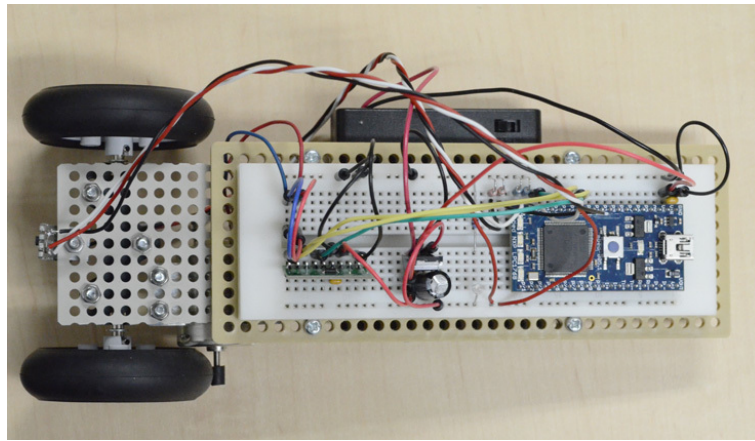


Figure 18: Complete Circuit Implementation Example

■ Integration Testing

With the complete circuit assembled, you can now test the integrated control system. The final control program combines motor control and sensor reading to implement feedback control. At this stage, you can use the comprehensive **Debugging Tool 3: Six-Channel Oscilloscope** (located in `debug_tools/3_six_channel_oscilloscope/`, also referred to as `plotter6ch`) to monitor all system variables simultaneously. This tool consists of:

- **3_six_channel_oscilloscope.cpp**: Complete mbed control program implementing PD control at 2000 Hz sampling frequency. Reads two photo-interrupters (p19, p20), calculates angle and angular velocity, controls motor (p13, p14, p18), and transmits 6 data channels via serial (230400 baud) using frame protocol (header: 0xAA 0x55, followed by 6 data bytes). Control parameters (KP, KD, VLIMIT) can be adjusted by editing the `#define` values at the top of the file.
- **3_six_channel_oscilloscope.bin**: Pre-compiled binary ready to flash to mbed.
- **3_six_channel_oscilloscope.pde**: Processing program that displays 6 channels in two 3-channel plots (upper: CH1-Red, CH2-Green, CH3-Blue; lower: CH4-Red, CH5-Green, CH6-Blue). The 6 channels typically show: (1) photo-interrupter 1 output, (2) photo-interrupter 2 output, (3) angle (difference of sensors), (4) angular velocity, (5) filtered angular velocity, (6) reference line.

This is the primary tool for tuning control parameters and debugging the complete inverted pendulum system. See the `debug_tools/README.md` file for detailed usage instructions, communication protocol, and how to modify the programs.

■ Team Final Task: Stand-up Experiment (Pre-Encoder)

At the end of Week 1 (before adding wheel encoders), your goal is to make the cart **stand** (balance upright) using **angle-only feedback** (P-only is sufficient for Week 1). You will use `plotter6ch` (**Debugging Tool 3**) and tune gains so the system can stand without violent vibration.

Expectation (Week 1): In principle, you should be able to achieve a basic stand-up with only a gentle nudge and small gain adjustments. If the result is not good enough yet, do not worry—you can preview the more detailed tuning procedure in Week 2 (including angular velocity, filtering, and adding D gain): [see Section 4.4](#).

Quick workflow:

1. Flash `3_six_channel_oscilloscope.bin` to mbed (or compile `3_six_channel_oscilloscope.cpp` after changing `#define` gains).
2. Open `3_six_channel_oscilloscope.pde` in Processing and run the oscilloscope GUI.
3. Press reset on mbed once after the GUI starts (to re-synchronize the frame alignment if the channel order looks wrong).

Power and safety

- While the battery box is ON, the motor may keep running. Turn it OFF when not actively testing.
- USB powers *mbed only*. If the motor does not move or sensor outputs look strange, double-check the battery box switch.
- If high-frequency rattling/shivering occurs, *turn off the battery box immediately* to avoid overheating and damage.

Zeroing (recommended before connecting the motor):

- Disconnect the motor from the motor driver.
- Reset mbed. During the 1 second idle period, turn on the battery box.
- Hold the system vertically and confirm the angle trace (typically the blue line) is near the screen center.
- If not, mechanically fine-tune the sensor bracket so the vertical state corresponds to zero angle.

First stand-up attempt (P-only):

- Reconnect the motor and motor driver. Keep the USB connected if you want.
- Start with $KD=0$ and tune KP first.
- Verify motor wiring direction: if the cart moves *in the direction it is falling*, it is correct. If it moves the opposite way (tries to fall by itself), turn off power and swap the motor wires (or the two wires between mbed and motor driver).
- While testing, *gently support* the top of the machine (or the USB cable) so it does not fall, but do not hinder its motion.

KP gain adjustment guideline:

1. When KP is low, relaxed (about 1–2 Hz) oscillation is seen.
2. Increasing KP increases oscillation frequency and decreases oscillation amplitude.
3. A good region is where it stands with minimal vibration (some small vibration may remain).
4. If KP is too high, high-frequency rattling appears.
5. Increasing further results in violent vibration.

Avoid high-frequency rattling. It can cause overheating and burnout of electronic parts and motors, and may loosen screws. If rattling occurs, *turn off the battery box immediately*.

3.5 Week 1 Submission

Week 1 — Stand demo video

30 pts

By the end of Week 1, your cart should be able to **stand** (balance upright)—congratulations on reaching this milestone!

Deliverable	Max	Scoring guide
Standing demonstration	20	20 pts: upright ≥ 15 s, full system in frame. 12 pts: upright achieved but < 15 s or marginal stability. 0 pts: upright not achieved.
Safety & assembly visibility	5	5 pts: complete assembly clearly visible—mechanical structure, breadboard, and wiring all shown. 0 pts: assembly not visible in the video.
Video clarity	5	5 pts: all key parts unobscured throughout the recording. 3 pts: key behavior still identifiable despite partial obstruction. 0 pts: footage unusable (major parts obscured or out of frame).

Optional attachments: Fritzing (.fzz/images), photos, or debugging tool screenshots/logs.

4 Week 2: Encoder & Position Control

4.1 Week 2 Overview & Objectives

Week 2 focuses on measuring the cart motion using a rotary encoder and using that information for both **simulation** and **real hardware control**. You will (1) build and install your own encoder, (2) integrate encoder feedback into the control code, and (3) extend the MATLAB/Simulink simulation to include **position** and **velocity** feedback.

Objectives:

- Fabricate, install, and validate a rotary encoder (clean A/B signals and correct direction)
- Add encoder-based position/velocity feedback to the control program and debug using waveform tools
- Extend the simulation to include x and $\dot{x}$ feedback and compare simulation vs hardware
- Achieve stable position control (or at least bounded drift) while maintaining inversion

4.2 Parallel Tasks

Recommended parallel roles: Student A (encoder fabrication/installation), Student B (encoder feedback in code + debugging), Student C (MATLAB/Simulink simulation extension). Roles can be rotated, but all students should understand the end-to-end flow.

- **Student A:** Task A (encoder fabrication/installation) — [see Section 4.3](#)
- **Student B:** Task B (encoder feedback in code, debugging, and position control) — [see Section 4.4](#)
- **Student C:** Task C (MATLAB/Simulink simulation extension and comparison) — [see Section 4.5](#)

4.3 Task A: Encoder Fabrication and Installation (Student A)

This task focuses on fabricating and installing a rotary encoder for cart position measurement.

4.3.1 Rotary Encoder

The inverted pendulum will not be stable unless position (or velocity) is detected and fed back in addition to the inverted angle. Therefore, let's consider the detection method of position (or velocity).

The most basic position detection method is to attach a rotary encoder to the tire or motor to measure the number of rotations. Below, the mechanism of the rotary encoder is explained.

A rotary encoder can be realized by reading a black and white pattern drawn on a disk with a photo-interrupter¹. As shown in Figure 19, if a photo-interrupter is placed at a position several mm away from a pattern painted in black and white and the slit is rotated, a sine wave-like output corresponding to the change in black and white is obtained from the photo-interrupter (it becomes close to a rectangular wave if the distance is close). If binarized with an appropriate threshold and replaced with rectangular pulses, the rotation angle can be measured by counting edges (changes in value). This is the basic principle of a rotary encoder.

Now, if the rotation direction is determined to be only one direction, this is fine, but when rotating in both directions, the above mechanism counts up regardless of the rotation direction, so it is useless. Therefore, when rotating in both directions, two photo-interrupters are used.

As shown in Figure 20, two photo-interrupters are placed at positions shifted by $1/4$ period of the slit pattern (or $(n + 1/4)$ period where n is an integer). The outputs of these two interrupters are called A-phase signal and B-phase signal. A-phase signal and B-phase signal are shifted in phase by 90 degrees like the relationship of sin and cos, but the way of phase shift changes depending on the rotation direction. If B-phase leads by 90 degrees in positive rotation (Clockwise, CW rotation), B-phase lags by 90 degrees in reverse rotation (Counter-Clockwise, CCW rotation).

¹ This time we use a reflective photo-interrupter so it is a black and white pattern, but generally transmissive photo-interrupters and slit disks are often used

Therefore, by performing calculation like Figure 21 for A-phase and B-phase signals pulsed with appropriate threshold, rotation angle can be measured correctly including rotation direction. In Figure 21, only edges of A-phase pulse are counted, which is called 2x counting (Counting rising edge is basic, counting falling edge too makes it 2x). Furthermore, doing similar counting for B-phase pulse doubles resolution (this is called 4x counting). Since we want to make encoder resolution as high as possible, program it to do 4x counting.

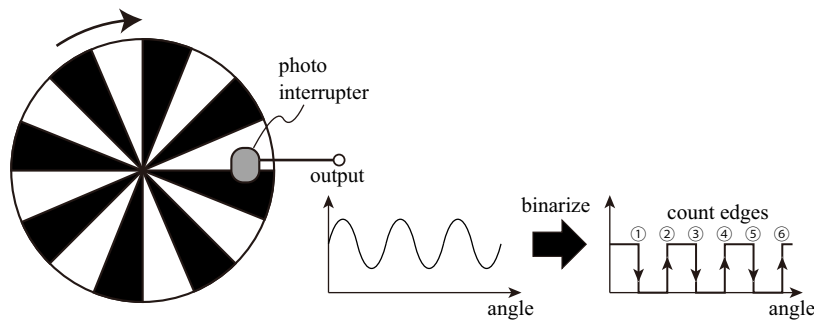


Figure 19: Basic concept of Rotary Encoder

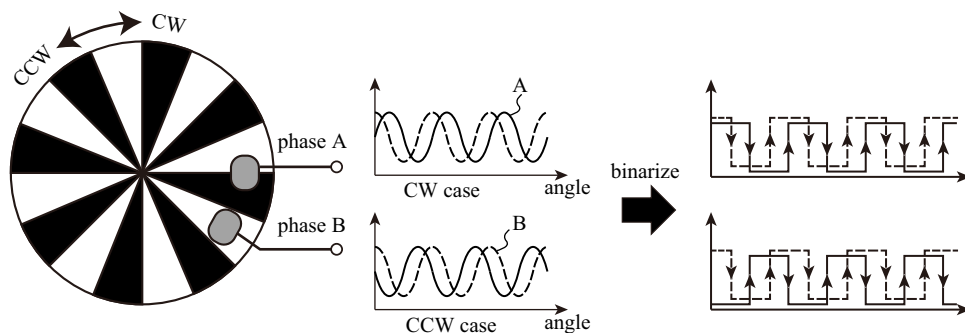


Figure 20: Measurement of CW/CCW rotation by 2 phases

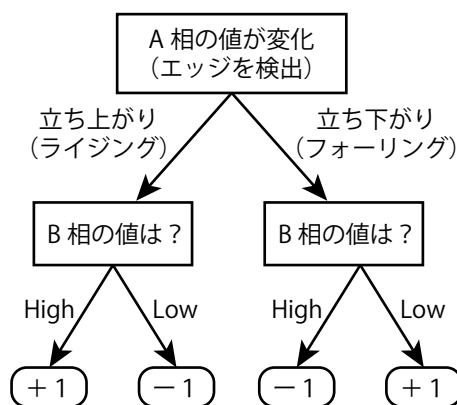


Figure 21: Counting A/B phase pulses (Example of 2x counting)

4.3.2 Fabrication of Rotary Encoder

Installation of Scale

When producing an encoder, first decide the installation location of the scale and the resolution of the scale. Installation locations are generally "attach to motor shaft" or "attach to tire (wheel)". When attaching to the tire, remove the tire from the shaft once (remove 1 nut in the center of the tire. Do not remove other screws), and stick paper printed with scale on the inside of the wheel with double-sided tape. However, if the paper is flimsy,

the distance between the scale and photo-interrupter fluctuates, so it is better to stick the scale on a prepared plastic plate (green disk) and stick the whole plate to the wheel.

Note that tires will be reused by the next group, so perform work within the range where original state can be restored.

When attaching to the motor shaft, attach paper printed with scale to the shaft slightly protruding behind the motor. In this case, use double-sided tape or tape glue skillfully to stick. It is difficult to stick firmly, but since almost no force is applied, light adhesion is sufficient (instead, make the scale as light as possible). When installing on the motor side, resolution increases by the reduction ratio (= ratio of tire radius and motor tip tube radius) compared to installing on the tire. Therefore, even a scale with coarse division number (e.g., 4 divisions every 90 degrees) provides sufficient performance.

The finer the pattern division number of the scale, the higher the resolution as a sensor, but fine scale requires high installation accuracy. This is because the finer the scale, the smaller the output change width of the photo-interrupter. If scale installation accuracy is low, distance between photo-interrupter and scale plate fluctuates with rotation, and baseline of photo-interrupter output also fluctuates. Therefore, with a fine scale, signal cannot be binarized well, and encoder does not work (see Figure 22. If baseline fluctuates, threshold set according to a specific place cannot perform binarization at other places).

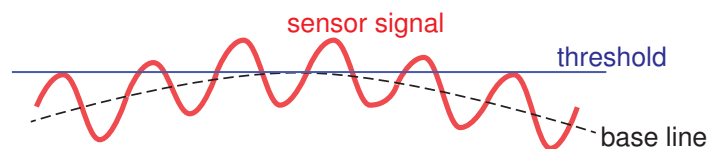


Figure 22: Example of baseline fluctuation

If scale is made coarse, sufficient signal amplitude is obtained even if photo-interrupter is placed somewhat away (about 5mm?) from scale plate. If distance is far, slight distance fluctuation does not matter. So, making scale coarse makes it easy to make, but coarse scale makes control rough. Find a good balance. In any case, scale installation accuracy is important. Fix the scale plate firmly to the wheel (see Figure 23).

Also, when deciding resolution of scale, keep photo-interrupter arrangement interval in mind. In the distributed kit, 2 photo-interrupters are already soldered on the board. Design the scale keeping this arrangement in mind. Design is done on a PC. You can design with Draw type software, or draw a pie chart in Excel etc. and use it as a scale. After designing, print with a printer (convenience stores can print if no printer) and cut out. If printing is absolutely impossible, draw manually. Black parts are filled with magic pen etc., but depending on the pen, even if it looks black, it may reflect infrared light easily (= not recognized as black by photo-interrupter), so be careful.



Figure 23: Example of encoder pattern. Left assumes reading common scale with 2 photo-interrupters arranged shifted by 1/4 period angle. Right assumes 2 photo-interrupters placed at same angle (but different distance from center), reading 2 slits shifted by 1/4 period from each other.

4.3.3 Soldering Photo-interrupter

Reading of scale is done by photo-interrupter. Referencing inclination angle sensor, solder photo-interrupter to (scrap of) universal board, and attach to universal board with L-bracket etc. (see Fig. 21). Board will be supplied so please ask. Points of soldering work will be explained in the exercise. Please listen to explanation well before doing (especially those not used to soldering work). (See Figure 24).

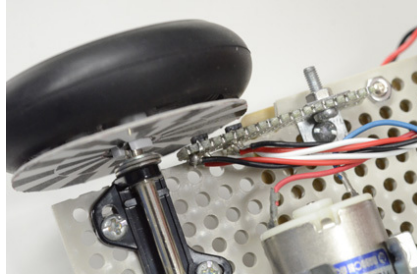


Figure 24: Photo-interrupter attachment

4.3.4 Adjustment and Reading of Photo-interrupter

After attachment to machine, connect wiring and resistors of photo-interrupter to breadboard. Although not drawn in distributed circuit diagram, connect in same way as inclination sensor. However, select resistor value connected to phototransistor according to your own design. (By the way, resistance value of inclination angle sensor at bottom of machine is $12\text{k}\Omega$). In many cases, distance to target will be closer than inclination angle sensor, so reflected light amount will increase. In that case, output saturates unless resistance value is reduced (when looking at output waveform, upper side is cut off and becomes flat). As a guide, around $1\text{k}\Omega$ to $5\text{k}\Omega$ is good.

After completion of attachment and wiring, determine threshold for binarization while looking at photo-interrupter output with simple oscilloscope. After confirming that phases of two binarized signals are shifted by roughly 90 degrees, write counting program referring to Figure 21. Phase difference does not need to be perfectly 90 degrees, but if order of edges is swapped, rotation direction will be read wrongly, so care is needed (Phase difference just needs to be larger than 0 degrees and less than 180 degrees). If desired phase difference is not obtained, adjust mounting position etc. (Make sure to rotate tire once and confirm readable at any position).

4.4 Task B: Add Encoder Feedback to Code and Debug (Student B)

■ How this relates to the weekly schedule

Week 1: your goal is to achieve a basic stand-up using **P-only** angle feedback (see [see Section 3.4.5](#)).

Week 2 and beyond: continue tuning and improvements (including angular velocity, filtering, and adding D gain). This section provides the detailed procedure and safety notes.

4.4.1 Using the Complete Control Program

The complete inverted pendulum control program is provided in `course_materials/debug_tools/3_six_channel_oscilloscope/`¹. This is not just a debugging tool—it is the **complete example program** for inverted pendulum control that implements PD control, sensor reading, motor control, and real-time data visualization.

Quick Start (Using Pre-compiled Binary):

1. Flash `3_six_channel_oscilloscope.bin` to mbed by drag-and-drop
2. Press reset button on mbed
3. The program starts with 1 second idle period

¹ Access the course materials repository at <https://github.com/UTokyo2026/UTokyo-Control-Practice-2026>

Understanding and Modifying the Program:

If you want to understand how the control works or adjust parameters, open `3_six_channel_oscilloscope.cpp` in Keil Studio Cloud. The program structure is:

- **Lines 3-11:** Control parameters (SAMP_FREQ, VLIMIT, VOFFSET, KP, KD)
- **Lines 17-30:** Pin definitions (LEDs, motor, sensors)
- **Lines 70-132:** `int0()` interrupt function—the main control loop
- **Lines 134-161:** `main()` function—initialization and startup

To adjust control parameters, simply modify the `#define` values at the top of the file, recompile, and flash to mbed. The initial program has only proportional control ($KP = 1.0$, $KD = 0.0$). You will add derivative control later in this section (see “Inversion adding Derivative Control”).

If you modify the program, click “Save” and “Compile”. If there are no errors, an executable file (.bin) is downloaded, which you can then flash to mbed. However, for initial testing, you can use the pre-compiled `3_six_channel_oscilloscope.bin` file directly.

4.4.2 Preparation for Communication

This time, a 6-channel oscilloscope is prepared to monitor the state of mbed in real time. First, install Processing according to the instruction in the appendix¹. The oscilloscope tool is located in `debug_tools/3_six_channel_oscilloscope/`.

How the Oscilloscope Works: The mbed program (`3_six_channel_oscilloscope.cpp`) runs a control loop at 2000 Hz. In each iteration, it reads sensors, calculates control outputs, drives the motor, and transmits 6 data values to the PC via USB serial at 230400 baud. The data is sent using a frame-based protocol: each frame starts with two header bytes (0xAA 0x55) followed by 6 data bytes (each 0-255). The Processing program (`3_six_channel_oscilloscope.pde`) continuously reads the serial port, searches for the header pattern to synchronize with the data stream, extracts the 6 data bytes, and plots them in real time. This protocol ensures reliable data transmission even if some bytes are corrupted or lost.

Using the Oscilloscope: Flash `3_six_channel_oscilloscope.bin` to mbed (or compile the .cpp file yourself if you want to modify control parameters). After installation is complete, open `3_six_channel_oscilloscope.pde` in Processing and press the triangular run button to open the oscilloscope screen. The screen is divided into top and bottom. 3 pieces of data in the upper half, 3 pieces of data in the lower half, a total of 6 pieces of data are displayed. If 6 pieces of data are sent every repetition period in the mbed program, they are displayed in the order sent (Upper screen R: Red, G: Green, B: Blue, Lower screen Red, Green, Blue order). However, immediately after launching the oscilloscope screen, it may not be in the correct order. By resetting mbed once after launching the screen, the display is also reset to the correct order (this re-synchronizes the frame alignment).

The `six_channel_oscilloscope` program is designed to display: (CH1) output of photo-interrupter 1 (Upper screen Red), (CH2) output of photo-interrupter 2 (Upper screen Green), (CH3) the difference between both outputs = equivalent to inclination angle (Upper screen Blue), (CH4) angular velocity = derivative of inclination angle (Lower screen Red), (CH5) filtered angular velocity (Lower screen Green), and (CH6) reference line at 128 (Lower screen Blue).

4.4.3 Initial Settings**Finally Experiment. But before that**

It's finally the experiment, but before that, let's understand the power supply mechanism of this board.

The battery box supplies power to the entire board (mbed + motor + photo-interrupter), and turning on the battery box activates the entire inverted pendulum (at this time, the LED on the board lights up). *Since the motor continues to operate while the battery box is on, please turn off the battery box frequently except when necessary = when doing inverted experiments.*

On the other hand, even when the battery box is off, connecting mbed to a PC with a USB cable **supplies power to mbed only via USB cable** (Power is not supplied to the motor and photo-interrupter from USB). At this

¹ Appendix C: Processing Oscilloscope Manual.

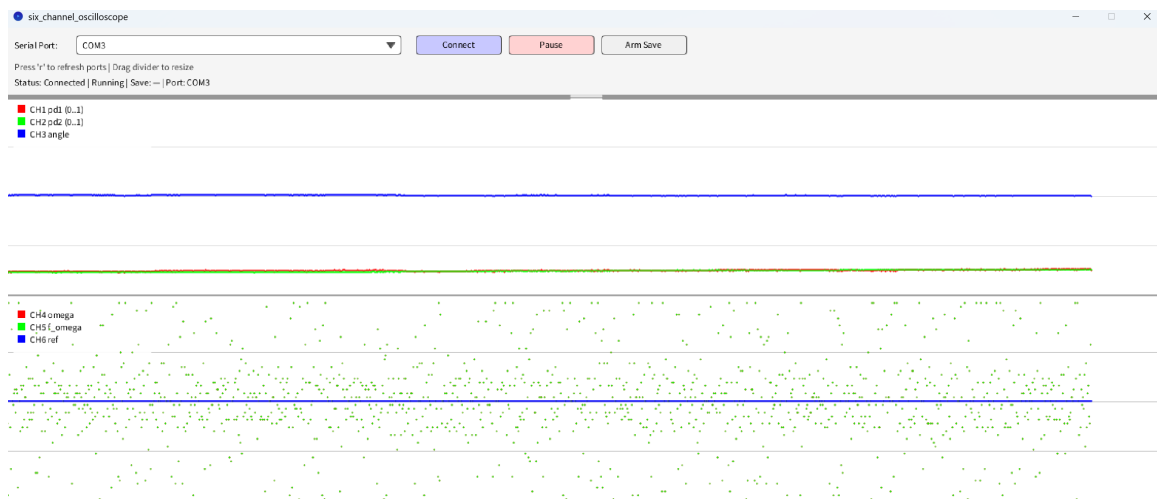


Figure 25: Debugging Tool 3: Six-Channel Oscilloscope showing real-time control data (CH1–CH3: sensor/angle; CH4–CH6: angular velocity, filtered velocity, reference)

time, the red LED on the board should be off, but since the blue LED of mbed shines brightly, there are cases where experiments are done without realizing that the battery box power switch is off. If the photo-interrupter output is strange or the motor does not move, check the battery box switch.

Zeroing the Inclination Sensor

Since it is troublesome if the motor moves in this work, *disconnect the connection between the motor and the motor driver*. With the USB cable connected to mbed, zero the sensor with the following procedure.

- Reset with the reset switch in the center of mbed (= Program start).
- Since the program is set to be in an idle state for 1 second after start, turn on the battery box power switch during that time.
- Stand the machine vertically while supporting it by hand.
- Check the inclination angle (blue line) displayed on the upper screen of the oscilloscope and confirm that it is approximately 0 (= center of the screen. Since 128 is added and sent from the program, the oscilloscope line comes to the center of the screen when sensor output is 0) when in the vertical state. If not, fine-tune the bending angle of the L-bracket attaching the sensor (Universal board is easily broken, so adjust by holding both ends of the bracket without applying force to the board).

4.4.4 Trying Inversion (Proportional Control)

After sensor adjustment is finished, let's invert it¹. Only P gain is set in the initial program ($K_P = 1.0$, $K_D = 0.0$), so proportional control is performed.

- Turn off the battery box power and reconnect the motor and motor driver. The USB cable can remain connected.
- After connecting the motor, reset mbed, turn on the battery box power switch within 1 second, and stand the machine vertically. Control starts 1 second after reset.
- Observe the reaction of the machine while lightly supporting it by hand. Support it gently so as not to fall, and check how the inverted pendulum wants to move. If the inverted pendulum moves in the direction it fell, it is correct. If it moves opposite to the inclination (= tries to fall by itself), the motor connection is reversed, so turn off the power once and swap the 2 wires connecting the motor and driver (or 2 wires connecting mbed and motor driver).

Lucky people might invert perfectly with just this. If it does not invert perfectly, it will likely move like

¹ A video demonstration of the debugging and tuning process is available in the course materials repository: [course_materials/video/3_debugging_tuning.mp4](#)

a simple harmonic oscillation. Proportional control is just a spring, so if there is no loss such as friction, the response becomes simple harmonic oscillation.

Gain Adjustment

Next, let's change the P gain and see the machine response. People who can invert at this point should lower the P gain once (to about half or less for now). Check the operation of the machine while *gently holding the top of the machine or the USB cable so as not to hinder the movement of the machine (and not to fall)*.

1. When P gain is low, relaxed (= about 1, 2Hz) simple harmonic oscillation is seen.
2. Increasing P gain (= stiffening the spring) increases the oscillation frequency and decreases the oscillation amplitude.
3. Raising the gain to some extent stabilizes it with almost no vibration (some tingling vibration may remain).
4. If the gain is increased too much, high frequency vibration (image of about 5Hz or more in the case of this machine) like rattling appears.
5. Increasing further results in violent vibration.

The state of 3 above is ideal, so adjust the gain to become state 3 after changing the gain from small to large. In the case of a machine with little friction, vibration may not completely disappear with P control alone. In that case, adjust to the gain where vibration is minimized.

Note that relaxed vibrations seen in 1 and 2 are not particularly harmful (just low stability), but high frequency vibrations in 4 and 5 can lead to *heat generation and burnout of electronic parts and motors* due to excessive current. Also, there is a risk that screws loosen due to vibration and the machine breaks, so avoid this state as much as possible.

If rattling vibration occurs, turn off the switch immediately.

4.4.5 Calculation of Angular Velocity

Next, add derivative control. Derivative control feeds back the derivative of inclination angle = inclination angular velocity, so first calculate the angular velocity. Since mathematically strict differentiation cannot be performed, the difference between the previous angle and the current angle is used as an approximate value of differentiation. That is,

$$\dot{\theta} \simeq \frac{\theta - \theta_1}{T_s} \quad (17)$$

where θ_1 is the inclination angle in the previous process, and T_s is the repetition time (sampling period). This process is already implemented in the `3_six_channel_oscilloscope.cpp` program (line 79-80 in the `int0()` function).

Since differentiation (difference) has the effect of amplifying high-frequency signals, the angular velocity obtained above contains a lot of high-frequency noise. If used for feedback as it is, fine vibration occurs due to noise (makes a "shhh" sound). In this state, a large current flows through the motor, which is not good for the circuit, so apply a low-pass filter (LPF) within the program to remove high-frequency noise.

There are two types of digital filters that can be realized by programs: IIR (Infinite Impulse Response) and FIR (Finite Impulse Response), but since phase delay is a problem in feedback control, a filter with as little phase delay as possible is preferred. Since FIR tends to introduce extra phase delay in the high frequency region, we use IIR, where phase delay is easy to grasp, to make a filter this time.

The higher the order of the filter, the better the cutoff performance, but the higher the order, the more the phase delay increases. Therefore, here we use the "first-order low-pass filter (first-order lag filter)" which is the simplest and has the least phase delay. The pulse transfer function of the first-order low-pass filter obtained by bilinear transform is:

$$F(z) = \frac{1 + z^{-1}}{(1 + \frac{2T}{T_s}) + (1 - \frac{2T}{T_s})z^{-1}} \quad (18)$$

where T is the time constant of the first-order lag, and T_s is the sampling period. The reciprocal of the time constant T is the cutoff angular frequency (rad/s). Decide an appropriate cutoff frequency and calculate the filter coefficients (be careful about the relationship between "Hz" and "rad/s").

Matlab may be used to calculate filter coefficients. In Matlab,

```
[b,a] = butter(1,0.01);
```

Typing this gives the coefficients of a 1st order low-pass filter. Here, the 1st argument of butter (function to find Butterworth filter) is "filter order", and the 2nd is "cutoff frequency normalized by Nyquist frequency" (= ratio to Nyquist frequency. Nyquist frequency is half of sampling frequency). Equation (18) does not apply frequency pre-warping, so manual calculation with this equation causes a slight shift in cutoff frequency due to frequency distortion of bilinear transform, but Matlab's butter function performs frequency pre-warping, so the specified cutoff frequency is obtained.

Note that in a first-order lag filter, phase starts to delay from a frequency 1/10 to 1/5 of the cutoff frequency¹. The process of differentiation advances the phase of a sine wave by 90 degrees (differentiating sin becomes cos with phase advanced by 90 degrees), so if phase delays, it does not become correct differentiation processing. Considering the vibration frequency of the machine in the proportional control of the previous section (which needs to be suppressed by derivative control), set the cutoff frequency of the filter to 10 times or more (at least 5 times or more) that frequency so that correct differentiation is obtained at that frequency.

For example, if vibrating at about 2Hz, set the cutoff frequency to e.g. 20Hz (= 40π rad/s) or more. Afterwards, adjust as necessary while observing the state of response in actual inverted experiments.

To turn the obtained pulse transfer function into a program, proceed as follows. First, make the transfer function a rational polynomial (fraction of polynomials) with respect to z^{-1} , and divide the denominator and numerator so that the constant term of the denominator becomes 1. Let the coefficients of the polynomial obtained thereby be $b_0, b_1, b_2, \dots$ for the numerator, and $a_0 = 1, a_1, a_2, \dots$ for the denominator. Since it is a 1st order filter this time, the numerator is only b_0, b_1 , and the denominator is only $a_0 = 1, a_1$. Note that if calculated with Matlab, these coefficients are obtained directly. At this time, the transfer function is:

$$F(z) = \frac{b_0 + b_1 z^{-1}}{1 + a_1 z^{-1}} = \frac{Y(z)}{X(z)} \quad (19)$$

where $X(z)$ is the signal input to the filter (Z transform), and $Y(z)$ is the filter output (Z transform). Recall that the transfer function is the ratio of input and output (Z transform).

Rearrange this and inverse Z transform to fix into a program.

$$(1 + a_1 z^{-1})Y(z) = (b_0 + b_1 z^{-1})X(z) \quad (20)$$

$$Y(z) + a_1 z^{-1}Y(z) = b_0 X(z) + b_1 z^{-1}X(z) \quad (21)$$

Inverse Z transform:

$$y[n] + a_1 y[n-1] = b_0 x[n] + b_1 x[n-1] \quad (22)$$

$$y[n] = b_0 x[n] + b_1 x[n-1] - a_1 y[n-1] \quad (23)$$

Just write this as a program. Showing only the main part (variable declaration omitted. Left side is line number for explanation):

```
1:    y = b0 * x + b1 * x1 - a1 * y1;
2:    x1 = x;
3:    y1 = y;
```

This completes it. Line 1 is filter calculation, and lines 2 and 3 save input and output ($x[n], y[n]$) in this cycle so that they can be referenced as previous values ($x[n-1], y[n-1]$) in the next cycle.

Once the program is written, let's see how the angular velocity signal changes before and after the filter on the oscilloscope screen. Modify the program to output values before and after the filter to the oscilloscope. If the filter is working correctly, you should be able to confirm that noise is reduced. Note that it is fine if some noise remains in the filter result. If you filter so much that noise becomes completely invisible, you have over-filtered. In that case, I think there will be a large delay in the signal. Filtering always causes time delay, but if it is obviously delayed visually, it is over-filtered. Increase the cutoff frequency to weaken the filter effect.

¹ If using Matlab, freqz(b,a) or bode(tf(b,a,sampling time)) allows checking the frequency response of the filter.

4.4.6 Inversion adding Derivative Control

Once angular velocity can be calculated, let's add D gain and perform PD control. Multiply D gain by the angular velocity passed through the filter to match equation (13). For a machine that is already stable with only P control, adding D gain may not look like stability has increased, but the effect of D gain should appear in response to disturbance. To give a disturbance, lightly poke the inverted machine with a finger. Without D gain, it will recover the inverted state while wobbling, but with appropriate D gain, the vibration converges quickly (depending on the machine, the effect may not be very visible).

Let's actually change the magnitude of D gain and see the response. The magnitude of D gain (here) is roughly several tenths (1/20?) of P gain. Note that **theoretically, the larger the D gain, the better the vibration should dampen, but in reality, if the D gain is too large, the vibration increases instead, so be careful**¹. If the machine is shivering, it is possible that D gain is too large, so try lowering D gain (depending on the machine, D gain 0 might be best).

4.4.7 Summary so far

If it inverts without large vibration and disturbance response becomes smooth, it is tentatively complete. Since position feedback is not applied, it is inevitable that the inverted pendulum gradually runs away (and falls).

Up to this point, you should arrive without problems if you make it as explained, but the real part of the exercise starts here. From here on, you will proceed with production while thinking for yourself.

4.4.8 Position Control

Once position can be read, similarly to inclination angle control, first apply only proportional gain (K_x) to position (and add it to manipulated variable of inclination angle control) to control. Specifically, motor voltage is

$$v_m = K_p\theta + K_d\dot{\theta} + K_x x + K_v \dot{x} \quad (24)$$

(At this point, position derivative gain K_v is zero).

Correct sign of gain K_x is unknown whether positive or negative (depends on setting of each rotary encoder). Try it for now, and if machine oscillates back and forth, sign is correct. If it runs away in one direction without oscillating, sign of position gain is reverse (or rather, encoder output is reverse), so change sign and try again (fixing encoder program is also fine).

Once correct sign is known, change magnitude of proportional gain and check change in response. While gently supporting machine by hand, try operating it with intention of reading movement of inverted pendulum. If gain is small, it oscillates slowly and largely, and if large, it oscillates fast and finely. In inclination angle control mentioned before, machine might have been stilled only with proportional gain, but regarding position control here, only oscillating response can be obtained with proportional gain alone (and due to signal delay and back EMF influence, oscillation amplitude gradually increases).

Note that, as in inclination angle control experiment, excessive gain causes violent vibration. Violent vibration may lead to damage of parts, so **if violent vibration occurs, turn off power switch immediately**.

In many cases, around where it oscillates slowly (about 0.5Hz?) is just right proportional gain, so aim there and adjust. Since this vibration will be suppressed by derivative control set next, it is important that frequency of this vibration is within range where derivative control works (about 1/10 or less of cutoff frequency of low-pass filter). People with high resolution encoder can set cutoff frequency of velocity filter high, and can dampen

¹In a 1st order LPF, phase delays by 90 degrees in high frequency band. Considering the case where inclination angle changes sinusoidally, its derivative (= angular velocity) must be advanced by 90 degrees phase relative to inclination angle, but at high frequencies, phase delays by 90 degrees due to the filter, so passing the differentiation result through the filter results in the same phase as the original inclination angle. This is no longer differentiation. Therefore, at high frequencies, even if intending to do derivative control, it is actually the same as doing proportional control. Thus, easily increasing derivative gain makes proportional gain substantially large in high frequency band, resulting in oscillatory and unstable response.

By the way, if it is a 2nd order filter, phase delays up to 180 degrees, but 180 degree phase delay is the same as inverting the signal. Therefore, multiplying a signal delayed by 180 degrees by derivative gain results in movement exactly opposite to original D control (= damper) (increasing velocity).

up to higher frequency, so it is okay to adjust to faster vibration (response of position becomes more agile that way). Perform adjustment keeping your own encoder performance and velocity low-pass filter characteristics in mind.

Generally, position control of inverted pendulum performs following interesting operation. When wanting to move machine to right, position control system (control realized by K_x and K_v) tries to rotate tire to opposite left direction. When tire moves left, machine falls right due to inertia, so inclination angle control system (K_p and K_d) turns tire to right to correct this inclination, and as a result machine moves right. In other words, position control system issues command to move in direction away from target position, contrary to intuition. However, since inclination angle control system is dominant, it moves in direction opposite to direction commanded by position control system as a result. Let's check what kind of operation inverted pendulum you produced performs.

4.4.9 Calculation of Velocity and Addition of Derivative Gain

Finally, calculate velocity and add derivative control. Calculation of velocity is done by differentiating (difference) output of encoder, but since output of encoder produced this time changes only occasionally (not counted unless tire rotates at least one scale pattern), simply taking difference from previous value results in strange velocity signal where impulse comes out only when encoder output changes. Apply low-pass filter to smooth this. Unlike angle differentiation, quite strong filter needs to be applied, so use 2nd order low-pass filter and set cutoff frequency low (design with Matlab butter function). However, setting cutoff low makes controllable motion frequency low as well. Position gain must be set low so that vibration frequency of machine position falls within controllable frequency range (consequently, spring constant of position control decreases, so machine may not stop perfectly and move unsteadily, this is unavoidable).

In this case, magnitude of derivative gain (velocity gain K_v) will be about same as proportional gain or fraction of it.¹

Since resolution of position sensor (= encoder) is low this time, it will be difficult to completely suppress vibration as long as differential value of encoder is used as velocity. It is sufficient if adjusted to extent that vibration does not diverge (sways with constant amplitude).

If you want to improve performance more, try estimating machine velocity by another means without using differential of encoder. Specifically, use applied voltage to motor. In steady state, that is, state where velocity is constant, if friction is ignored, motor torque becomes zero. Looking at equation (16), when motor torque is zero, applied voltage and back EMF (= proportional to velocity) match. Therefore, motor rotation speed (= machine velocity) should be estimable from applied voltage. However, this is story in steady state, so it does not hold when velocity is fluctuating. But if used thinking well around there, better control should be possible than relying only on encoder. Note that when estimating velocity in this way, it is better to build velocity control system instead of position control system (that is, P control on average value of back EMF instead of PD control on machine position). In this case, ignore explanation in next section.

¹Ratio of P gain and D gain is easy to understand when compared with standard form of 2nd order lag. If spring constant and damping coefficient are K_p and K_d , transfer function from external force to position is

$$\frac{X(s)}{F(s)} = \frac{1}{ms^2 + K_d s + K_p} = \frac{1/m}{s^2 + (K_d/m)s + K_p/m}$$

On the other hand, standard form is

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

Comparing both, it can be seen that $K_d/K_p = 2\zeta/\omega_n$. For example, if damping ratio $\zeta = 0.5$ (degree of slight overshoot), ratio of D gain and P gain is natural angular frequency ω_n . In first pendulum angle control, natural vibration was several Hz, so angular frequency should have been about 20rad/s, but in position control natural vibration is 0.1Hz range, so angular frequency is about 1rad/s. Therefore, P gain and D gain become same order.

4.4.10 Let's Run

Once it can invert at approximately constant position, run it. When running, change target position of control little by little. Specifically, command value of motor voltage is as follows.

$$v_m = K_p\theta + K_d\dot{\theta} + K_x(x - x_{ref}) + K_v\dot{x} \quad (25)$$

x_{ref} in formula is target position of control. Since PD control regarding position is applied, machine is in state equivalent to being connected to target position with spring and damper. Therefore, if this target position x_{ref} is changed little by little in program, inverted pendulum should move as if pulled to target position (like walking a dog. However, since it is image of pulling with spring, vibration due to spring occurs. Tends to be like moving big and resting a little). For example, increasing target position by 0.001 every time in control loop, if control frequency is 2000Hz, it should advance 2 encoder pulses per second.

Note that immediately after starting inverted control, attitude control of machine is not yet sufficiently stable, so it is better to program to start changing target value after waiting about 1 second after starting inverted control.

4.4.11 The rest depends on ingenuity

After that, aim for high performance inverted pendulum by adjusting variously by yourself. Improvement of encoder, adjustment of control gain (including filter adjustment), and since motor rotation speed is known by encoder, it is good to think about motor back EMF which has been ignored so far.

Here are some adjustment points.

- Combination of control gain, filter setting, target velocity is important. Trying various combinations will find setting that runs stably. Due to individual differences in parts and assembly, optimal setting differs for each machine.
- Actually, this motor (not limited to this, many model motors of same type) has determined rotation direction, and characteristics (torque) differ slightly between positive rotation and reverse rotation. When rotating in direction where torque is weak, adding slightly (at most about 10%?) extra voltage may reduce difference in characteristics.
- Since there is friction in motor itself and power transmission part of motor and tire, motor does not rotate losing to friction at small voltage near 0V. It might be good to add slight offset to command voltage to compensate friction. However, be careful as too large offset makes response of machine oscillatory. Setting value to `VOFFSET` #defined at the top of `3_six_channel_oscilloscope.cpp` adds offset (Value is safe up to about 0.01 at most).
- Since inclination angle is measured by reflected light amount from floor surface, gain of inclination sensor changes depending on infrared reflectance of floor surface. When moving on different floor surface, fine-tune control gain as necessary.

4.5 Task C: MATLAB Simulation with Encoder Feedback (Student C)

4.5.1 Role of Simulation

Computer simulation is a powerful tool for developing and validating control algorithms. Simulation provides a safe, risk-free environment for testing control strategies: mistakes do not damage hardware, and unstable controllers do not cause physical hazards. For the inverted pendulum, which can fall or move violently with improper control, simulation protects the motor, electronics, and mechanical structure from damage.

MATLAB and Simulink enable rapid iteration through controller designs. You can adjust parameters, observe system responses, and compare different strategies without repeatedly downloading code to the microcontroller or reassembling hardware. Simulation also clarifies the theoretical behavior of the system and builds intuition about how control parameters affect stability and performance. Parameters validated in simulation can then be transferred to real hardware with higher confidence.

Beyond this experiment, simulation is an essential skill for modern control engineers. The ability to model, simulate, and analyze control systems using professional tools like MATLAB/Simulink is widely used in industry and research, making this a valuable learning experience in its own right.

4.5.2 Setting Up MATLAB Environment

Accessing MATLAB at the University of Tokyo

The University of Tokyo provides MATLAB licenses for all members through a campus-wide license agreement. You can access MATLAB online or install it on your personal computer. Follow these steps to get started:

- Access the UTokyo MATLAB information page: <https://utelecon.adm.u-tokyo.ac.jp/matlab/>
- Follow the instructions under "How to start using MATLAB"
- Create a MathWorks Account using your UTokyo email address
- Link your MathWorks account to the UTokyo campus-wide license by signing in through the UTokyo portal

If you encounter any issues with license activation, refer to the utelecon support page or contact MathWorks support at service@mathworks.co.jp.

Required Toolboxes

For this experiment, you will need the following MATLAB products:

- **MATLAB** (base installation)
- **Simulink** - for building block diagrams and simulation
- **Simscape** - for physical system modeling
- **Simscape Multibody** - for multibody mechanical dynamics (inverted pendulum, joints, etc.)
- **Simulink Control Design** - required if you want to linearize a Simulink/Simscape model and use model-based tools such as PID Tuner / Model Linearizer on the Simulink model
- **Control System Toolbox** - recommended for analysis/design with LTI models (e.g., LQR, frequency response) and for PID Tuner when the plant is an LTI model

During the MATLAB installation process, make sure to select these products. If you have already installed MATLAB without them, you can add them later through the MATLAB Add-On Explorer (Home tab → Add-Ons → Get Add-Ons). The provided model uses Simscape and Simscape Multibody to represent the physical dynamics of the inverted pendulum (e.g. 3D mechanical components) rather than Control System Toolbox or hand-written state-space blocks.

4.5.3 Building the Simulink Model

This course provides a reference Simulink/Simscape model as a milestone example. The goal of the example is intentionally modest: it demonstrates how to build a physically meaningful plant model in Simscape Multibody, and how to stabilize an inverted pendulum by closing a feedback loop on the *tilt angle* using a PID controller. In the provided setup, the controller is tuned to balance the pendulum, while position and speed regulation are left as an extension task for students.

Figure 26 shows the overall Simscape Multibody diagram of the plant used in the example. Even at this top level, the model is organized to mirror the real hardware: the cart is represented with wheel-related components and the "body" (the pendulum and its attached parts) is represented as a separate rigid assembly. This separation makes it easy to identify what parameters belong to the cart (wheel radius, wheel inertia, cart mass) versus what belongs to the pendulum body (mass distribution, length, and center of mass).

4.5.4 How the Plant is Built

The plant is built in Simscape Multibody by assembling 3D rigid bodies, joints, and fixed frame transforms. Instead of typing linearized state-space matrices, you specify physical properties—geometry, mass, and inertia—and Simscape Multibody derives the multibody equations of motion automatically. This approach is particularly suitable for this experiment because it stays close to the actual machine and makes it straightforward to change mass distribution (battery, breadboard, brackets) and see how the dynamics change.

In the provided model, the cart and the body are constructed primarily using two standard solid blocks: **Brick Solid** (rectangular block) and **Cylindrical Solid** (cylinder). Wheels, shafts, and spacers are naturally

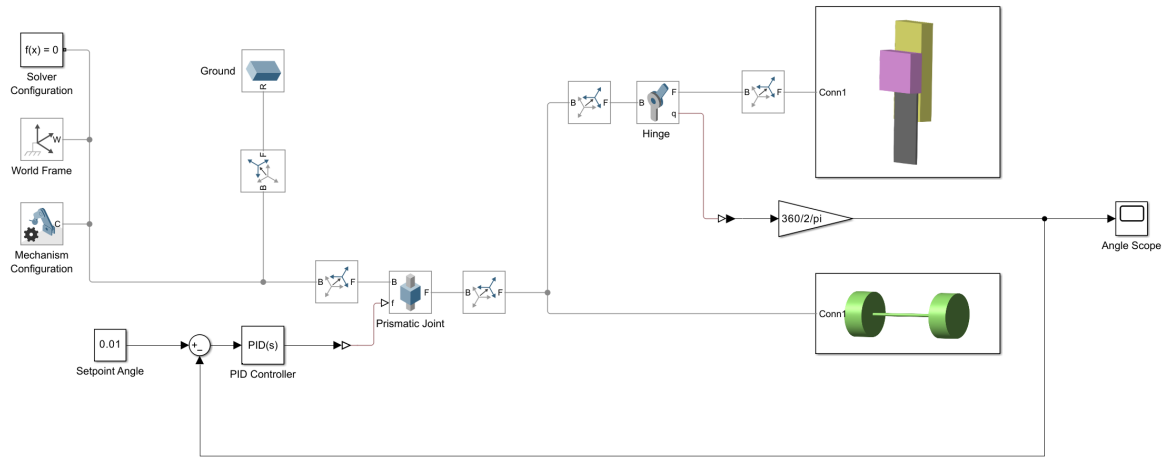
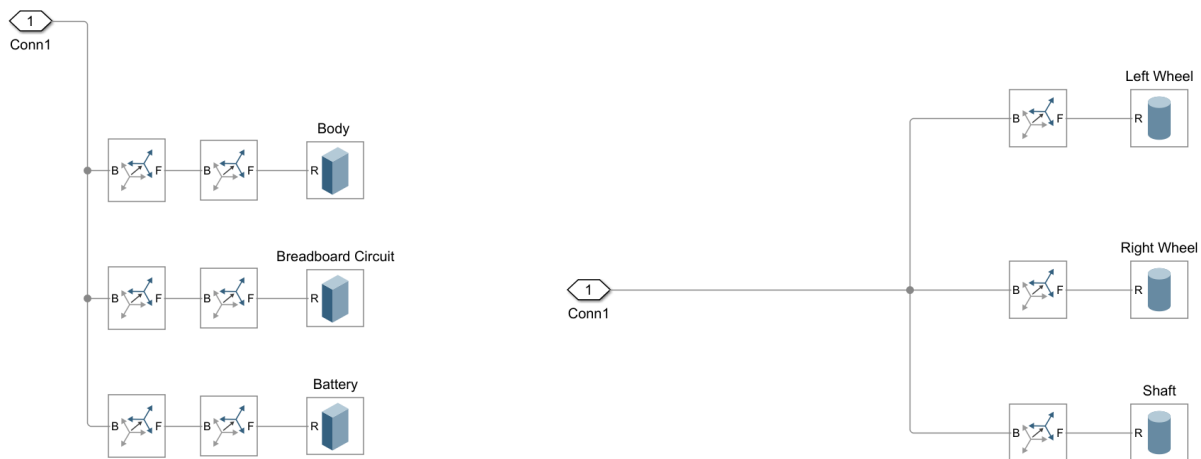


Figure 26: Overall Simscape Multibody plant diagram of the provided milestone example. The model is organized into the cart (wheel-related assembly) and the body (pendulum-related assembly), connected by joints and frame transforms.

represented by cylindrical solids, while the body plate, brackets, and electronic modules are well approximated by brick solids. The objective is not photorealistic geometry; rather, it is to capture the dominant mass and inertia so that control design decisions made in simulation remain meaningful on hardware.

Figure 27a shows the rigid *body* subassembly, while Figure 27b shows the *wheel* subassembly used for the cart/wheel-related part of the plant.



(a) Body model in Simscape Multibody.

(b) Wheel model in Simscape Multibody.

Figure 27: Simscape Multibody subassemblies used in the inverted pendulum model.

To connect these solid parts into a coherent rigid assembly, the model uses the **Rigid Transform** block. Rigid Transform defines a fixed relative translation and rotation between two frames, and is used throughout the model to place wheels relative to the cart frame, to set offsets between the pendulum body and its reference frame, and to position auxiliary masses such as the breadboard and the battery. In practice, the most important parameters in these blocks are the *mass* and *size* of the wheel and body solids, and the mass/size assigned to the breadboard circuit and battery modules. These parameters shape the inertia and therefore strongly affect how

“aggressive” the stabilizing controller must be.

One advantage of the multibody approach is that you can visualize the 3D motion during simulation and directly connect measured signals (joint angles, positions) to the controller. If you prefer a purely analytical plant, you may also represent the dynamics using a State-Space block (Simulink → Continuous) by entering the linearized A , B , C , D matrices derived earlier. Both representations are valid; this task focuses on understanding the multibody modeling workflow provided by the course.

4.5.5 Controller Internal Setup

The physical model contains joints that define how bodies move relative to each other. In the provided example, the body is connected to the cart through a hinge-like rotational joint, and the cart is allowed to move along the ground direction through a translational joint. In Simscape Multibody, the hinge joint is the **Revolute Joint** block. The cart translation is modeled using a **Prismatic Joint** block. These joints are not only mechanical constraints; they are also the main source of feedback signals in the simulation.

In the real hardware, the tilt angle is estimated from two photo-reflectors (photo-interrupter style sensors) that observe the reflected infrared light from the ground surface. The interaction between the wheel and ground is also not “measured” as wheel-ground friction; instead, the cart motion is the result of the motor drive and the ground reaction. To keep the milestone model focused and controllable, the simulation abstracts these realities: the tilt angle is taken directly from the Revolute Joint angle, and the cart-ground interaction is represented by the Prismatic Joint coordinate. This makes it straightforward to close a feedback loop while still reflecting the essential mechanical constraints of the system.

Figure 28 highlights how the angle joint and the translation joint are configured in the provided model. The Revolute Joint provides the body angle (and, if enabled, angular velocity). The Prismatic Joint provides the cart position (and velocity). In addition, the Prismatic Joint is used as the actuation point in the milestone example: by driving the prismatic motion/force (depending on the actuation setting), the model produces cart motion that stabilizes the body. If you later implement position control, you will naturally read cart position from the Prismatic Joint, and if you implement velocity control you will either read prismatic velocity directly or estimate it through encoder-like quantization (see the assignment at the end of this section).

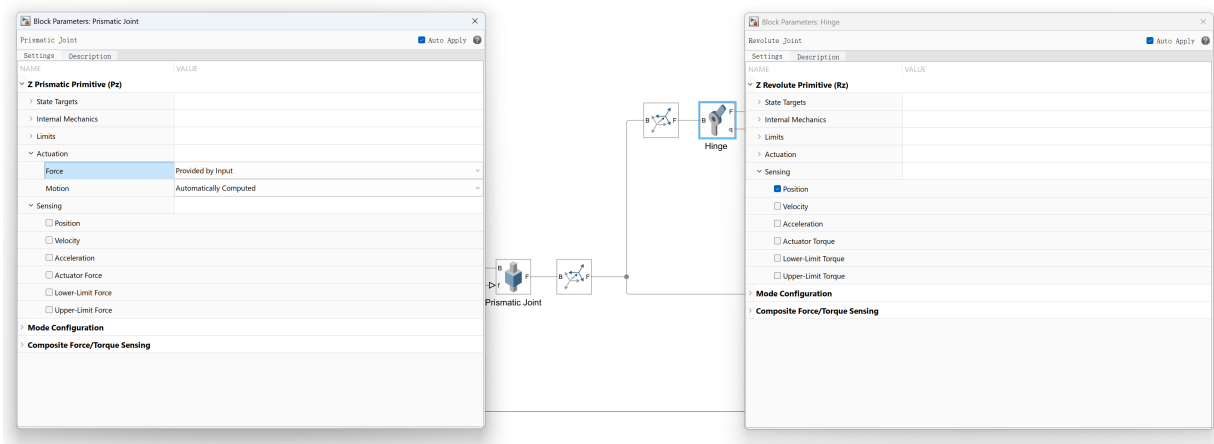


Figure 28: Joint configuration in the provided multibody model. The body angle is obtained from a **Revolute Joint** (hinge), while cart translation along the ground direction is modeled by a **Prismatic Joint**. The prismatic joint can also be used as the actuation point and as a source of position/velocity signals for outer-loop control.

With these signals available, the controller block closes the stabilizing loop. In the milestone example, the controller uses only the pendulum tilt angle as the primary feedback signal, i.e., it is an *angle-only* stabilizer. Position and speed loops are intentionally left out of the default configuration so that students can extend the model systematically.

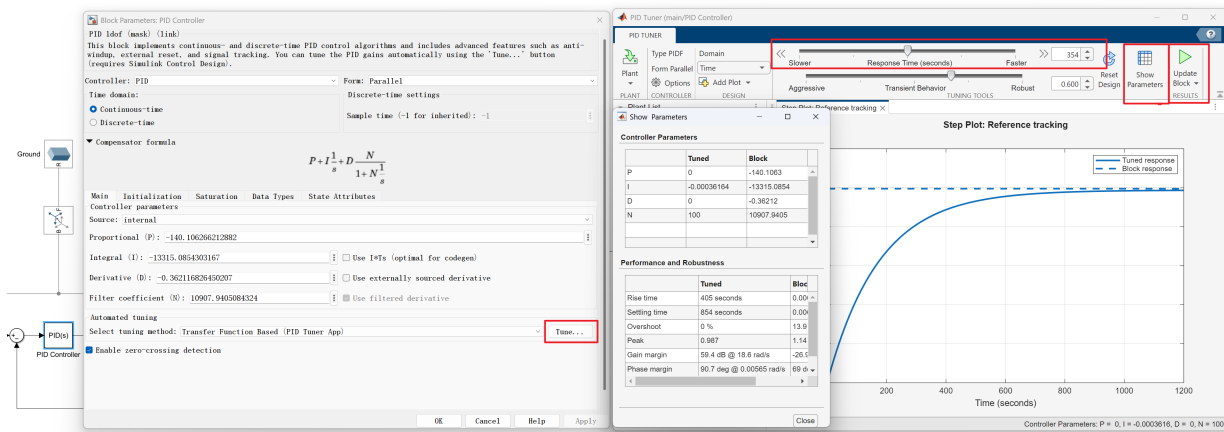


Figure 29: PID controller subsystem used in the milestone example. The provided controller stabilizes the inverted pendulum by regulating the tilt angle. PID parameters can be tuned manually or with the PID Tuner workflow.

4.5.6 Running the Simulation

Locate the provided `.slx` model file in the course material folder and open it in MATLAB. Before running, configure the simulation to match a realistic digital control loop. In **Simulation** → **Model Configuration Parameters** (Ctrl+E), use a **fixed-step** solver and choose a step size consistent with your intended control frequency (for example, 1 ms to match a 1 kHz loop, or another value if your real implementation uses a different rate). Matching the sample time is important because it affects stability margins and makes simulation results more predictive when you later transfer the controller to hardware.

To observe stabilization, set a small initial tilt angle in the Revolute Joint (for example, $\theta(0) = 0.1$ rad $\approx 5.7^\circ$) so the controller has a disturbance to reject. Then run the simulation and open the angle scope to check whether the angle converges back to the upright equilibrium. If the model includes a disturbance input, you can also inject a brief impulse or step disturbance and examine how quickly the controller recovers.

4.5.7 PD Controller Design and Parameter Tuning

The milestone controller focuses on balancing and therefore uses only the pendulum angle as feedback. In its simplest form, this is a PD law based on the angle θ and angular velocity $\dot{\theta}$:

$$u = K_p \theta + K_d \dot{\theta} \quad (26)$$

Here, u represents the actuation signal that drives the cart motion in the model. The proportional term provides the restoring tendency toward upright posture, while the derivative term provides damping that suppresses oscillation. In practice, angular velocity feedback is often filtered to reduce sensitivity to measurement noise; the Simulink PID block provides an internal derivative filter parameter for this purpose.

Accessing and Adjusting PID Parameters

To modify the gains, open the controller subsystem (Figure 29) and double-click the PID block responsible for angle control. The block dialog exposes K_p , K_d , and the derivative filter coefficient N . The provided model contains one working parameter set as a starting point; however, you should still change gains deliberately and observe how the time response changes, because the most valuable outcome of this task is understanding the relationship between physical parameters (mass/inertia) and control aggressiveness.

If you want to tune more systematically, use the **PID Tuner** workflow. From the PID block dialog, select the tuning method that opens PID Tuner and click **Tune**. PID Tuner linearizes the plant at an operating point and proposes gains based on a chosen design focus. In this experiment, it is recommended to set the target response speed to match the real control bandwidth implied by your sample time. After tuning, use the button

that updates the tuned gains back to the PID block so the simulation runs with the new parameters.

4.5.8 Analyzing Simulation Results

After running the simulation, use the angle scope to evaluate whether the inverted pendulum is truly stabilized. A good stabilization run shows the angle converging back to zero (upright) with bounded overshoot, and the control effort decreasing as the system approaches equilibrium. If the model contains a control input scope, check that the actuation signal does not remain saturated for long periods; persistent saturation typically indicates gains that are too aggressive or a plant configuration that requires re-checking mass/inertia parameters.

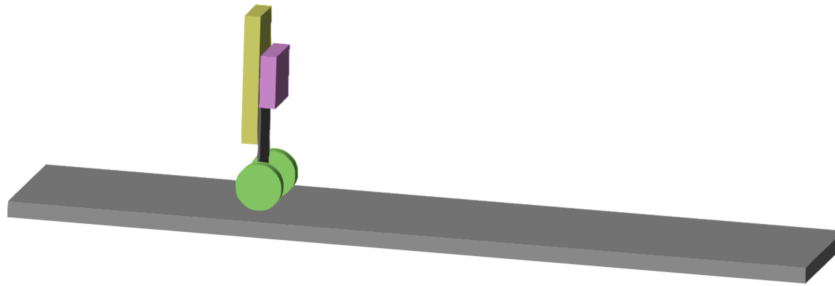


Figure 30: Screenshot of the inverted pendulum walking (Sim6).

What to Look For

In a healthy run, the angle response should return toward zero with a decay of oscillation, and should not drift away over time. If the response diverges, the stabilizing gains are effectively too weak or have the wrong sign. If oscillation persists with nearly constant amplitude, damping is insufficient (derivative gain/filtering may need revision). If the response is extremely slow, gains are too conservative. If the actuation saturates for long durations, reduce aggressiveness or revisit the plant parameters (for example, overly large mass assignments to battery/breadboard can easily make the plant harder to balance).

4.5.9 Optional Extensions

Once you can stabilize the angle in the milestone model, there are many directions to extend the simulation. A natural next step is to add outer-loop regulation for cart position and cart velocity using the signals available from the Prismatic Joint. Another direction is to compare manual PID tuning with more systematic methods such as full-state feedback or LQR, using the same test scenarios so that performance trade-offs are clear.

4.5.10 From Simulation to Real Hardware

Once you are satisfied with simulation behavior, you can transfer the controller concept to the real hardware. However, it is important to remember what the simulation *abstracts away*. In hardware, the angle sensor is built from two photo-reflectors and therefore includes noise, floor reflectance dependence, and nonlinearities. Similarly, in hardware the cart motion is generated by motor torque and wheel-ground interaction, not by a perfectly actuated prismatic joint. This is why matching sample time and keeping control effort within realistic limits are critical if you want simulation results to remain useful.

Discretization

The microcontroller implementation is discrete-time (sampling-based). Even if you use a continuous-time PID block in Simulink, the real controller will effectively operate with a sample time T_s . For an inverted pendulum, T_s in the 1–10 ms range is typical. A smaller T_s generally improves phase margin and stability, but

it also increases computational load and may magnify noise if velocity is estimated by differencing a quantized encoder signal.

In the provided mbed code (see Section 4.4), the timer interrupt is typically set to 1 ms (0.001 seconds), which corresponds to a sampling frequency of 1000 Hz. Make sure your Simulink simulation uses a fixed-step solver with the same time step for accurate prediction.

Parameter Transfer

Record the final K_p and K_d values from your Simulink model. These will be directly entered into your mbed program as variables. For example, if your simulation uses $K_p = 50$ and $K_d = 5$, you will write in your mbed code:

```
float Kp = 50.0;
float Kd = 5.0;
```

Make sure the units are consistent. In simulation, angles might be in radians, while your sensor readings might need conversion from encoder counts to radians.

Expected Differences Between Simulation and Reality

No simulation is perfect, and you should expect gaps between simulated and actual behavior. Model parameters (mass/inertia), unmodeled friction and backlash, and actuator saturation/dead-zone effects all matter. On the sensing side, real signals are noisy and quantized: photo-reflector angle estimates drift with floor reflectance, and encoder-based velocity estimates can become very noisy if obtained by simple differencing. Finally, computational delays and discretization can reduce stability margins. Therefore, treat simulation as a safe environment to validate ideas and obtain reasonable initial gains, then tune conservatively on the real system.

4.5.11 Assignment: Add Position and Velocity Loops with Encoder-like Feedback

The milestone model stabilizes only the tilt angle. Your task is to extend it toward a more realistic control system by adding outer-loop regulation for cart motion and by modeling encoder-like feedback. A typical direction is to add a position loop that generates a velocity command, then a velocity loop that generates the actuation command used by the plant, while keeping the angle stabilizer as the innermost loop. You do not need to follow a single prescribed architecture, but you should clearly describe your chosen loop structure and justify it using simulation evidence.

To mimic an encoder, do not use perfect continuous position and velocity signals directly. Instead, quantize the cart position into discrete counts, then reconstruct velocity by differencing (and filtering if needed). This will naturally introduce the same kind of jitter and delay that you will face on hardware. Use this encoder-like signal path to close the position/velocity loops and examine how robust the balancing behavior remains.

Finally, evaluate your extended controller under meaningful commands and disturbances. At minimum, test a set of position references that includes a step, a ramp-like motion, and a back-and-forth (reciprocating) motion such as a sine wave or a piecewise constant command that alternates between two positions. In addition, add an external disturbance input (for example, a short impulse/force applied to the cart or body) and record how quickly the system returns to balance. Your report should include time responses of angle, cart position, and control effort, together with a short discussion of tuning decisions and the observed limits (saturation, oscillation, sensitivity to quantization noise).

4.6 Supplements: Troubleshooting & Speed

Supplement 1: Huh? Motor became weak? If you think so

Breadboard has relatively large contact resistance, and especially around motor driver where large current flows, troubles caused by that contact resistance tend to occur. If you think motor power became weak, try inserting and removing motor driver and jumper wires around it several times.

This driver flows maximum 1A current to motor (voltage at that time is about 1V). On the other hand, jumper wire is only in contact sandwiched by 2 metal plates inside breadboard, and there is certain contact resistance in that contact part. For example, if contact resistance is 0.5Ω , voltage drop when 1A flows here is 0.5V, which becomes value that cannot be ignored compared to motor drive voltage.

Actually contact resistance is not that large, but it seems there are many cases where contact resistance increases over time due to oxidation of metal on terminal surface. Especially, there is a story that contact part where large current flows generates heat by contact resistance, and that heat promotes oxidation and contact resistance tends to increase. In other words, very inconvenient situation for motor and driver exists on breadboard. Therefore, if you feel "Performance became worse though program is unchanged from before", suspect increase in contact resistance. By inserting and removing driver chip and jumper wires several times, oxide film on surface can be removed and contact state can be restored.

Pay special attention to path where large current flows around driver. Tracing current path to motor: "Battery + pole" → "VCC terminal (of driver)" → "OUT1(or 2) terminal" → "Motor" → "OUT2(or 1) terminal" → "ISEN terminal" → "GND" → "Battery - pole". It is good to check this path intensively.

Supplement 2: For Speeding Up

According to datasheet (not simplified version distributed, but formal datasheet issued by TI¹), motor driver used this time has protection circuit that works when current exceeds 1.3A and trips (cuts off circuit and makes current 0A). When actually connecting motor and measuring, it has been confirmed that it trips at around average current 1.1A probably because motor current is pulsating. Since motor stops when tripping, the control program sets upper limit of motor voltage so as not to trip driver (see `VLIMIT` definition in `3_six_channel_oscilloscope.cpp`). Let's consider this upper limit value.

Relationship between voltage V applied by motor driver to motor and current I actually flowing in motor is expressed as follows if inductance of motor winding is ignored.

$$V = (R_m + R_c)I + \kappa\omega_m \quad (27)$$

where R_m is winding resistance of motor, R_c is resistance of motor drive circuit, κ is back EMF constant ($\approx$ torque constant), ω_m is motor rotation speed (rad/s). In this motor, winding resistance R_m (fluctuates depending on how brush inside motor hits = angle of motor shaft) is estimated to be about 0.7Ω on average, and back EMF constant κ about 2.7mV s/rad . Circuit resistance R_c is about 0.5Ω (Assume ON resistance of driver 0.45Ω + resistance of breadboard 0.05Ω). Most current flows when motor rotation speed is zero where back EMF disappears, but to keep motor current I at this time below 1.1A, it is understood that driver output voltage V should be limited to 1.32V or less.

Motor driver is designed to output 4 times voltage applied to VSET pin to motor. Therefore, upper limit of voltage applied to VSET is $1.32/4=0.33\text{V}$. VSET pin is connected to p18 (AnalogOut) of mbed, but to suppress voltage of p18 to 0.33V, upper limit of value written to p18 in program is $0.33/3.3 = 0.1$ (1.0 corresponds to maximum value of DA converter = 3.3V). For this reason, the control program sets `VLIMIT = 0.1` by default. (Note that since there are individual differences in drivers and motors, people whose driver trips immediately should try lowering this upper limit value a little).

Now, this is fine when motor is stationary, but when motor starts rotating, voltage decreases by back EMF, so sufficient current cannot flow with this upper limit. For example, if motor rotates 20 times per second (at this time, tire rotates about 2 times per second), back EMF of about 0.34V is generated. Then, even if upper limit 0.1 is written to p18 and driver generates 1.32V, only about 0.8A flows to motor. Since motor torque is proportional to flowing current, sufficient torque cannot be generated as it is.

Therefore, if you want to increase running speed of inverted pendulum, it is necessary to increase output upper limit according to speed taking above back EMF into account. If increased well, stable inversion can be maintained even at high speed (Be careful as driver trips and inverted pendulum falls easily if upper limit is not set well).

¹<https://www.tij.co.jp/product/jp/DRV8832#tech-docs>

Supplement 3: Defect of Jumper Wire

In this practice, jumper wires for breadboard (wires with pins installed at both ends to be easily inserted into breadboard) are used. Pin part of this jumper wire rarely breaks. Operation failure due to disconnection of jumper wire is quite difficult to find, but when "Circuit should be built correctly, but it doesn't work at all!", suspect defect of jumper wire. To actually find defect of jumper wire, pull out jumper wire of suspicious place and check continuity with tester.

Such defects are often caused by poor handling of jumper wires. Be careful not to forcibly bend or pull jumper wire (tip part). Even if you do not suffer damage, next group person might suffer damage.

4.7 Week 2 Submission

Week 2 — Group report

20 pts

Your system should demonstrate encoder-based position/velocity sensing and improved stability with position control. Submit a **single group report** containing:

Deliverable	Max	Scoring guide
Position-control video	8	8 pts: inversion ≥ 10 s within ± 0.5 m of set-point. 5 pts: inversion held but position drift is large. 0 pts: inversion lost within 5 s.
Encoder A/B plots	7	7 pts: A/B waveforms clearly labeled <i>and</i> direction reversal verified (CW vs. CCW phase shift). 5 pts: waveforms shown but direction not verified. 2 pts: pulse count only, no phase analysis. 0 pts: not submitted.
Sim vs. hardware comparison	5	5 pts: $x(t)$ and $\dot{x}(t)$ plots for both sim and hardware, <i>with</i> discrepancy explained. 3 pts: plots included but no explanation of differences. 1 pt : qualitative description only (no plots).

5 Week 3: Advanced Modeling & Load Testing

5.1 Week 3 Overview & Objectives

Week 3 focuses on bridging the gap between an ideal simulation and real hardware by (1) improving the simulation model (wheel dynamics + ground friction), (2) performing controlled **load tests** on the real system, and (3) systematically tuning and comparing controllers (PID vs. at least one advanced method such as LQR or MPC).

Objectives:

- Extend the simulation to include wheel dynamics and a friction model, and document the assumptions clearly
- Demonstrate stable inversion under added load and record repeatable test evidence
- Tune PID gains in MATLAB under load and compare with at least one alternative controller (e.g., LQR/MPC)
- Compare simulation vs hardware performance using consistent metrics and plots

IMPORTANT: Safety during load tests

- **Secure the load firmly.** If the load falls off, it can damage the hardware or injure people.
- **Avoid prolonged high-frequency rattling.** If you hear harsh buzzing/rattling or see violent vibration, turn off the battery immediately (risk of overheating or loosening screws).
- **Start from conservative gains.** Add load increases required torque and makes saturation/tripping more likely.
- **Work on a clear floor and keep hands away from moving wheels.** Prepare a “catch” posture to prevent hard impacts when the system falls.

5.2 Parallel Tasks

Recommended parallel roles: Student A (hardware load attachment + load testing), Student B (simulation model enhancement), Student C (MATLAB tuning + controller comparison). Roles can be rotated, but keep a single shared test log (same naming and metrics).

- **Student A:** Task A (load attachment and hardware tests) — [see Section 5.3](#)
- **Student B:** Task B (extend simulation: wheel dynamics + friction) — [see Section 5.4](#)
- **Student C:** Task C (PID tuning + advanced controller comparison) — [see Section 5.5](#)

5.3 Task A: Load Attachment and Hardware Tests (Student A)

This task adds controlled load to the cart/pendulum and evaluates stability and performance on real hardware.

5.3.1 A1. Choose and attach the load

- **Load candidates:** coins/metal washers/small weights or a small object with known mass.
- **Suggested range:** start small (e.g., 50–100 g total) and increase gradually only if stable.
- **Where to attach:** record the attachment location (height and forward/back offset). A higher or more offset load changes the dynamics more strongly.
- **Attachment method:** use tape/zip-tie/rubber bands such that the load cannot slide or detach during vibration.

5.3.2 A2. Hardware test protocol (baseline → load)

1. **Baseline run (no load):** run your best Week 2 configuration and record a short video.
2. **Loaded run:** attach the load and repeat the same test conditions (same floor, similar initial angle, same

gains).

3. **Controller variations:** for each load condition, test at least:
 - (i) tuned PID (or PD) and (ii) one alternative controller configuration (from Task C).
4. **Stop criteria:** if the motor driver trips repeatedly, if the system rattles violently, or if temperature rises noticeably, stop and reduce aggressiveness.

5.3.3 A3. What to record (minimum evidence)

- A clear video showing the system balancing under load (include a few seconds before/after enabling control)
- A simple test log table with: load mass, attachment location, gains/controller type, sampling rate (if changed), notes (success/failure)
- If available, screenshots/plots of key signals (angle, control input, encoder-based motion) for at least one baseline and one loaded trial

5.4 Task B: Extend Simulation (Wheel Dynamics + Ground Friction) (Student B)

This task improves the simulation so it can explain observed differences between Week 2 simulation and hardware.

5.4.1 B1. Add wheel dynamics

- Include wheel inertia and motor-to-wheel dynamics consistently (at minimum: additional rotational inertia term and its coupling to x)
- Ensure the output signals needed for comparison are available (e.g., θ , x , control input, and any estimated velocities)

5.4.2 B2. Add a friction model

- Implement at least one friction effect such as:
viscous friction ($F = -b\dot{x}$) and/or **Coulomb friction** ($F = -F_c \operatorname{sgn}(\dot{x})$), or an equivalent block in Simulink
- Document assumptions: which friction terms are used, parameter values, and how you selected/estimated them

5.4.3 B3. Run and document enhanced simulation

- Compare Week 2 (simplified) vs Week 3 (enhanced) simulation for the same controller settings
- Provide at least: time responses of θ and x (or their proxies), and control input (check saturation)
- Save plots in a consistent format so they can be included in the Week 3 group report

5.5 Task C: PID Tuning and Controller Comparison (Student C)

This task tunes PID gains in MATLAB under load and compares with at least one alternative controller.

5.5.1 C1. Tune PID (or PD) under load in MATLAB

- Use your Week 3 enhanced model (including load) for tuning; if you use PID Tuner, save the tuned gains and the tuning settings. If you tune directly from a Simulink/Simscape model, you typically need **Simulink Control Design** (for model linearization). If you tune using an LTI plant model (transfer function/state-space), **Control System Toolbox** is typically sufficient.
- Record the final gains and the expected step/disturbance response (rise time, overshoot, settling time)
- Ensure the control input remains within realistic limits (avoid persistent saturation in simulation)

5.5.2 C2. Try at least one advanced controller (recommended: LQR)

- **LQR (recommended):** linearize your model, choose Q , R , compute gains, and simulate under baseline and load. If you linearize from the Simulink/Simscape model, **Simulink Control Design** is typically required; alternatively, you can use the linearized A , B matrices derived in Section 2 (then **Control System Toolbox** is sufficient for `lqr`).
- **MPC (optional):** if attempted, clearly state constraints, horizon, and sampling time
- Compare against PID using the same test scenario and metrics

5.5.3 C3. Simulation vs hardware comparison

- Define common metrics (pick at least three): stability duration, peak angle, RMS angle, position drift, control effort (RMS or peak), saturation time ratio
- Summarize results in a small table for: baseline vs load, PID vs alternative controller, simulation vs hardware
- Briefly explain the gaps (likely causes: friction mismatch, saturation, sensor noise, discretization, delays)

5.6 Week 3 Submission

Week 3 — Group report

20 pts

Your work should show a clear progression: **enhanced model** → **tuned control** → **validated under load on real hardware**. Submit a **single group report** containing:

Deliverable	Max	Scoring guide
Enhanced simulation results	6	6 pts: model includes wheel inertia + ground friction + numerical parameter values, all documented. 4 pts: two of the three elements included. 2 pts: one of the three elements included. 0 pts: no enhancement over Week 2 model.
Load test video	7	7 pts: stable inversion ≥ 5 s under load; load mass and attachment location clearly identified (caption/overlay). 5 pts: stable under load but load mass/location not quantified. 2 pts: load test attempted with documentation but stability not achieved. 0 pts: not submitted.
Controller comparison	7	7 pts: PID metrics reported (rise time, overshoot, settling time) <i>and</i> a structurally different controller (LQR / state feedback / feedforward) compared using the same metrics. 4 pts: PID only, or alternative from the same family (e.g., PID variant). 2 pts: qualitative discussion only (no quantitative metrics).

6 Week 4: Final Presentation

6.1 Week 4 Overview & Objectives

Week 4 is the final wrap-up. You will present your design and results, demonstrate a working system, and answer technical questions. The key is to provide a coherent evidence chain:

modeling assumptions → **controller design** → **implementation** → **validation (simulation and hardware)** → **lessons learned**.

Objectives:

- Prepare a clear 10-minute presentation (slides) explaining theory, design choices, and results
- Demonstrate stable inverted-pendulum control on hardware (live)
- Answer questions with evidence (plots, logs, reasoning, and limitations)

6.2 Week 4 Requirements

- **Presentation):** 10 minutes, 8–12 slides, with a clear story from modeling assumptions to controller design, implementation, and validation (simulation + hardware).
- **Live demo:** demonstrate stable inverted-pendulum control on hardware with a safe, repeatable setup (have a conservative fallback configuration ready).
- **Evidence for Q&A:** be able to justify design choices using plots/logs/metrics; explain simulation–hardware gaps, limits, and robustness under load/disturbances.

6.3 Week 4 Submission

Week 4 — Final submission

30 pts

Submit the following package and deliver a live presentation:

Deliverable	Max	Scoring guide
Slides (PDF)	10	10 pts: all four elements covered with supporting data: (1) theory, (2) design, (3) validation (sim + hardware), (4) lessons learned. 7 pts: three of the four elements covered. 3 pts: ≤ 2 elements, or text-only slides without data.
Final demo video	10	10 pts: stable run ≥ 10 s with clear enable/disable sequence (a few seconds shown before and after enabling control). 6 pts: stable run but enable/disable sequence unclear or missing. 2 pts: demo fails but failure is honestly documented. 0 pts: not submitted.
Presentation & Q&A	10	5 pts: ≥ 2 technical questions answered correctly during Q&A. 5 pts: answers are consistent with the submitted materials (slides, video, report). Partial credit applied within each sub-criterion.

List of Appendices

Appendix A Parts List

Appendix B RE-280RA datasheet (Mabuchi Motor)

Appendix C Processing Oscilloscope Manual

Appendix D Circuit example

Appendix E DRV8832 datasheet (Texas Instruments)

Appendix F TPR-105F datasheet

Appendix G LD1117V33 datasheet

実践演習 P5 パーツ内容

実践演習（P5）で利用するパーツのリストです。授業の最初に中身を確認し、足りないものがありましたら、声をかけてください。

1. 中身の確認

- ✓ 下のリストと照合し、すべてのパーツがあることを確認してください。パーツが足りない、あるいは、破損している場合には、声をかけてください。

2. 電池の充電

- ✓ 演習開始前に充電しているはず。

【パーツリスト】

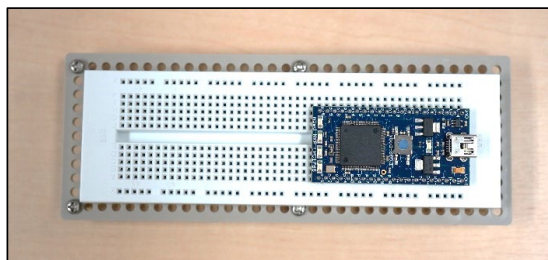
パーツは、演習終了後に返送してもらいますので、壊さない(汚さない)ようにしてください。

****** パーツの入ったビニール袋は返送時に使うので捨てないこと ******



このような箱に入って届きます。
中には、以下の部品が入っています。

(1) ブレッドボード



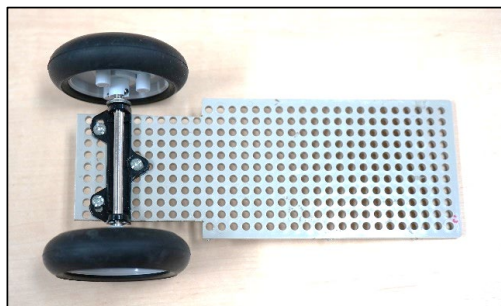
工作用ユニバーサルボードに電子回路用ブレッドボードが貼り付けてあります。ブレッドボード上には、MBED マイコンが設置してあります。**決して、外さないでください。**

ユニバーサルボードに、**ボルト+ナットが4個**ついていることを確認してください。

(2) 車輪付きユニバーサルボード

一部を加工したユニバーサルボードに車輪がついています。

横から見ると、車輪が中央部分でナット固定されているはずです。中央部のナットがあることを確認してください。



(3) 充電電池＋充電器セット



EVOLTA 電池が 4 本入っています。演習の期間の間でつねに充電してください。

(4) 工具(ドライバ、ラジオペンチ)

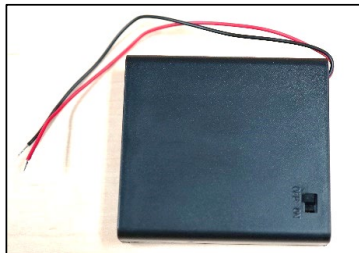


ドライバ（＋）とラジオペンチがビニール袋に入っています。
ビニール袋は返送時にも使いますので、捨てないでください。

(5) 両面テープ



(6) 電池ボックス



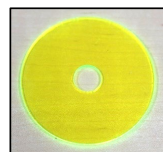
電池を抑えるバネが緩んだり、線が切れたりしている場合は交換します。

(7) USB ケーブル

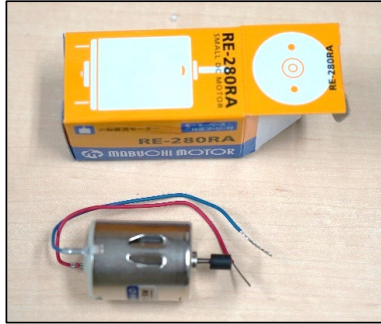


巻き取り式の USB ケーブルです。巻き取るときは、中央ボタン（ELECOM と書いてある銀色部分）を押します。

(8) エンコーダ用プレート → 2 回目の授業で配布します。



(9) モータ



モータは箱から出して中身を確認してください。

モータの軸の先端部分に、黒いローラ（ゴムチューブ）がついてい
ます。ローラが無い人は連絡してください。

(10) モータドライバ IC



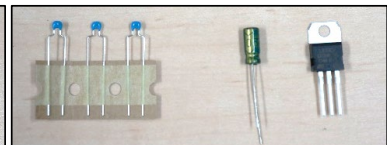
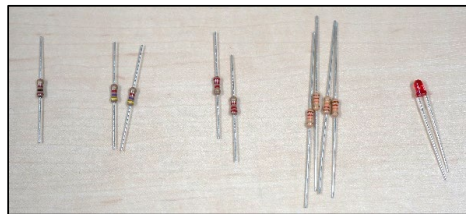
袋の中にモータドライバ IC が入っています。

電子部品は人体の静電気では壊れる場合がありますので、なるべく端子部分に触れないように。

(11) 電気電子部品一式



袋の中に抵抗・コンデンサ・レギュレータ、LED が入っています。
中身を確認してください。



青いコンデンサはテープで
留めていない場合もあります。

抵抗 100Ω 1つ、12KΩ 4つ、2.2KΩ 2つ、4.6KΩ 2つ

コンデンサ 0.1μF（青または茶色）3個、47μF（緑）

赤色 LED （外観は赤いものと、透明があります）

三端子レギュレータ

(12) ジャンパ線



長さの異なるワイヤが全部で 20 本入っています。

こちらは、本数があっていない学生がいるかもしれませんので、
配線時に足りなければ、随時取りに来てください。

→回路作成時、短い線が足りない場合も声をかけてください。

黒 8 本

赤 4 本

白（or 黄色）8 本

（全部は使わないので、2, 3 本足りなくても問題ありません）

(13) フォトインタラプタ(重要)



フォトインタラプタとワイヤを実装したパーツが、1種類入っています。

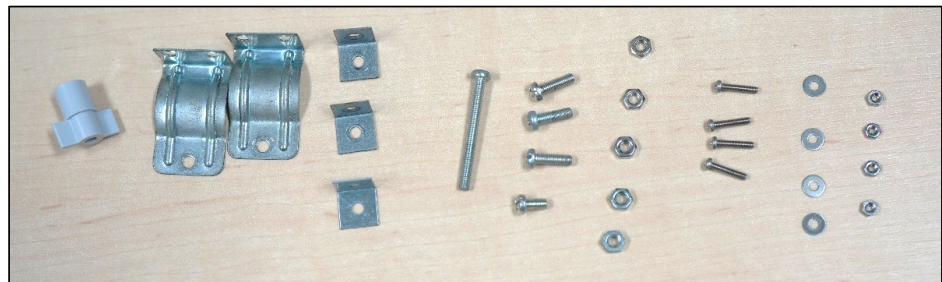
黒いワイヤ（全部で2本あるはず）の先端には抵抗が半田付
してあります。抵抗がない場合は連絡してください。

(15) ネジ, 固定用部品



ビニール袋に以下のパーツが入っています。すべて必要ですので、個数を確認してください。

- ・プラスチック製レンチ×1
- ・モータブラケット金具×2
- ・L字金具×3
- ・M3 鍋ネジ 30mm×1
- ・M3 鍋ネジ 10mm×3 （10mm と 5mm が各 2 本の場合もあります）
- ・M3 鍋ネジ 5mm×1
- ・M3 ナット×5
- ・M2 鍋ネジ 10mm×4
- ・M2 ワッシャ×4
- ・M2 ナット×4





WEIGHT : 42g (APPROX)

RE-280RA/SA



OUTPUT : 0.8W ~ 4.5W (APPROX)

金属ブラシ | Metal-brush motors | 金属电刷

代表的用途

家電機器：マッサージャー／パイプレーター
玩具・模型：玩具／プラモデル

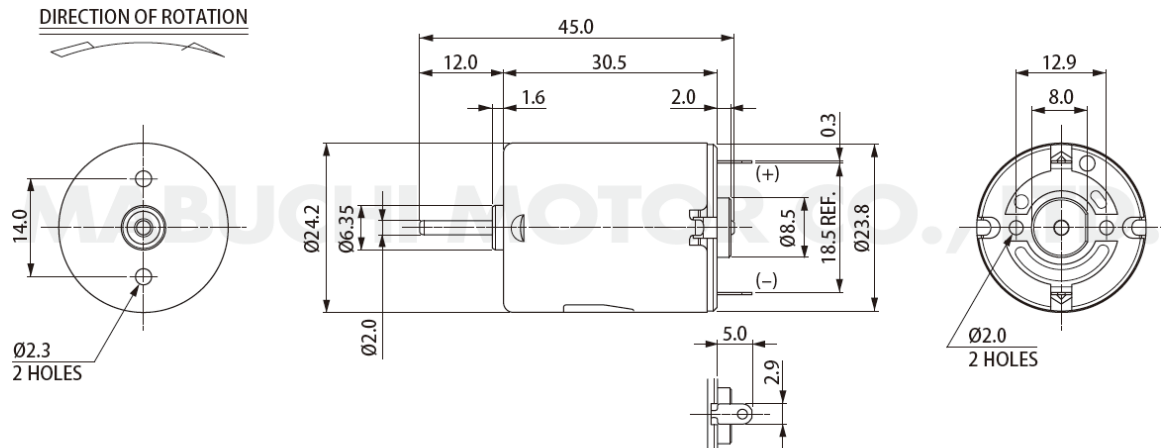
Typical Applications

Home Appliances : Massager / Vibrator
Toys and Models : Motorized Toy / Motorized Plastic Model

主要用途

家電機器：按摩棒
玩具、模型：玩具、組合式模型

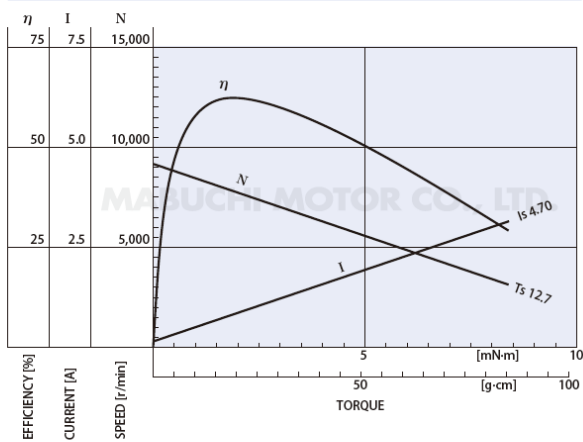
MODEL	VOLTAGE		NO LOAD		AT MAXIMUM EFFICIENCY						STALL	
	OPERATING RANGE	NOMINAL	SPEED	CURRENT	SPEED	CURRENT	TORQUE		OUTPUT	TORQUE		CURRENT
			r/min	A	r/min	A	mN·m	g·cm	W	mN·m	g·cm	A
RE-280RA-2865	1.5~3.0	3V CONSTANT	9200	0.16	7770	0.87	1.98	20.2	1.61	12.7	129	4.70
RE-280RA-2485	1.5~4.5	3V CONSTANT	7000	0.12	5800	0.58	1.68	17.2	1.02	9.81	100	2.80
RE-280SA-2865	1.5~4.5	3V CONSTANT	7100	0.16	6000	0.88	2.58	26.3	1.62	16.7	170	4.80
RE-280SA-2295	3~6	6V CONSTANT	9600	0.14	8150	0.78	3.27	33.3	2.79	21.6	220	4.40



UNIT : MILLIMETERS

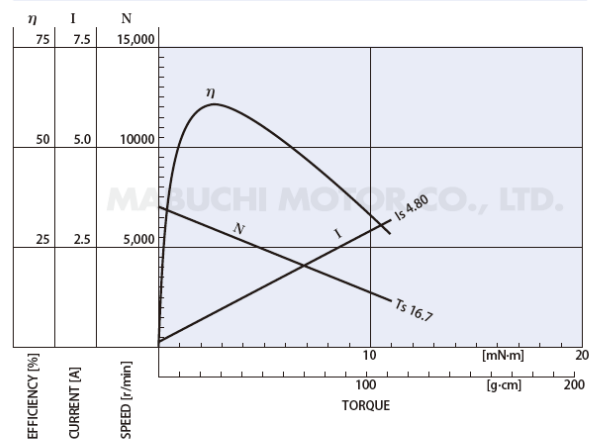
RE-280RA-2865

3.0V



RE-280SA-2865

3.0V



マブチモーター株式会社 (本社 営業部)
MABUCHI MOTOR CO., LTD.
(Headquarters Sales Dept. | 总公司 营业部)

千葉県松戸市松飛台 430 番地 〒270-2280 Tel. 047-710-1106 Fax. 047-710-1132
430 Matsuhidai, Matsudo City, Chiba 270-2280, Japan Tel. 81-47-710-1106 Fax. 81-47-710-1132
E-mail: slsinq@mabuchi-motor.co.jp

簡易オシロスコープの使い方

【事前インストール作業】

1. (Windows のみ) mbed シリアルポートドライバをインストール

<https://os.mbed.com/handbook/Windows-serial-configuration>

こちらのページからドライバをダウンロードし、指示に従ってインストールしてください。

2. Processing のインストール

<https://processing.org/>

こちらから Processing をダウンロードしてインストールしてください (ZIP ファイルを解凍して、好きなフォルダに設置するだけです)。最初から自分のパソコンに Processing が入っている人は、Ver. 3 系であることを確認してください。Ver. 2 系と Ver. 3 系は、一部互換性がありません。

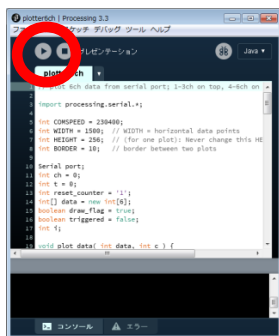
3. オシロスコーププログラムの設置

Google Drive の [plotter6ch](#) の中にある 2 つのファイルをダウンロードします。自分のパソコンの中に (場所はどこでも良いので) plotter6ch という名称のフォルダを作り、その中にダウンロードした 2 個のファイルを入れてください。Processing は、フォルダ名とプログラムファイル名が同じで無いと動作しませんので、フォルダ名を必ず “plotter6ch” としてください。

【使い方】

- Mbed に制御用プログラムをコンパイルしてダウンロードし、パソコンと USB 接続します。plotter6ch.pde (環境により拡張子は表示されないかもしれませんが) をダブルクリックで開き、下図赤丸の部分 (丸に三角形の実行ボタン) を押すと、簡易オシロスコープが立ち上がります。

(【Windows】初回のみ、右クリック→プログラムから開く→Processing)



実行ボタンを押したときに、次ページに示すオシロスコープ画面が出ない人や、エラーで止まってしまう人は、Processing ウィンドウ下部の console 画面をみてください。そこに、パソコンの中にあるシリアルポートの一覧が出ています。例えば、Mac の場合だと

```
[0] "/dev/cu.Bluetooth-Incoming-Port"
[1] "/dev/tty.Bluetooth-Incoming-Port"
[2] "/dev/tty.usbmodem1412"
```

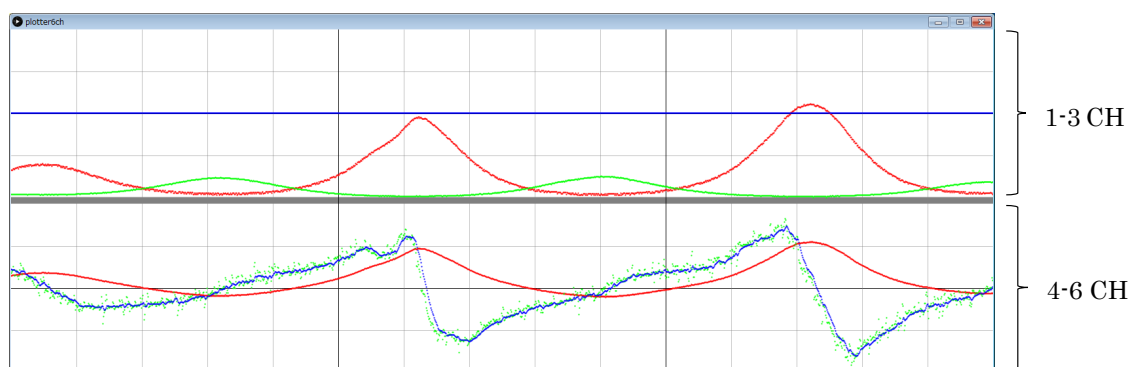
などと表示されますので、"dev/tty.usbmodemXXXX" (XXXX は数字が入る) を探してください。そのポートの番号 (行頭にある[3]などの数字) を、プログラムの 63 行目に書き込みます。

```
port = new Serial( this, Serial.list()[0], COMSPEED);
```

下線部の 0 となっている部分を書き換えてください。

Windows の人は、表示されるポートを上から (0 番から) 順に試してみてください。

- 画面は上下に 2 分割されています。上半分に 1～3 チャンネルが、下半分に 4～6 チャンネルが、それぞれ（赤、緑、青）の順で表示されます。



- オシロスコープが動作している最中に MBED をリセットすることで、チャンネル順序が正しい順序にリセットされます（MBED 側のプログラムで書いた順に 1～6 チャンネルになります）。リセットしていない状況では、チャンネル順序は意図した順序になっていない場合があります。
- 画面上でクリックすると、現在の描画が右端まで進んだところで描画が停止します。また、スクリーンショットが、`data.png` という名前で保存されます（同名ファイルがある場合は上書きされます）。再度画面上でクリックすると、再び画面の更新が始まります。スクリーンショットの保存先は、`plotter6ch` フォルダです。
- 標準では画面の横幅は 1500 点です。MBED プログラムが 2000Hz で動いている場合、画面の横幅は 0.75 秒に相当します。
- 各データは 1 バイト (0-255) のデータとして送受信されています。MBED 側から送信する際には、データを 0-255 の範囲に整えた上で、`char` 型に cast してください。画面表示は、上下それぞれにおいて、一番下が 0、一番上が 255 になります。
- この簡易オシロスコーププログラムは、毎制御周期に必ず 6 個のデータ（かつ、全て 1 バイトデータ）が送られてくるという想定で動いています。MBED 側からは、（必要が無い場合でも）必ず 6 チャンネル分のデータを送ってください。そうしないと、おかしいグラフになります。

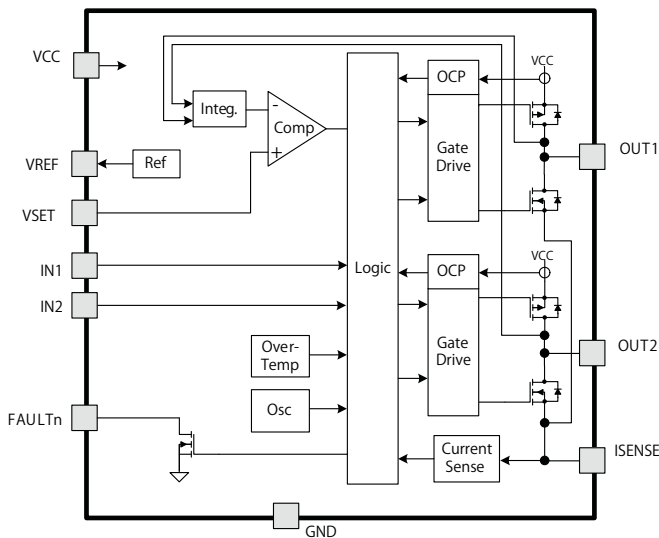
最大連続駆動電流：1A

動作電源電圧範囲：2.75V～6V

DRV8832 低電圧 モータドライバモジュール

特 長

- Hブリッジ電圧制御モータドライバ
 - ー DCモータ、ステッピングモータの1巻線、または他のアクチュエータ/負荷を駆動可能
 - ー 高効率のPWM電圧制御により、電源電圧の変化に対してモータ速度を一定に保持
 - ー 低オン抵抗：ハイサイド+ローサイド=450mΩ
- 最大連続駆動電流：1A
- 動作電源電圧範囲：2.75V～6V
- スリープ・モード時消費電流：300nA (Typ)
- リファレンス電圧出力
- 電流制限回路および障害通知出力

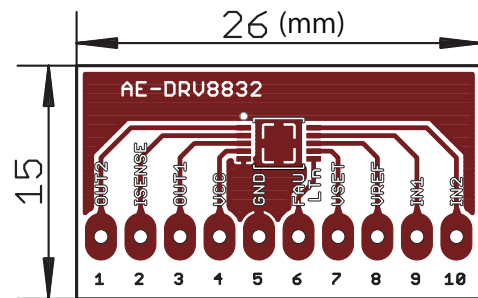


TEXAS INSTRUMENTS 社製 DRV8832使用

DRV8832は、電池駆動の玩具や、プリンタ、その他の低電圧またはバッテリー駆動の動作制御アプリケーションに対して統合されたモータ・ドライバ・ソリューションを提供します。1つのHブリッジ・ドライバを搭載し、1個のDCモータ、またはステッピング・モータの1つの巻線を駆動でき、ソレノイドなどの負荷も駆動できます。出力ドライバ・ブロックはNチャネルおよびPチャネル・パワー・MOSFETで構成され、Hブリッジとしてモータ巻線を駆動します。PCBに十分なヒートシンクが備えられていれば、DRV8832は最大1Aの連続出力電流を供給できます。DRV8832は、2.75V～6Vの電源電圧で動作します。バッテリー寿命を長く保ちながら、バッテリー電圧の変動に対して一定のモータ速度を維持するため、PWM電圧レギュレーション方式が採用されています。レギュレーション電圧は、入力ピンを使用して設定できます。また、電圧リファレンス出力を内蔵しています。

過電流保護、短絡保護、低電圧誤動作防止 (UVLO)、および過熱保護のために、内部保護機能が用意されています。DRV8832は、モータの起動時や強制停止時などにモータ電流を制御する電流制限機能、および外部に障害を通知する出力ピンも備えています。

ピッチ変換基板に実装済みで、2.54mmピッチのユニバーサル基板などに付属のピンヘッダで取り付けできます。



基板パターン図(基板ピン番号)

名前	ピン	I/O ⁽¹⁾	説明	外部部品または接続
GND	5	-	デバイスのグラウンド	
VCC	4	-	デバイスおよびモータの電源	0.1 μF (最小) のセラミック・コンデンサを使用してGNDにバイパスします。
IN1	9	I	ブリッジA入力1	このピンがHighのとき、OUT1がHighになります。
IN2	10	I	ブリッジA入力2	このピンがHighのとき、OUT2がHighになります。
VREF	8	O	リファレンス電圧出力	リファレンス電圧出力です。
VSET	7	I	電圧設定入力	入力電圧によって出力レギュレーション電圧が設定されます。
FAULTn	6	OD	障害通知出力	障害状態が発生するとLowになるオープン・ドレイン出力です。
OUT1	3	O	ブリッジ出力1	モータ巻線に接続します。
OUT2	1	O	ブリッジ出力2	モータ巻線に接続します。
ISENSE	2	IO	電流センス抵抗	GNDとの間に電流センス抵抗を接続します。この抵抗値によって電流制限レベルが設定されます。

(1) 方向 : I = 入力, O = 出力, OZ = 3ステート出力, OD = オープン・ドレイン出力, IO = 入力/出力

DRV8832は、低電圧、過電流、および過熱状態から完全に保護されています。障害発生時にはFAULTnピンがLowになり、外部に通知することができます。

内部チップ温度が約160°Cを超えた場合、デバイスは、温度が安全なレベルに低下するまでディスエーブルとなります。デバイスが過熱シャットダウン状態になる傾向がある場合には、消費電力が過剰であるか、ヒートシンクが不足しているか、または周囲温度が高すぎることを示しています。

消費電力DRV8832の消費電力で大勢を占めるのは、出力FET抵抗 $R_{DS(ON)}$ で消費される電力です。ステッピング・モータを駆動したときの平均消費電力は、おおまかに見積もることができます。ここで、 P_{TOT} は合計消費電力、 $R_{DS(ON)}$ は各FETの抵抗、 $I_{OUT(RMS)}$ は各巻線に流れるRMS出力電流とします。 $I_{OUT(RMS)}$ は、フルスケール出力電流設定×0.7にほぼ等しくなります。

推奨動作条件 (動作温度範囲内)		MIN	NOM	MAX	単位
VCC	モータ電源電圧範囲	2.75		6	V
IOUT	連続Hブリッジ出力電流 ※	0		1	A

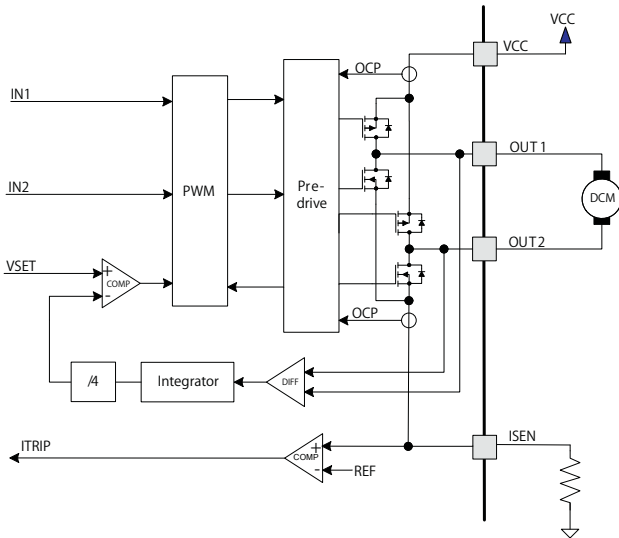
(*消費電力および温度の制限に従う必要があります)

係数の2は、各巻線について任意の時点で2つのFET (ハイサイドとローサイド) に巻線電流が流れているためです。デバイスで消費できる最大電力は、周囲温度およびヒートシンクに依存します。 $R_{DS(ON)}$ は温度とともに増加するため、デバイスの温度が上昇すると、消費電力は増加します。ヒートシンクのサイズを決定する際には、この点を考慮する必要があります。

$$P_{TOT} = 2 \times R_{DS(ON)} \times (I_{OUT(RMS)})^2$$

PWMモータ・ドライバ

DRV8832には、Hブリッジ・モータ・ドライバ、PWM電圧制御回路、および電流制限回路が搭載されています。

**ブリッジ制御**

Hブリッジの出力は、IN1およびIN2制御ビットによってイネーブルになります。

IN1	IN2	OUT1	OUT2	機能
0	0	Z	Z	スタンバイ/惰走
0	1	L	H	逆転
1	0	H	L	正転
1	1	H	H	ブレーキ

両方のビットが0の場合、出力ドライバはディスエーブルになり、デバイスは低電力シャットダウン状態となります。電流制限障害状態が発生していた場合は、クリアされます。

ブレーキまたはスタンバイ・モードから正転または逆転へと遷移する際には、電圧制御PWMがゼロ・デューティ・サイクルから開始されることに注意してください。デューティ・サイクルは徐々に上昇し、指定電圧に到達します。スタンバイから100%デューティ・サイクルに至るまでは最大で12msかかります。そのため、IN1およびIN2ピンに高速PWM信号を印加することはできません。モータ速度を制御するには、後述のようにVSETピンを使用します。

電圧レギュレーション

DRV8832は、モータ巻線に印加される電圧のレギュレーション機能を備えています。この機能により、放電中のバッテリーなど、変動する電源電圧で動作している場合でも、モータ速度を一定に保持できます。

DRV8832は、リニア回路の代わりにPWM（パルス幅変調）回路を使用することで、消費電流を最小限に抑え、バッテリー寿命を長く保持します。

この回路は、出力ピン間の電圧差を監視し、それを積分することで、平均DC電圧値を求めます。この電圧を1/4にした後、VSETピンの電圧と比較します。平均出力電圧の1/4がVSETより小さい場合は、PWM出力のデューティ・サイクルが増加します。平均出力電圧の1/4がVSETより大きい場合は、PWM出力のデューティ・サイクルが減少します。

PWMレギュレーション中は、PWMオン時間の間、Hブリッジによるモータ巻線電流の駆動がイネーブルになります。これは、右図で①として示されています。図中の電流の流れる方向は、IN1 = HighおよびIN2 = Lowのときの状態を示しています。

≧ (VSET × 4) により設定される出力電圧が電源電圧よりも大きい場合、デバイスは100%のデューティ・サイクルで動作し、電圧レギュレーション機能はディスエーブルになります。このモードでは、デバイスは従来型のHブリッジ・ドライバとして動作します。

PWMオフ時間の間は、ブリッジ内の両方のハイサイドFET

をイネーブルにすることで、巻線電流が再循環されます。これは、下図で②として示されています。

リファレンス出力

DRV8832には、モータ電圧の設定に使用できるリファレンス電圧出力が内蔵されています。一般に、定速アプリケーションの場合、VSETはVREFから分圧抵抗を通して駆動され、目的のモータ駆動電圧の1/4に等しい電圧を供給します。

例えば、VREFを直接VSETに接続した場合、電圧は5.14Vにレギュレーションされます。目的のモータ電圧が3Vの場合、VREFは0.75Vとする必要があります。これは、VREF-VSET間に53kΩ、VSET-GND間に75kΩの分圧抵抗を使用することで実現できます。

電流制限

過電流状態の発生時にシステムを保護するために、電流制限回路が搭載されています。これは、DCモータのスタートアップ時、または異常な機械的負荷（強制停止状態）がかった場合などに発生します。

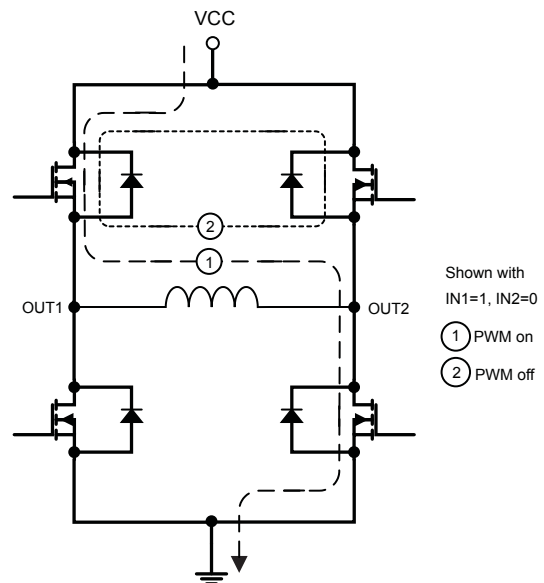
モータ電流は、外部センサ抵抗の両端の電圧を監視することで検知されます。この電圧が200mVのリファレンス電圧を上回る状態が約3ms以上続くと、PWMデューティ・サイクルが減少し、この値になるまでモータ電流を制限します。この電流制限により、電流を制御しながらモータを起動することができます。電流制限状態がある程度長く続く場合は、モータが強制的に停止状態に陥っているなど、なんらかの障害状態が発生している可能性があります。過電流状態が約275ms継続すると、障害状態であると認識します。約275msの経過後、FAULTn出力信号がLowになることで、外部に障害が通知されます。このときモータ・ドライバの動作は続行されます。

この電流制限障害状態は、IN1およびIN2ビットを両方ともLowにしてモータ電流をディスエーブルにするか、またはデバイスへの電源をいったん遮断して再度印加することによりクリアされます。

電流制限の設定に使用する抵抗は、1Ω未満にする必要があります。値は次の式で計算できます。

$$R_{ISENSE} = \frac{200\text{mV}}{I_{LIMIT}}$$

ここで R_{ISENSE} は電流センサ抵抗値です。 I_{LIMIT} は、目的の電流制限値（mA）です。電流制限機能が必要ない場合は、ISENSEピンを直接グランドに接続できます。



本紙記載のデータは、参考データです。ご使用にあたっては、テキサスインスツルメンツ社の最新データをご参照ください。

2013_01

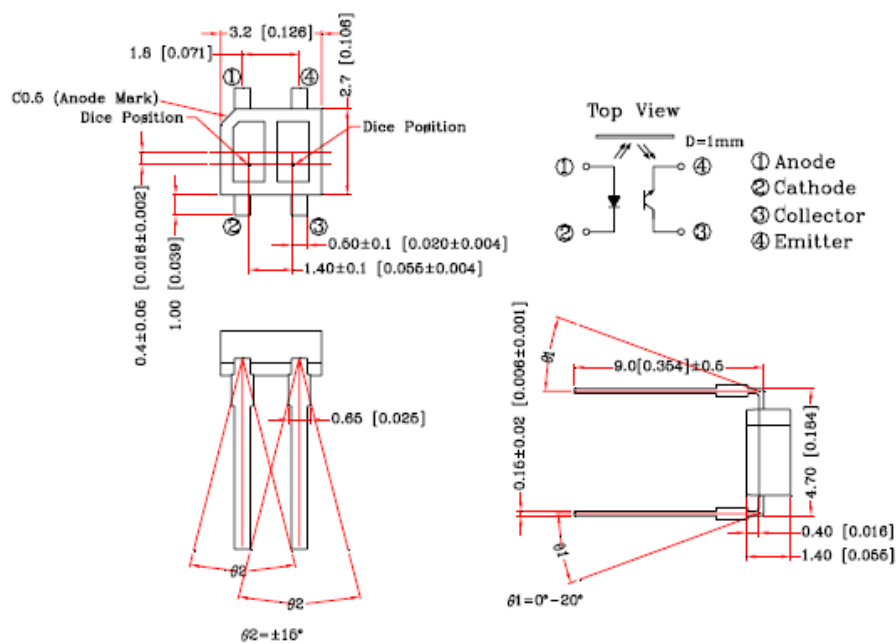


GENIXTEK CORP.

DEVICE NO.: TPR-105F

This photo interrupters is non-contact switching and for direct pc board or dual-in-line socket mounting. It offers Fast switching speed. And this product doesn't contain restriction substance, comply ROHS standard.

PACKAGE DIMENSIONS:



NOTE:

All dimensions are in millimeters
 Tolerance is ± 0.25 mm unless otherwise noted
 Lead spacing is measured where the leads emerge from the package.



GENIXTEK CORP.

DEVICE NO.: TPR-105F

Electrical Optical Characteristics (Ta=25°C)

Parameter		Symbol	Condition	MIN.	TYP.	MAX	UNIT
Input	Forward Voltage	V _F	I _F =20mA	---	1.2	1.5	V
	Reverse Current	I _R	V _R =5V	---	---	100	μ A
	Peak Wavelength	λ _p	I _F =10mA	---	940	---	nm
Output	Dark Current	I _D	V _{CE} =10V	---	---	200	nA
	C-E Saturation Voltage	V _{CE (sat)}	I _C =0.25mA I _F =10mA	---	---	0.4	V
Light Current		I _L	V _{CE} =5V, I _F =10mA, D=1.0MM 90% Reflective white paper	80			μ A
Speed	Rise Time	T _r	I _{FP} =20mA, V _{CE} =5V	---	20	---	μ sec
	Fall Time	T _f	R _L =1000Ω	---	20	---	μ sec

Absolute Maximum Rating (Ta=25°C)

Item		Symbol	Rating	Unit
Input	Power Dissipation	P _d	75	mW
	Reverse Voltage	V _R	5	V
	Forward Current	I _F	50	mA
	Peak Forward Current (*1)	I _{FP}	1	A
Output	Collector Power Dissipation	P _C	100	mW
	Collector Current	I _C	20	mA
	C-E Voltage	V _{CEO}	30	V
	E-C Voltage	V _{ECO}	5	V
Operating Temperature		T _{opr}	-40 ~ +85	°C
Storage Temperature		T _{stg}	-40 ~ +100	°C
Soldering Temperature (*2)		T _{sol}	260	°C

(*1) tw=100 uSec. ∙ T=10 mSec.

(*2) t=3 Sec

※Specifications are subject to change without notice.



GENIXTEK CORP.

DEVICE NO.:TPR-105F

Typical Electro-Optical Characteristics Curves

Fig.1 Power Dissipation vs. Ambient Temperature

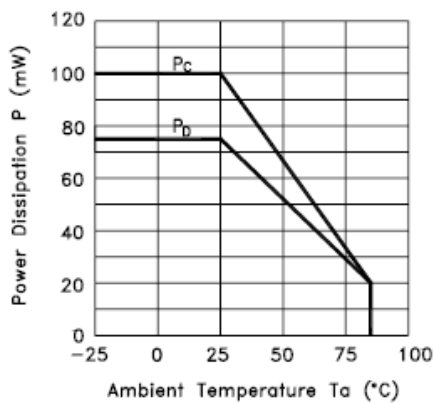


Fig.2 Forward Current vs. Forward Voltage

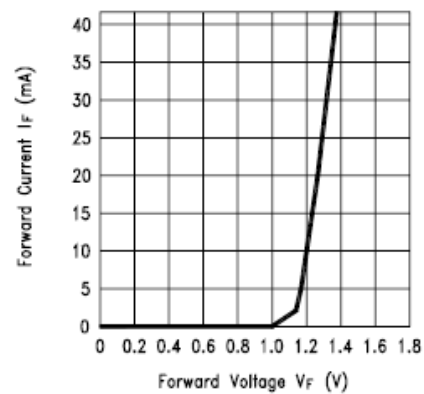


Fig.3 Collector Current vs. Collector-emitter Voltage

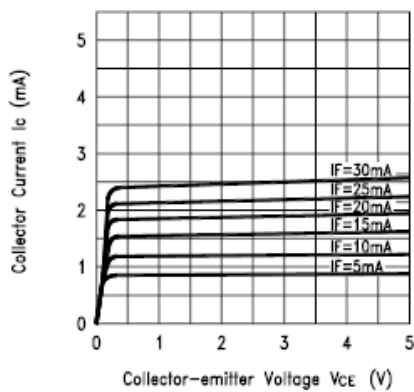
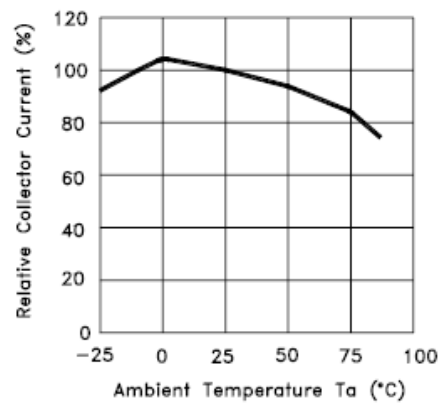


Fig.4 Collector Current vs. Ambient Temperature





GENIXTEK CORP.

DEVICE NO.: TPR-105F

Typical Electro-Optical Characteristics Curves

Fig.5 Collector-emitter Saturation Voltage vs. Ambient Temperature

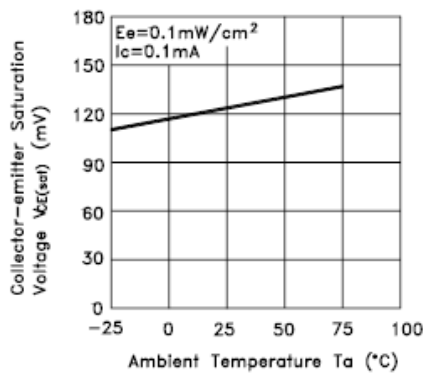


Fig.6 Response Time vs. Load Resistance

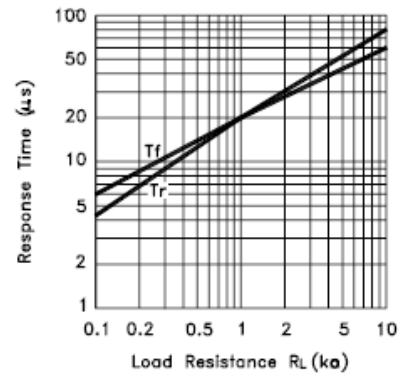
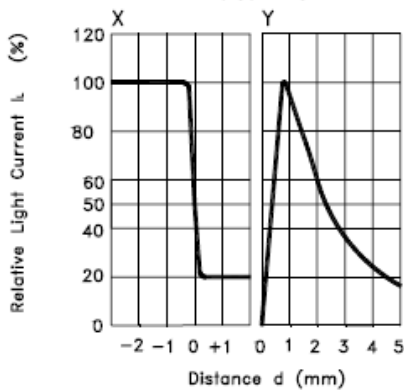
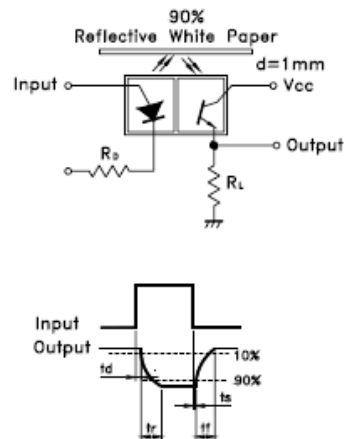


Fig.7 Sensing Position Characteristics (Typical)



Test Circuit for Response Time



TPR-105, TEST REPORT FOR 20K PCS					
TEST CONDITION: VCE: 5V, IF=10mA					
BIN CLASS	CA	D	DB	ttl q'ty	no good
	IL=0.71 - 0.9mA	IL=0.91-1.85mA	IL=1.76mA-2.1mA		
q'ty by pc	1078	17300	2700	21120	42
percentage	5.10%	81.91%	12.78%	100.00%	0.21%

TPR-105F, TEST REPORT FOR 20K PCS					
TEST CONDITION: VCE: 5V, IF=10mA					
BIN CLASS	CA	D	DB	ttl q'ty	no good
	IL=0.71 - 0.9mA	IL=0.91-1.85mA	IL=1.76mA-2.1mA		
q'ty by pc	0	20000	0	20000	0
percentage	0	100%	0	100%	0



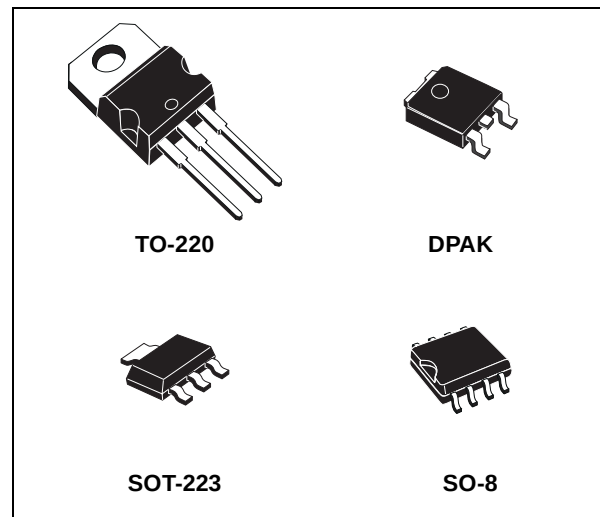
LD1117 SERIES

LOW DROP FIXED AND ADJUSTABLE POSITIVE VOLTAGE REGULATORS

- LOW DROPOUT VOLTAGE (1V TYP.)
- 2.85V DEVICE PERFORMANCES ARE SUITABLE FOR SCSI-2 ACTIVE TERMINATION
- OUTPUT CURRENT UP TO 800 mA
- FIXED OUTPUT VOLTAGE OF: 1.2V, 1.8V, 2.5V, 2.85V, 3.0V, 3.3V, 5.0V
- ADJUSTABLE VERSION AVAILABILITY ($V_{ref}=1.25V$)
- INTERNAL CURRENT AND THERMAL LIMIT
- AVAILABLE IN $\pm 1\%$ (AT 25°C) AND 2% IN FULL TEMPERATURE RANGE
- SUPPLY VOLTAGE REJECTION: 75dB (TYP.)

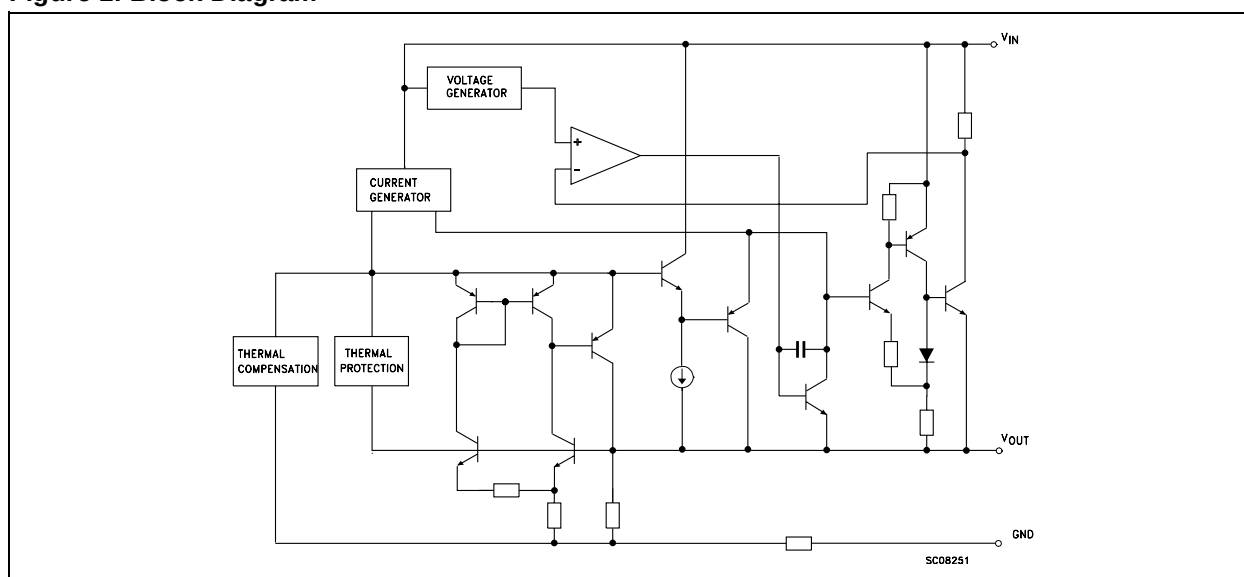
DESCRIPTION

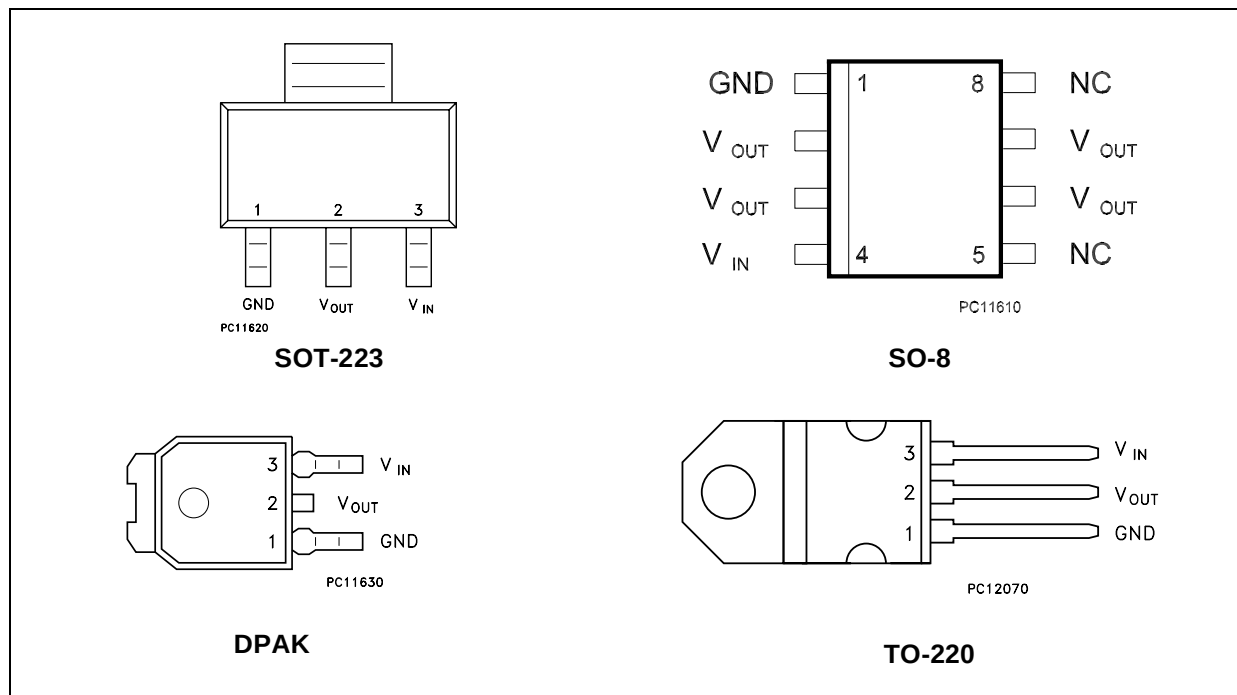
The LD1117 is a LOW DROP Voltage Regulator able to provide up to 800mA of Output Current, available even in adjustable version ($V_{ref}=1.25V$). Concerning fixed versions, are offered the following Output Voltages: 1.2V, 1.8V, 2.5V, 2.85V, 3.0V, 3.3V and 5.0V. The 2.85V type is ideal for SCSI-2 lines active termination. The device is supplied in: SOT-223, DPAK, SO-8 and TO-220. The SOT-223 and DPAK surface mount packages optimize the thermal characteristics even offering a relevant space saving effect. High efficiency is assured by NPN pass transistor. In fact in this



case, unlike than PNP one, the Quiescent Current flows mostly into the load. Only a very common 10 μ F minimum capacitor is needed for stability. On chip trimming allows the regulator to reach a very tight output voltage tolerance, within $\pm 1\%$ at 25°C. The ADJUSTABLE LD1117 is pin to pin compatible with the other standard. Adjustable voltage regulators maintaining the better performances in terms of Drop and Tolerance.

Figure 1: Block Diagram



LD1117 SERIES**Figure 2: Pin Connection (top view)**NOTE: The TAB is connected to the V_{OUT}.**Table 1: Order Codes**

SOT-223	SO-8	DPAK	DPAK (T&R)	TO-220	OUTPUT VOLTAGE
LD1117S12TR	LD1117D12TR (*)	LD1117DT12 (*)	LD1117DT12TR	LD1117V12 (*)	1.2 V
LD1117S12CTR (*)	LD1117D12CTR (*)	LD1117DT12C (*)		LD1117V12C (*)	1.2 V
LD1117S18TR	LD1117D18TR (*)	LD1117DT18	LD1117DT18TR	LD1117V18	1.8 V
LD1117S18CTR (*)	LD1117D18CTR (*)	LD1117DT18C	LD1117DT18CTR	LD1117V18C (*)	1.8 V
LD1117S25TR	LD1117D25TR (*)	LD1117DT25	LD1117DT25TR	LD1117V25	2.5 V
LD1117S25CTR	LD1117D25CTR (*)	LD1117DT25C	LD1117DT25CTR	LD1117V25C	2.5 V
LD1117S28TR	LD1117D28TR (*)		LD1117DT28TR		2.85 V
LD1117S30TR	LD1117D30TR (*)				3 V
LD1117S33TR	LD1117D33TR	LD1117DT33	LD1117DT33TR	LD1117V33	3.3 V
LD1117S33CTR	LD1117D33CTR	LD1117DT33C	LD1117DT33CTR	LD1117V33C	3.3 V
LD1117S50TR	LD1117D50TR	LD1117DT50	LD1117DT50TR	LD1117V50	5 V
LD1117S50CTR	LD1117D50CTR (*)	LD1117DT50C	LD1117DT50CTR		5 V
LD1117STR	LD1117DTR (*)	LD1117DT	LD1117DTTR	LD1117V	ADJ FROM 1.25 TO 15V
LD1117SC-R	LD1117DC-R (*)	LD1117DTC (*)	LD1117DTC-R	LD1117VC (*)	ADJ FROM 1.25 TO 15V

(*) Available on request

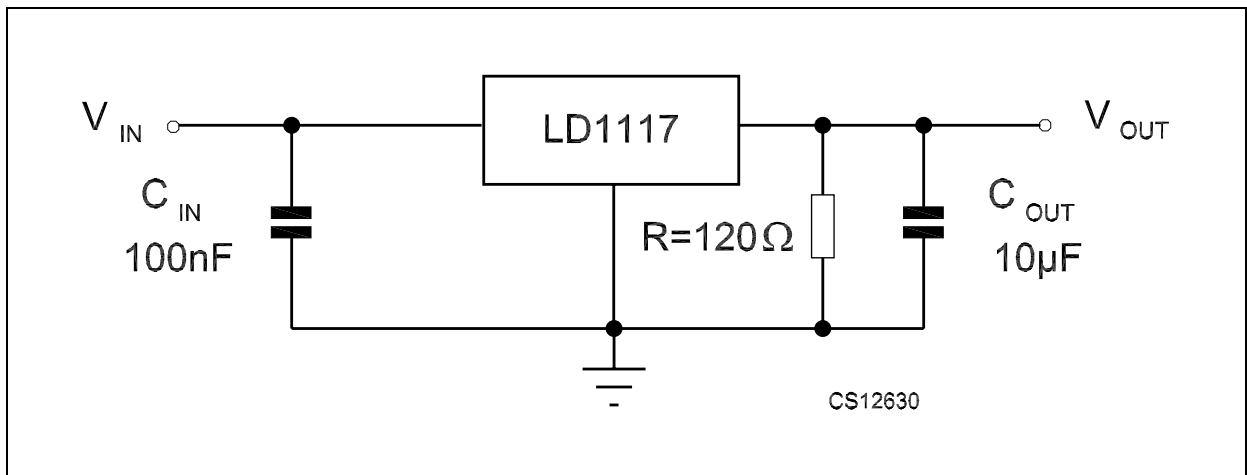
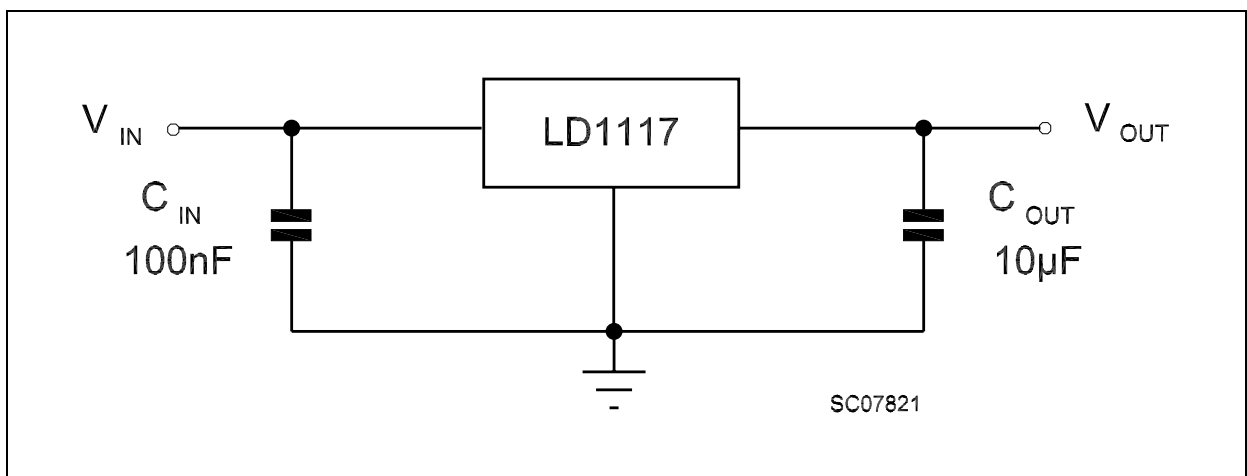
LD1117 SERIES**Table 2: Absolute Maximum Ratings**

Symbol	Parameter		Value	Unit
V_{IN}	DC Input Voltage		15	V
P_{tot}	Power Dissipation		12	W
T_{stg}	Storage Temperature Range		-40 to +150	°C
T_{op}	Operating Junction Temperature Range	for C Version	-40 to +150	°C
		for standard Version	0 to +150	°C

Absolute Maximum Ratings are those values beyond which damage to the device may occur. Functional operation under these condition is not implied. Over the above suggested Max Power Dissipation a Short Circuit could definitively damage the device.

Table 3: Thermal Data

Symbol	Parameter	SOT-223	SO-8	DPAK	TO-220	Unit
$R_{thj-case}$	Thermal Resistance Junction-case	15	20	8	3	°C/W
$R_{thj-amb}$	Thermal Resistance Junction-ambient				50	°C/W

Figure 3: Application Circuit (FOR 1.2 V)**Figure 4: Application Circuit (FOR OTHER FIXED OUTPUT VOLTAGES)**

LD1117 SERIES**Table 4: Electrical Characteristics Of LD1117#12** (refer to the test circuits, $T_J = 0$ to 125°C , $C_O = 10\ \mu\text{F}$, $R = 120\ \Omega$ between GND and OUT pins, unless otherwise specified)

Symbol	Parameter	Test Conditions	Min.	Typ.	Max.	Unit
V_O	Output Voltage	$V_{in} = 3.2\ \text{V}$ $I_O = 10\ \text{mA}$ $T_J = 25^\circ\text{C}$	1.188	1.20	1.212	V
V_O	Reference Voltage	$I_O = 10$ to $800\ \text{mA}$ $V_{in} - V_O = 1.4$ to $10\ \text{V}$	1.140	1.20	1.260	V
ΔV_O	Line Regulation	$V_{in} - V_O = 1.5$ to $13.75\ \text{V}$ $I_O = 10\ \text{mA}$		0.035	0.2	%
ΔV_O	Load Regulation	$V_{in} - V_O = 3\ \text{V}$ $I_O = 10$ to $800\ \text{mA}$		0.1	0.4	%
ΔV_O	Temperature Stability			0.5		%
ΔV_O	Long Term Stability	1000 hrs, $T_J = 125^\circ\text{C}$		0.3		%
V_{in}	Operating Input Voltage				15	V
I_{adj}	Adjustment Pin Current	$V_{in} \leq 15\ \text{V}$		60	120	μA
ΔI_{adj}	Adjustment Pin Current Change	$V_{in} - V_O = 1.4$ to $10\ \text{V}$ $I_O = 10$ to $800\ \text{mA}$		1	5	μA
$I_{O(min)}$	Minimum Load Current	$V_{in} = 15\ \text{V}$		2	5	mA
I_O	Output Current	$V_{in} - V_O = 5\ \text{V}$ $T_J = 25^\circ\text{C}$	800	950	1300	mA
eN	Output Noise (% V_O)	$B = 10\text{Hz}$ to 10KHz $T_J = 25^\circ\text{C}$		0.003		%
SVR	Supply Voltage Rejection	$I_O = 40\ \text{mA}$ $f = 120\text{Hz}$ $T_J = 25^\circ\text{C}$ $V_{in} - V_O = 3\ \text{V}$ $V_{ripple} = 1\ V_{PP}$	60	75		dB
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$		1	1.1	V
		$I_O = 500\ \text{mA}$		1.05	1.15	
		$I_O = 800\ \text{mA}$		1.10	1.2	
	Thermal Regulation	$T_a = 25^\circ\text{C}$ 30ms Pulse		0.01	0.1	%/W

Table 5: Electrical Characteristics Of LD1117#18 (refer to the test circuits, $T_J = 0$ to 125°C , $C_O = 10\ \mu\text{F}$ unless otherwise specified)

Symbol	Parameter	Test Conditions	Min.	Typ.	Max.	Unit
V_O	Output Voltage	$V_{in} = 3.8\ \text{V}$ $I_O = 10\ \text{mA}$ $T_J = 25^\circ\text{C}$	1.78	1.8	1.82	V
V_O	Output Voltage	$I_O = 0$ to $800\ \text{mA}$ $V_{in} = 3.3$ to $8\ \text{V}$	1.76		1.84	V
ΔV_O	Line Regulation	$V_{in} = 3.3$ to $8\ \text{V}$ $I_O = 0\ \text{mA}$		1	6	mV
ΔV_O	Load Regulation	$V_{in} = 3.3\ \text{V}$ $I_O = 0$ to $800\ \text{mA}$		1	10	mV
ΔV_O	Temperature Stability			0.5		%
ΔV_O	Long Term Stability	1000 hrs, $T_J = 125^\circ\text{C}$		0.3		%
V_{in}	Operating Input Voltage	$I_O = 100\ \text{mA}$			10	V
I_d	Quiescent Current	$V_{in} \leq 8\ \text{V}$		5	10	mA
I_O	Output Current	$V_{in} = 6.8\ \text{V}$ $T_J = 25^\circ\text{C}$	800	950	1300	mA
eN	Output Noise Voltage	$B = 10\text{Hz}$ to 10KHz $T_J = 25^\circ\text{C}$		100		μV
SVR	Supply Voltage Rejection	$I_O = 40\ \text{mA}$ $f = 120\text{Hz}$ $T_J = 25^\circ\text{C}$ $V_{in} = 5.5\ \text{V}$ $V_{ripple} = 1\ V_{PP}$	60	75		dB
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$		1	1.1	V
		$I_O = 500\ \text{mA}$		1.05	1.15	
		$I_O = 800\ \text{mA}$		1.10	1.2	
	Thermal Regulation	$T_a = 25^\circ\text{C}$ 30ms Pulse		0.01	0.1	%/W

LD1117 SERIES**Table 6: Electrical Characteristics Of LD1117#25** (refer to the test circuits, $T_J = 0$ to 125°C , $C_O = 10\ \mu\text{F}$ unless otherwise specified)

Symbol	Parameter	Test Conditions	Min.	Typ.	Max.	Unit
V_O	Output Voltage	$V_{in} = 4.5\ \text{V}$ $I_O = 10\ \text{mA}$ $T_J = 25^\circ\text{C}$	2.475	2.5	2.525	V
V_O	Output Voltage	$I_O = 0$ to $800\ \text{mA}$ $V_{in} = 3.9$ to $10\ \text{V}$	2.45		2.55	V
ΔV_O	Line Regulation	$V_{in} = 3.9$ to $10\ \text{V}$ $I_O = 0\ \text{mA}$		1	6	mV
ΔV_O	Load Regulation	$V_{in} = 3.9\ \text{V}$ $I_O = 0$ to $800\ \text{mA}$		1	10	mV
ΔV_O	Temperature Stability			0.5		%
ΔV_O	Long Term Stability	1000 hrs, $T_J = 125^\circ\text{C}$		0.3		%
V_{in}	Operating Input Voltage	$I_O = 100\ \text{mA}$			15	V
I_d	Quiescent Current	$V_{in} \leq 10\ \text{V}$		5	10	mA
I_O	Output Current	$V_{in} = 7.5\ \text{V}$ $T_J = 25^\circ\text{C}$	800	950	1300	mA
eN	Output Noise Voltage	$B = 10\text{Hz to } 10\text{KHz}$ $T_J = 25^\circ\text{C}$		100		μV
SVR	Supply Voltage Rejection	$I_O = 40\ \text{mA}$ $f = 120\text{Hz}$ $T_J = 25^\circ\text{C}$ $V_{in} = 5.5\ \text{V}$ $V_{\text{ripple}} = 1\ \text{V}_{PP}$	60	75		dB
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$		1	1.1	V
		$I_O = 500\ \text{mA}$		1.05	1.15	
		$I_O = 800\ \text{mA}$		1.10	1.2	
	Thermal Regulation	$T_a = 25^\circ\text{C}$ 30ms Pulse		0.01	0.1	%/W

Table 7: Electrical Characteristics Of LD1117#28 (refer to the test circuits, $T_J = 0$ to 125°C , $C_O = 10\ \mu\text{F}$ unless otherwise specified)

Symbol	Parameter	Test Conditions	Min.	Typ.	Max.	Unit
V_O	Output Voltage	$V_{in} = 4.85\ \text{V}$ $I_O = 10\ \text{mA}$ $T_J = 25^\circ\text{C}$	2.82	2.85	2.88	V
V_O	Output Voltage	$I_O = 0$ to $800\ \text{mA}$ $V_{in} = 4.25$ to $10\ \text{V}$	2.79		2.91	V
ΔV_O	Line Regulation	$V_{in} = 4.25$ to $10\ \text{V}$ $I_O = 0\ \text{mA}$		1	6	mV
ΔV_O	Load Regulation	$V_{in} = 4.25\ \text{V}$ $I_O = 0$ to $800\ \text{mA}$		1	10	mV
ΔV_O	Temperature Stability			0.5		%
ΔV_O	Long Term Stability	1000 hrs, $T_J = 125^\circ\text{C}$		0.3		%
V_{in}	Operating Input Voltage	$I_O = 100\ \text{mA}$			15	V
I_d	Quiescent Current	$V_{in} \leq 10\ \text{V}$		5	10	mA
I_O	Output Current	$V_{in} = 7.85\ \text{V}$ $T_J = 25^\circ\text{C}$	800	950	1300	mA
eN	Output Noise Voltage	$B = 10\text{Hz to } 10\text{KHz}$ $T_J = 25^\circ\text{C}$		100		μV
SVR	Supply Voltage Rejection	$I_O = 40\ \text{mA}$ $f = 120\text{Hz}$ $T_J = 25^\circ\text{C}$ $V_{in} = 5.85\ \text{V}$ $V_{\text{ripple}} = 1\ \text{V}_{PP}$	60	75		dB
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$		1	1.1	V
		$I_O = 500\ \text{mA}$		1.05	1.15	
		$I_O = 800\ \text{mA}$		1.10	1.2	
	Thermal Regulation	$T_a = 25^\circ\text{C}$ 30ms Pulse		0.01	0.1	%/W

LD1117 SERIES**Table 8: Electrical Characteristics Of LD1117#30** (refer to the test circuits, $T_J = 0$ to 125°C , $C_O = 10\ \mu\text{F}$ unless otherwise specified)

Symbol	Parameter	Test Conditions	Min.	Typ.	Max.	Unit
V_O	Output Voltage	$V_{in} = 5\ \text{V}$ $I_O = 10\ \text{mA}$ $T_J = 25^\circ\text{C}$	2.97	3	3.03	V
V_O	Output Voltage	$I_O = 0$ to $800\ \text{mA}$ $V_{in} = 4.5$ to $10\ \text{V}$	2.94		3.06	V
ΔV_O	Line Regulation	$V_{in} = 4.5$ to $12\ \text{V}$ $I_O = 0\ \text{mA}$		1	6	mV
ΔV_O	Load Regulation	$V_{in} = 4.5\ \text{V}$ $I_O = 0$ to $800\ \text{mA}$		1	10	mV
ΔV_O	Temperature Stability			0.5		%
ΔV_O	Long Term Stability	1000 hrs, $T_J = 125^\circ\text{C}$		0.3		%
V_{in}	Operating Input Voltage	$I_O = 100\ \text{mA}$			15	V
I_d	Quiescent Current	$V_{in} \leq 12\ \text{V}$		5	10	mA
I_O	Output Current	$V_{in} = 8\ \text{V}$ $T_J = 25^\circ\text{C}$	800	950	1300	mA
eN	Output Noise Voltage	$B = 10\text{Hz}$ to 10KHz $T_J = 25^\circ\text{C}$		100		μV
SVR	Supply Voltage Rejection	$I_O = 40\ \text{mA}$ $f = 120\text{Hz}$ $T_J = 25^\circ\text{C}$ $V_{in} = 6\ \text{V}$ $V_{\text{ripple}} = 1\ \text{V}_{PP}$	60	75		dB
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$		1	1.1	V
		$I_O = 500\ \text{mA}$		1.05	1.15	
		$I_O = 800\ \text{mA}$		1.10	1.2	
	Thermal Regulation	$T_a = 25^\circ\text{C}$ 30ms Pulse		0.01	0.1	%/W

Table 9: Electrical Characteristics Of LD1117#33 (refer to the test circuits, $T_J = 0$ to 125°C , $C_O = 10\ \mu\text{F}$ unless otherwise specified)

Symbol	Parameter	Test Conditions	Min.	Typ.	Max.	Unit
V_O	Output Voltage	$V_{in} = 5.3\ \text{V}$ $I_O = 10\ \text{mA}$ $T_J = 25^\circ\text{C}$	3.267	3.3	3.333	V
V_O	Output Voltage	$I_O = 0$ to $800\ \text{mA}$ $V_{in} = 4.75$ to $10\ \text{V}$	3.235		3.365	V
ΔV_O	Line Regulation	$V_{in} = 4.75$ to $15\ \text{V}$ $I_O = 0\ \text{mA}$		1	6	mV
ΔV_O	Load Regulation	$V_{in} = 4.75\ \text{V}$ $I_O = 0$ to $800\ \text{mA}$		1	10	mV
ΔV_O	Temperature Stability			0.5		%
ΔV_O	Long Term Stability	1000 hrs, $T_J = 125^\circ\text{C}$		0.3		%
V_{in}	Operating Input Voltage	$I_O = 100\ \text{mA}$			15	V
I_d	Quiescent Current	$V_{in} \leq 15\ \text{V}$		5	10	mA
I_O	Output Current	$V_{in} = 8.3\ \text{V}$ $T_J = 25^\circ\text{C}$	800	950	1300	mA
eN	Output Noise Voltage	$B = 10\text{Hz}$ to 10KHz $T_J = 25^\circ\text{C}$		100		μV
SVR	Supply Voltage Rejection	$I_O = 40\ \text{mA}$ $f = 120\text{Hz}$ $T_J = 25^\circ\text{C}$ $V_{in} = 6.3\ \text{V}$ $V_{\text{ripple}} = 1\ \text{V}_{PP}$	60	75		dB
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$		1	1.1	V
		$I_O = 500\ \text{mA}$		1.05	1.15	
		$I_O = 800\ \text{mA}$		1.10	1.2	
	Thermal Regulation	$T_a = 25^\circ\text{C}$ 30ms Pulse		0.01	0.1	%/W

LD1117 SERIES**Table 10: Electrical Characteristics Of LD1117#50** (refer to the test circuits, $T_J = 0$ to 125°C , $C_O = 10\ \mu\text{F}$ unless otherwise specified)

Symbol	Parameter	Test Conditions	Min.	Typ.	Max.	Unit
V_O	Output Voltage	$V_{in} = 7\ \text{V}$ $I_O = 10\ \text{mA}$ $T_J = 25^\circ\text{C}$	4.95	5	5.05	V
V_O	Output Voltage	$I_O = 0$ to $800\ \text{mA}$ $V_{in} = 6.5$ to $15\ \text{V}$	4.9		5.1	V
ΔV_O	Line Regulation	$V_{in} = 6.5$ to $15\ \text{V}$ $I_O = 0\ \text{mA}$		1	10	mV
ΔV_O	Load Regulation	$V_{in} = 6.5\ \text{V}$ $I_O = 0$ to $800\ \text{mA}$		1	15	mV
ΔV_O	Temperature Stability			0.5		%
ΔV_O	Long Term Stability	1000 hrs, $T_J = 125^\circ\text{C}$		0.3		%
V_{in}	Operating Input Voltage	$I_O = 100\ \text{mA}$			15	V
I_d	Quiescent Current	$V_{in} \leq 15\ \text{V}$		5	10	mA
I_O	Output Current	$V_{in} = 10\ \text{V}$ $T_J = 25^\circ\text{C}$	800	950	1300	mA
eN	Output Noise Voltage	$B = 10\text{Hz}$ to 10KHz $T_J = 25^\circ\text{C}$		100		μV
SVR	Supply Voltage Rejection	$I_O = 40\ \text{mA}$ $f = 120\text{Hz}$ $T_J = 25^\circ\text{C}$ $V_{in} = 8\ \text{V}$ $V_{\text{ripple}} = 1\ \text{V}_{PP}$	60	75		dB
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$		1	1.1	V
		$I_O = 500\ \text{mA}$		1.05	1.15	
		$I_O = 800\ \text{mA}$		1.10	1.2	
	Thermal Regulation	$T_a = 25^\circ\text{C}$ 30ms Pulse		0.01	0.1	%/W

Table 11: Electrical Characteristics Of LD1117 (ADJUSTABLE) (refer to the test circuits, $T_J = 0$ to 125°C , $C_O = 10\ \mu\text{F}$ unless otherwise specified)

Symbol	Parameter	Test Conditions	Min.	Typ.	Max.	Unit
V_{ref}	Reference Voltage	$V_{in} - V_O = 2\ \text{V}$ $I_O = 10\ \text{mA}$ $T_J = 25^\circ\text{C}$	1.238	1.25	1.262	V
V_{ref}	Reference Voltage	$I_O = 10$ to $800\ \text{mA}$ $V_{in} - V_O = 1.4$ to $10\ \text{V}$	1.225		1.275	V
ΔV_O	Line Regulation	$V_{in} - V_O = 1.5$ to $13.75\ \text{V}$ $I_O = 10\ \text{mA}$		0.035	0.2	%
ΔV_O	Load Regulation	$V_{in} - V_O = 3\ \text{V}$ $I_O = 10$ to $800\ \text{mA}$		0.1	0.4	%
ΔV_O	Temperature Stability			0.5		%
ΔV_O	Long Term Stability	1000 hrs, $T_J = 125^\circ\text{C}$		0.3		%
V_{in}	Operating Input Voltage				15	V
I_{adj}	Adjustment Pin Current	$V_{in} \leq 15\ \text{V}$		60	120	μA
ΔI_{adj}	Adjustment Pin Current Change	$V_{in} - V_O = 1.4$ to $10\ \text{V}$ $I_O = 10$ to $800\ \text{mA}$		1	5	μA
$I_{O(min)}$	Minimum Load Current	$V_{in} = 15\ \text{V}$		2	5	mA
I_O	Output Current	$V_{in} - V_O = 5\ \text{V}$ $T_J = 25^\circ\text{C}$	800	950	1300	mA
eN	Output Noise ($\%V_O$)	$B = 10\text{Hz}$ to 10KHz $T_J = 25^\circ\text{C}$		0.003		%
SVR	Supply Voltage Rejection	$I_O = 40\ \text{mA}$ $f = 120\text{Hz}$ $T_J = 25^\circ\text{C}$ $V_{in} - V_O = 3\ \text{V}$ $V_{\text{ripple}} = 1\ \text{V}_{PP}$	60	75		dB
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$		1	1.1	V
		$I_O = 500\ \text{mA}$		1.05	1.15	
		$I_O = 800\ \text{mA}$		1.10	1.2	
	Thermal Regulation	$T_a = 25^\circ\text{C}$ 30ms Pulse		0.01	0.1	%/W

LD1117 SERIES**Table 12: Electrical Characteristics Of LD1117#12C** (refer to the test circuits, $T_J = 0$ to 125°C , $C_O = 10\ \mu\text{F}$, $R = 120\ \Omega$ between GND and OUT pins, unless otherwise specified)

Symbol	Parameter	Test Conditions	Min.	Typ.	Max.	Unit
V_{ref}	Reference Voltage	$V_{\text{in}} - V_O = 2\text{V}$ $I_O = 10\text{ mA}$ $T_J = 25^\circ\text{C}$	1.176	1.20	1.224	V
V_{ref}	Reference Voltage	$I_O = 10$ to 800 mA $V_{\text{in}} - V_O = 1.4$ to 10 V	1.120	1.20	1.280	V
ΔV_O	Line Regulation	$V_{\text{in}} - V_O = 1.5$ to 13.75 V $I_O = 10\text{ mA}$			1	%
ΔV_O	Load Regulation	$V_{\text{in}} - V_O = 3\text{ V}$ $I_O = 10$ to 800 mA			1	%
ΔV_O	Temperature Stability			0.5		%
ΔV_O	Long Term Stability	1000 hrs, $T_J = 125^\circ\text{C}$		0.3		%
V_{in}	Operating Input Voltage				15	V
I_{adj}	Adjustment Pin Current	$V_{\text{in}} \leq 15\text{ V}$		60	120	μA
ΔI_{adj}	Adjustment Pin Current Change	$V_{\text{in}} - V_O = 1.4$ to 10 V $I_O = 10$ to 800 mA		1	5	μA
$I_{O(\text{min})}$	Minimum Load Current	$V_{\text{in}} = 15\text{ V}$		2	5	mA
I_O	Output Current	$V_{\text{in}} - V_O = 5\text{ V}$ $T_J = 25^\circ\text{C}$	800	950	1300	mA
eN	Output Noise (% V_O)	$B = 10\text{Hz}$ to 10KHz $T_J = 25^\circ\text{C}$		0.003		%
SVR	Supply Voltage Rejection	$I_O = 40\text{ mA}$ $f = 120\text{Hz}$ $T_J = 25^\circ\text{C}$ $V_{\text{in}} - V_O = 3\text{ V}$ $V_{\text{ripple}} = 1\text{ V}_{\text{PP}}$	60	75		dB
V_d	Dropout Voltage	$I_O = 100\text{ mA}$ $T_J = 0$ to 125°C		1	1.1	V
		$I_O = 500\text{ mA}$ $T_J = 0$ to 125°C		1.05	1.2	
		$I_O = 800\text{ mA}$ $T_J = 0$ to 125°C		1.10	1.3	
	Thermal Regulation	$T_a = 25^\circ\text{C}$ 30ms Pulse		0.01	0.1	%/W

LD1117 SERIES**Table 13: Electrical Characteristics Of LD1117#18C** (refer to the test circuits, $T_J = -40$ to 125°C , $C_O = 10\ \mu\text{F}$ unless otherwise specified)

Symbol	Parameter	Test Conditions	Min.	Typ.	Max.	Unit
V_O	Output Voltage	$V_{in} = 3.8\ \text{V}$ $I_O = 10\ \text{mA}$ $T_J = 25^\circ\text{C}$	1.76	1.8	1.84	V
V_O	Output Voltage	$I_O = 0$ to $800\ \text{mA}$ $V_{in} = 3.9$ to $10\ \text{V}$	1.73		1.87	V
ΔV_O	Line Regulation	$V_{in} = 3.3$ to $8\ \text{V}$ $I_O = 0\ \text{mA}$		1	30	mV
ΔV_O	Load Regulation	$V_{in} = 3.3\ \text{V}$ $I_O = 0$ to $800\ \text{mA}$		1	30	mV
ΔV_O	Temperature Stability			0.5		%
ΔV_O	Long Term Stability	1000 hrs, $T_J = 125^\circ\text{C}$		0.3		%
V_{in}	Operating Input Voltage	$I_O = 100\ \text{mA}$			10	V
I_d	Quiescent Current	$V_{in} \leq 8\ \text{V}$		5	10	mA
I_O	Output Current	$V_{in} = 6.8\ \text{V}$ $T_J = 25^\circ\text{C}$	800	950	1300	mA
eN	Output Noise Voltage	$B = 10\text{Hz}$ to 10KHz $T_J = 25^\circ\text{C}$		100		μV
SVR	Supply Voltage Rejection	$I_O = 40\ \text{mA}$ $f = 120\text{Hz}$ $T_J = 25^\circ\text{C}$ $V_{in} = 5.5\ \text{V}$ $V_{\text{ripple}} = 1\ \text{V}_{PP}$	60	75		dB
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$ $T_J = 0$ to 125°C		1	1.1	V
		$I_O = 500\ \text{mA}$ $T_J = 0$ to 125°C		1.05	1.15	
		$I_O = 800\ \text{mA}$ $T_J = 0$ to 125°C		1.10	1.2	
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$			1.1	V
		$I_O = 500\ \text{mA}$			1.2	
		$I_O = 800\ \text{mA}$			1.3	
	Thermal Regulation	$T_a = 25^\circ\text{C}$ 30ms Pulse		0.01	0.1	%/W

LD1117 SERIES**Table 14: Electrical Characteristics Of LD1117#25C** (refer to the test circuits, $T_J = -40$ to 125°C , $C_O = 10\ \mu\text{F}$ unless otherwise specified)

Symbol	Parameter	Test Conditions	Min.	Typ.	Max.	Unit
V_O	Output Voltage	$V_{in} = 4.5\ \text{V}$ $I_O = 10\ \text{mA}$ $T_J = 25^\circ\text{C}$	2.45	2.5	2.55	V
V_O	Output Voltage	$I_O = 0$ to $800\ \text{mA}$ $V_{in} = 3.9$ to $10\ \text{V}$	2.4		2.6	V
ΔV_O	Line Regulation	$V_{in} = 3.9$ to $10\ \text{V}$ $I_O = 0\ \text{mA}$		1	30	mV
ΔV_O	Load Regulation	$V_{in} = 3.9\ \text{V}$ $I_O = 0$ to $800\ \text{mA}$		1	30	mV
ΔV_O	Temperature Stability			0.5		%
ΔV_O	Long Term Stability	1000 hrs, $T_J = 125^\circ\text{C}$		0.3		%
V_{in}	Operating Input Voltage	$I_O = 100\ \text{mA}$			15	V
I_d	Quiescent Current	$V_{in} \leq 10\ \text{V}$		5	10	mA
I_O	Output Current	$V_{in} = 7.5\ \text{V}$ $T_J = 25^\circ\text{C}$	800	950	1300	mA
eN	Output Noise Voltage	B = 10Hz to 10KHz $T_J = 25^\circ\text{C}$		100		μV
SVR	Supply Voltage Rejection	$I_O = 40\ \text{mA}$ $f = 120\text{Hz}$ $T_J = 25^\circ\text{C}$ $V_{in} = 5.5\ \text{V}$ $V_{\text{ripple}} = 1\ \text{V}_{PP}$	60	75		dB
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$ $T_J = 0$ to 125°C		1	1.1	V
		$I_O = 500\ \text{mA}$ $T_J = 0$ to 125°C		1.05	1.15	
		$I_O = 800\ \text{mA}$ $T_J = 0$ to 125°C		1.10	1.2	
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$			1.1	V
		$I_O = 500\ \text{mA}$			1.2	
		$I_O = 800\ \text{mA}$			1.3	
	Thermal Regulation	$T_a = 25^\circ\text{C}$ 30ms Pulse		0.01	0.1	%/W

LD1117 SERIES**Table 15: Electrical Characteristics Of LD1117#30C** (refer to the test circuits, $T_J = -40$ to 125°C , $C_O = 10\ \mu\text{F}$ unless otherwise specified)

Symbol	Parameter	Test Conditions	Min.	Typ.	Max.	Unit
V_O	Output Voltage	$V_{in} = 5\ \text{V}$ $I_O = 10\ \text{mA}$ $T_J = 25^\circ\text{C}$	2.94	3	3.06	V
V_O	Output Voltage	$I_O = 0$ to $800\ \text{mA}$ $V_{in} = 4.5$ to $10\ \text{V}$	2.88		3.12	V
ΔV_O	Line Regulation	$V_{in} = 4.5$ to $12\ \text{V}$ $I_O = 0\ \text{mA}$		1	30	mV
ΔV_O	Load Regulation	$V_{in} = 4.5\ \text{V}$ $I_O = 0$ to $800\ \text{mA}$		1	30	mV
ΔV_O	Temperature Stability			0.5		%
ΔV_O	Long Term Stability	1000 hrs, $T_J = 125^\circ\text{C}$		0.3		%
V_{in}	Operating Input Voltage	$I_O = 100\ \text{mA}$			15	V
I_d	Quiescent Current	$V_{in} \leq 12\ \text{V}$		5	10	mA
I_O	Output Current	$V_{in} = 8\ \text{V}$ $T_J = 25^\circ\text{C}$	800	950	1300	mA
eN	Output Noise Voltage	$B = 10\text{Hz}$ to 10KHz $T_J = 25^\circ\text{C}$		100		μV
SVR	Supply Voltage Rejection	$I_O = 40\ \text{mA}$ $f = 120\text{Hz}$ $T_J = 25^\circ\text{C}$ $V_{in} = 6\ \text{V}$ $V_{\text{ripple}} = 1\ \text{V}_{PP}$	60	75		dB
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$ $T_J = 0$ to 125°C		1	1.1	V
		$I_O = 500\ \text{mA}$ $T_J = 0$ to 125°C		1.05	1.15	
		$I_O = 800\ \text{mA}$ $T_J = 0$ to 125°C		1.10	1.2	
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$			1.1	V
		$I_O = 500\ \text{mA}$			1.2	
		$I_O = 800\ \text{mA}$			1.3	
	Thermal Regulation	$T_a = 25^\circ\text{C}$ 30ms Pulse		0.01	0.1	%/W

LD1117 SERIES**Table 16: Electrical Characteristics Of LD1117#33C** (refer to the test circuits, $T_J = -40$ to 125°C , $C_O = 10\ \mu\text{F}$ unless otherwise specified)

Symbol	Parameter	Test Conditions	Min.	Typ.	Max.	Unit
V_O	Output Voltage	$V_{in} = 5.3\ \text{V}$ $I_O = 10\ \text{mA}$ $T_J = 25^\circ\text{C}$	3.24	3.3	3.36	V
V_O	Output Voltage	$I_O = 0$ to $800\ \text{mA}$ $V_{in} = 4.75$ to $10\ \text{V}$	3.16		3.44	V
ΔV_O	Line Regulation	$V_{in} = 4.75$ to $15\ \text{V}$ $I_O = 0\ \text{mA}$		1	30	mV
ΔV_O	Load Regulation	$V_{in} = 4.75\ \text{V}$ $I_O = 0$ to $800\ \text{mA}$		1	30	mV
ΔV_O	Temperature Stability			0.5		%
ΔV_O	Long Term Stability	1000 hrs, $T_J = 125^\circ\text{C}$		0.3		%
V_{in}	Operating Input Voltage	$I_O = 100\ \text{mA}$			15	V
I_d	Quiescent Current	$V_{in} \leq 15\ \text{V}$		5	10	mA
I_O	Output Current	$V_{in} = 8.3\ \text{V}$ $T_J = 25^\circ\text{C}$	800	950	1300	mA
eN	Output Noise Voltage	$B = 10\text{Hz}$ to 10KHz $T_J = 25^\circ\text{C}$		100		μV
SVR	Supply Voltage Rejection	$I_O = 40\ \text{mA}$ $f = 120\text{Hz}$ $T_J = 25^\circ\text{C}$ $V_{in} = 6.3\ \text{V}$ $V_{\text{ripple}} = 1\ \text{V}_{PP}$	60	75		dB
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$ $T_J = 0$ to 125°C		1	1.1	V
		$I_O = 500\ \text{mA}$ $T_J = 0$ to 125°C		1.05	1.15	
		$I_O = 800\ \text{mA}$ $T_J = 0$ to 125°C		1.10	1.2	
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$			1.1	V
		$I_O = 500\ \text{mA}$			1.2	
		$I_O = 800\ \text{mA}$			1.3	
	Thermal Regulation	$T_a = 25^\circ\text{C}$ 30ms Pulse		0.01	0.1	%/W

LD1117 SERIES**Table 17: Electrical Characteristics Of LD1117#50C** (refer to the test circuits, $T_J = -40$ to 125°C , $C_O = 10\ \mu\text{F}$ unless otherwise specified)

Symbol	Parameter	Test Conditions	Min.	Typ.	Max.	Unit
V_O	Output Voltage	$V_{in} = 7\ \text{V}$ $I_O = 10\ \text{mA}$ $T_J = 25^\circ\text{C}$	4.9	5	5.1	V
V_O	Output Voltage	$I_O = 0$ to $800\ \text{mA}$ $V_{in} = 6.5$ to $15\ \text{V}$	4.8		5.2	V
ΔV_O	Line Regulation	$V_{in} = 6.5$ to $15\ \text{V}$ $I_O = 0\ \text{mA}$		1	50	mV
ΔV_O	Load Regulation	$V_{in} = 6.5\ \text{V}$ $I_O = 0$ to $800\ \text{mA}$		1	50	mV
ΔV_O	Temperature Stability			0.5		%
ΔV_O	Long Term Stability	1000 hrs, $T_J = 125^\circ\text{C}$		0.3		%
V_{in}	Operating Input Voltage	$I_O = 100\ \text{mA}$			15	V
I_d	Quiescent Current	$V_{in} \leq 15\ \text{V}$		5	10	mA
I_O	Output Current	$V_{in} = 10\ \text{V}$ $T_J = 25^\circ\text{C}$	800	950	1300	mA
eN	Output Noise Voltage	$B = 10\text{Hz}$ to 10KHz $T_J = 25^\circ\text{C}$		100		μV
SVR	Supply Voltage Rejection	$I_O = 40\ \text{mA}$ $f = 120\text{Hz}$ $T_J = 25^\circ\text{C}$ $V_{in} = 8\ \text{V}$ $V_{\text{ripple}} = 1\ \text{V}_{PP}$	60	75		dB
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$ $T_J = 0$ to 125°C		1	1.1	V
		$I_O = 500\ \text{mA}$ $T_J = 0$ to 125°C		1.05	1.15	
		$I_O = 800\ \text{mA}$ $T_J = 0$ to 125°C		1.10	1.2	
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$			1.1	V
		$I_O = 500\ \text{mA}$			1.2	
		$I_O = 800\ \text{mA}$			1.3	
	Thermal Regulation	$T_a = 25^\circ\text{C}$ 30ms Pulse		0.01	0.1	%/W

LD1117 SERIES**Table 18: Electrical Characteristics Of LD1117C (ADJUSTABLE)** (refer to the test circuits, $T_J = -40$ to 125°C , $C_O = 10\ \mu\text{F}$ unless otherwise specified)

Symbol	Parameter	Test Conditions	Min.	Typ.	Max.	Unit
V_{ref}	Reference Voltage	$V_{\text{in}} - V_O = 2\ \text{V}$ $I_O = 10\ \text{mA}$ $T_J = 25^\circ\text{C}$	1.225	1.25	1.275	V
V_{ref}	Reference Voltage	$I_O = 10$ to $800\ \text{mA}$ $V_{\text{in}} - V_O = 1.4$ to $10\ \text{V}$	1.2		1.3	V
ΔV_O	Line Regulation	$V_{\text{in}} - V_O = 1.5$ to $13.75\ \text{V}$ $I_O = 10\ \text{mA}$			1	%
ΔV_O	Load Regulation	$V_{\text{in}} - V_O = 3\ \text{V}$ $I_O = 10$ to $800\ \text{mA}$			1	%
ΔV_O	Temperature Stability			0.5		%
ΔV_O	Long Term Stability	1000 hrs, $T_J = 125^\circ\text{C}$		0.3		%
V_{in}	Operating Input Voltage				15	V
I_{adj}	Adjustment Pin Current	$V_{\text{in}} \leq 15\ \text{V}$		60	120	μA
ΔI_{adj}	Adjustment Pin Current Change	$V_{\text{in}} - V_O = 1.4$ to $10\ \text{V}$ $I_O = 10$ to $800\ \text{mA}$		1	10	μA
$I_{O(\text{min})}$	Minimum Load Current	$V_{\text{in}} = 15\ \text{V}$		2	5	mA
I_O	Output Current	$V_{\text{in}} - V_O = 5\ \text{V}$ $T_J = 25^\circ\text{C}$	800	950	1300	mA
eN	Output Noise (% V_O)	B = 10Hz to 10KHz $T_J = 25^\circ\text{C}$		0.003		%
SVR	Supply Voltage Rejection	$I_O = 40\ \text{mA}$ $f = 120\text{Hz}$ $T_J = 25^\circ\text{C}$ $V_{\text{in}} - V_O = 3\ \text{V}$ $V_{\text{ripple}} = 1\ \text{V}_{\text{PP}}$	60	75		dB
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$ $T_J = 0$ to 125°C		1	1.1	V
		$I_O = 500\ \text{mA}$ $T_J = 0$ to 125°C		1.05	1.15	
		$I_O = 800\ \text{mA}$ $T_J = 0$ to 125°C		1.10	1.2	
V_d	Dropout Voltage	$I_O = 100\ \text{mA}$			1.1	V
		$I_O = 500\ \text{mA}$			1.2	
		$I_O = 800\ \text{mA}$			1.3	
	Thermal Regulation	$T_a = 25^\circ\text{C}$ 30ms Pulse		0.01	0.1	%/W

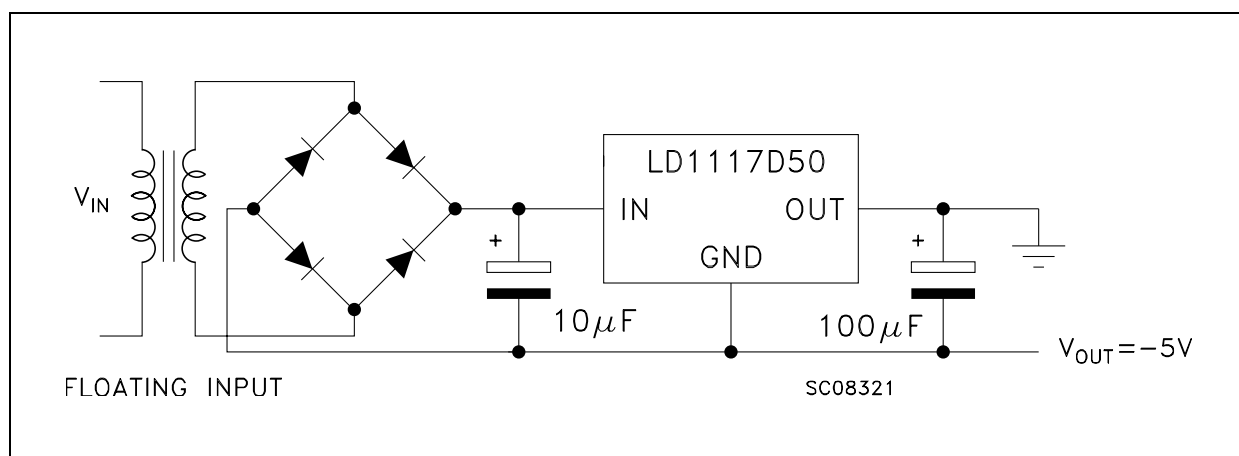
TYPICAL APPLICATIONS**Figure 5: Negative Supply**

Figure 6: Active Terminator for SCSI-2 BUS

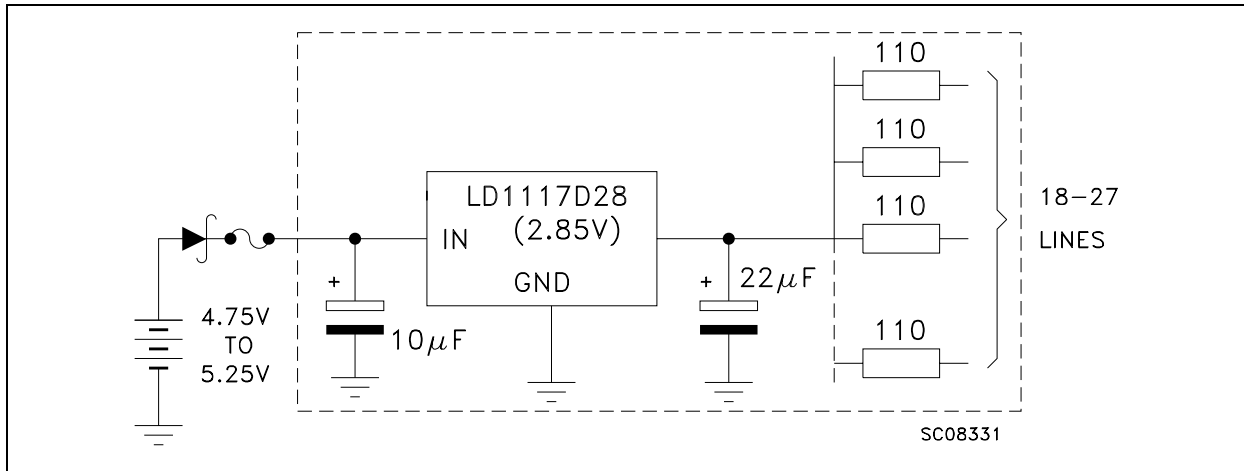


Figure 7: Circuit for Increasing Output Voltage

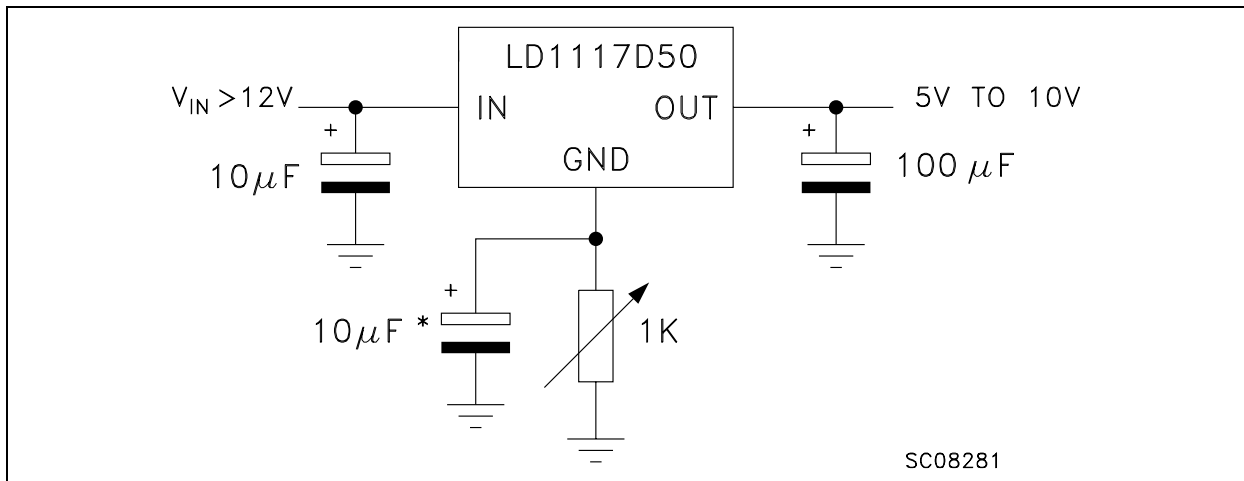
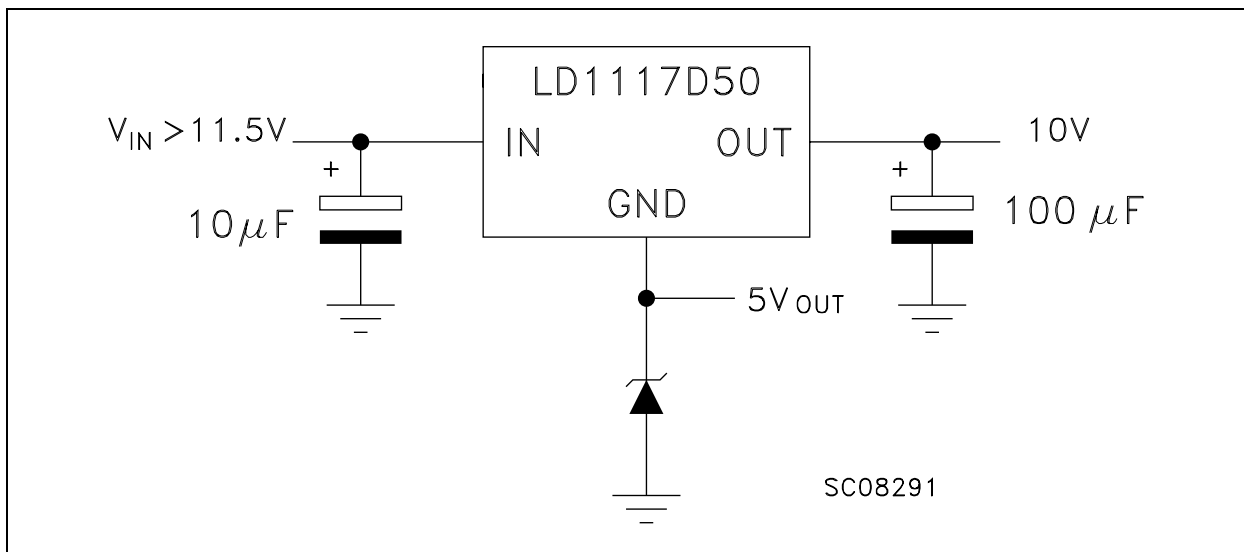


Figure 8: Voltage Regulator With Reference



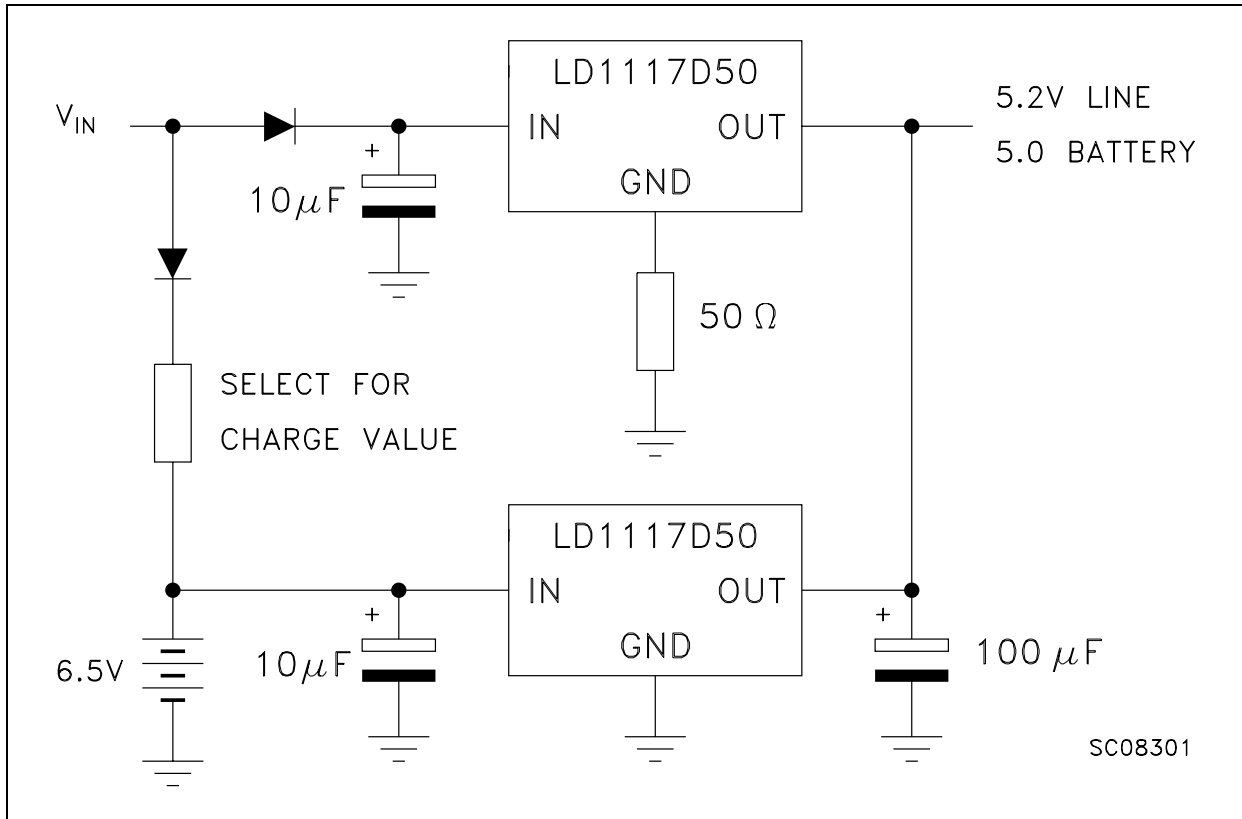
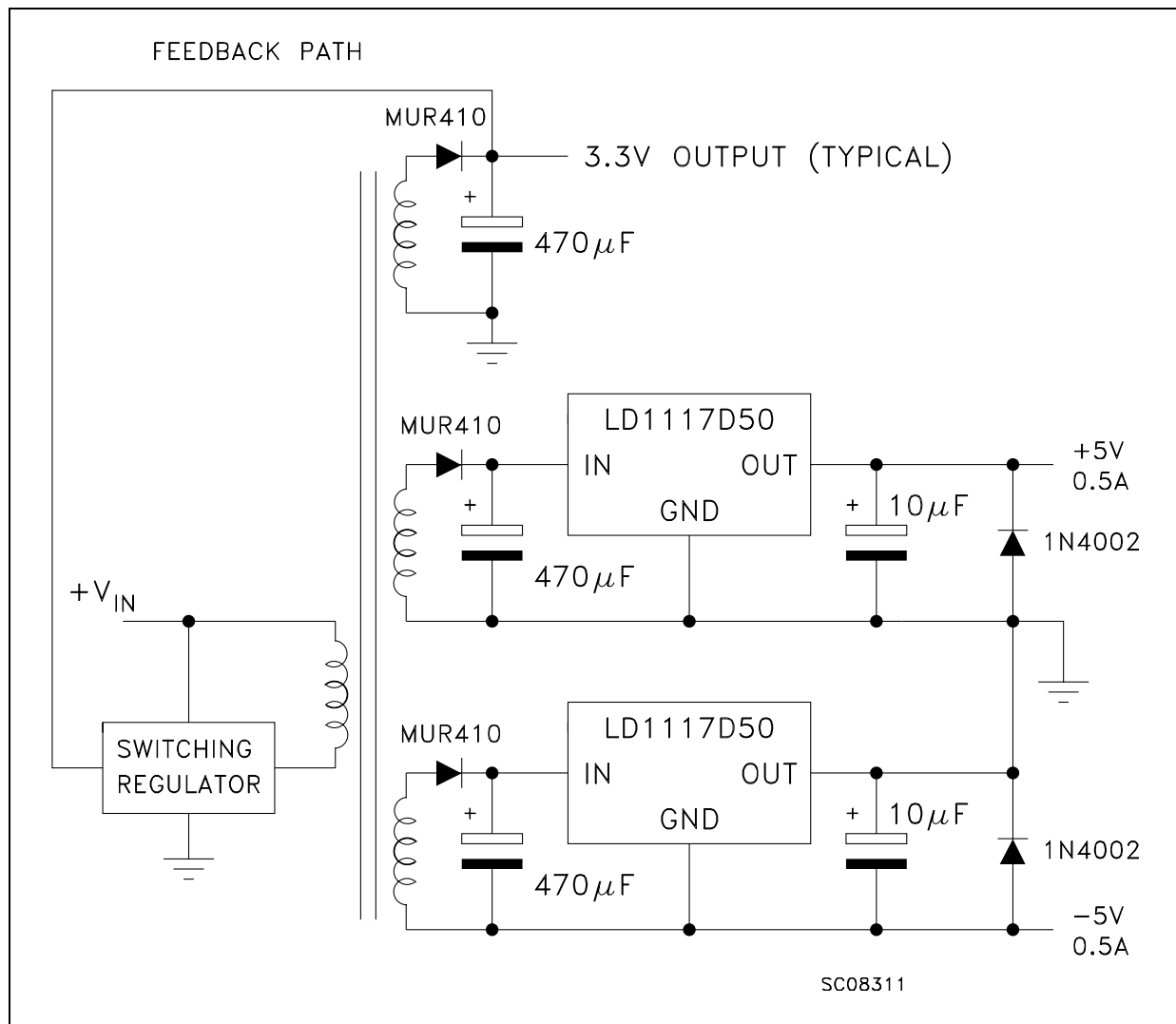
LD1117 SERIES**Figure 9: Battery Backed-up Regulated Supply**

Figure 10: Post-Regulated Dual Supply

**LD1117 ADJUSTABLE: APPLICATION NOTE**

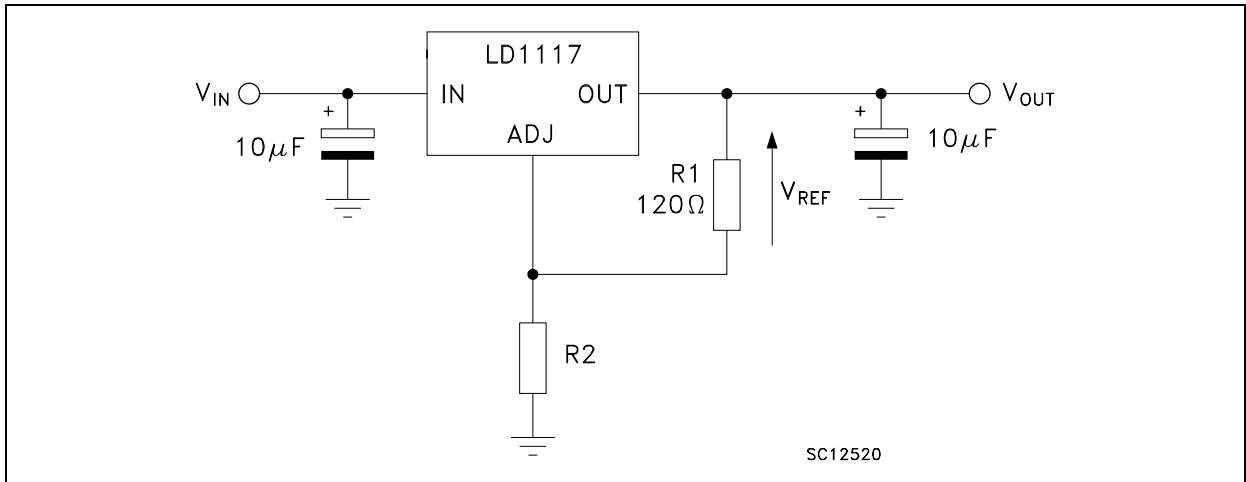
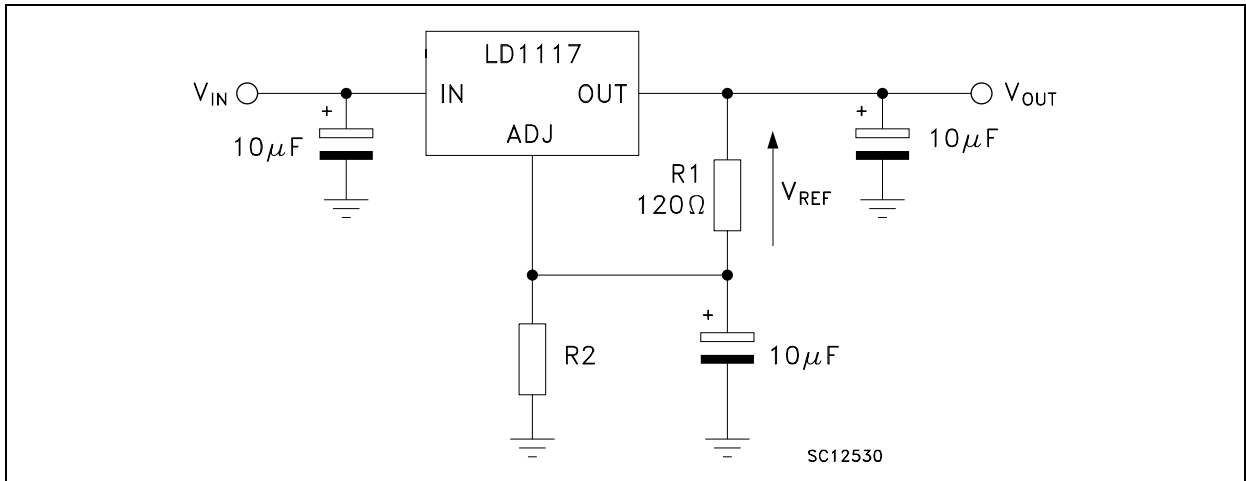
The LD1117 ADJUSTABLE has a thermal stabilized $1.25 \pm 0.012V$ reference voltage between the OUT and ADJ pins. I_{ADJ} is 60 μA typ. (120 μA max.) and ΔI_{ADJ} is 1 μA typ. (5 μA max.). R_1 is normally fixed to 120 Ω . From figure 11 we obtain:

$$V_{OUT} = V_{REF} + R_2 (I_{ADJ} + I_{R1}) = V_{REF} + R_2 (I_{ADJ} + V_{REF}/R_1) = V_{REF} (1 + R_2/R_1) + R_2 \times I_{ADJ}$$

In normal application R_2 value is in the range of few kohm, so the $R_2 \times I_{ADJ}$ product could not be considered in the V_{OUT} calculation; then the above expression becomes:

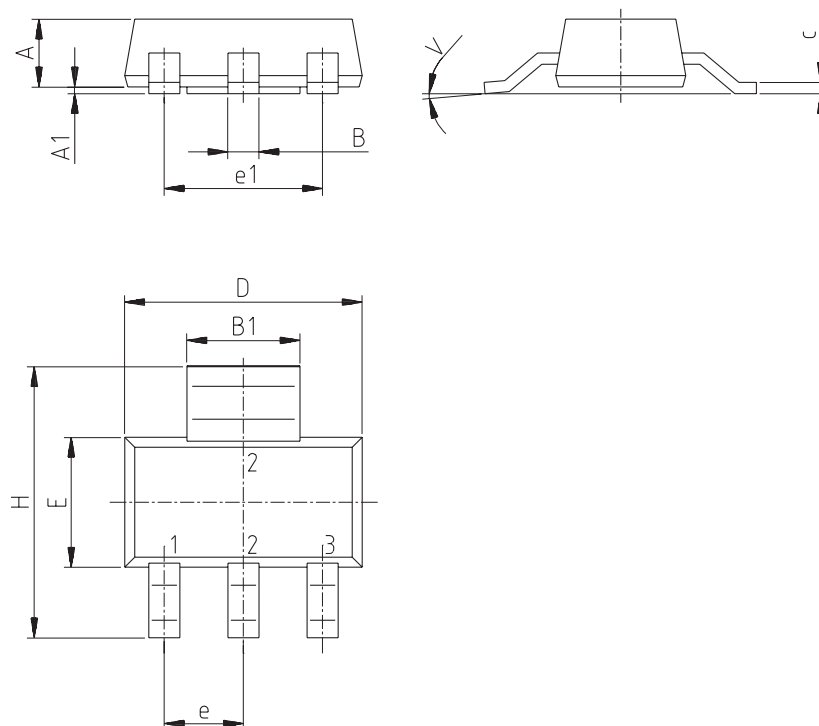
$$V_{OUT} = V_{REF} (1 + R_2/R_1)$$

In order to have the better load regulation it is important to realize a good Kelvin connection of R_1 and R_2 resistors. In particular R_1 connection must be realized very close to OUT and ADJ pin, while R_2 ground connection must be placed as near as possible to the negative Load pin. Ripple rejection can be improved by introducing a 10 μF electrolytic capacitor placed in parallel to the R_2 resistor (see Fig. 12).

LD1117 SERIES**Figure 11: Adjustable Output Voltage Application****Figure 12: Adjustable Output Voltage Application with improved Ripple Rejection**

SOT-223 MECHANICAL DATA

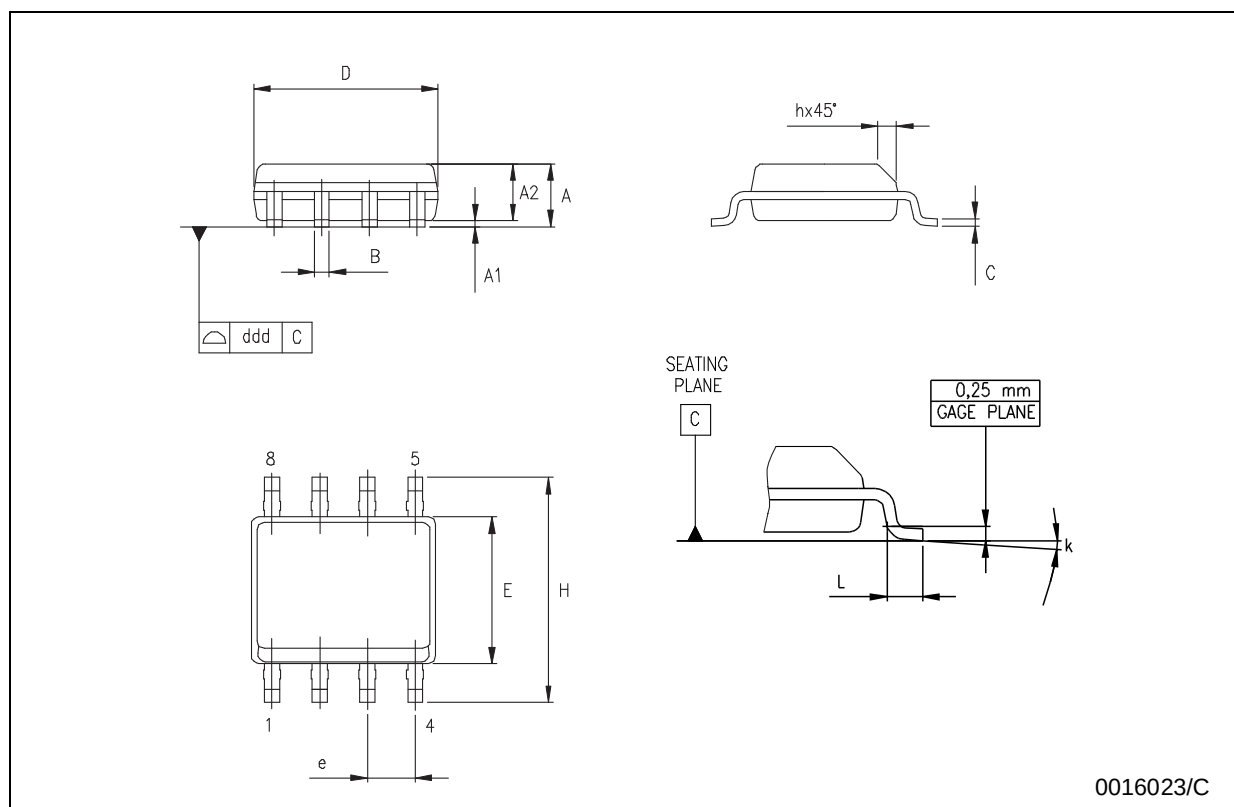
DIM.	mm.			mils		
	MIN.	TYP	MAX.	MIN.	TYP.	MAX.
A			1.8			70.9
A1	0.02		0.1	0.8		3.9
B	0.6	0.7	0.85	23.6	27.6	33.5
B1	2.9	3	3.15	114.2	118.1	124.0
c	0.24	0.26	0.35	9.4	10.2	13.8
D	6.3	6.5	6.7	248.0	255.9	263.8
e		2.3			90.6	
e1		4.6			181.1	
E	3.3	3.5	3.7	129.9	137.8	145.7
H	6.7	7	7.3	129.9	137.8	145.7
V			10°			10°



0046067/H

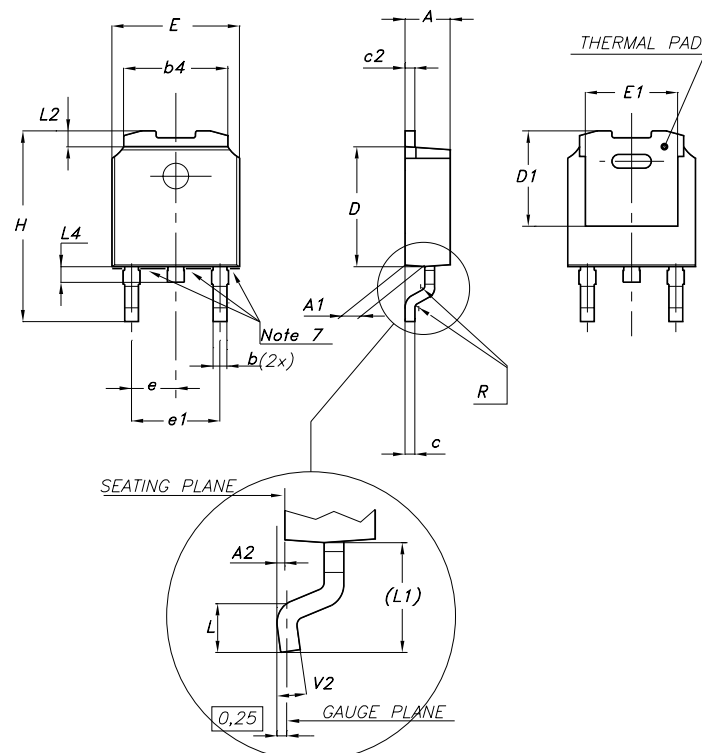
LD1117 SERIES**SO-8 MECHANICAL DATA**

DIM.	mm.			inch		
	MIN.	TYP	MAX.	MIN.	TYP.	MAX.
A	1.35		1.75	0.053		0.069
A1	0.10		0.25	0.04		0.010
A2	1.10		1.65	0.043		0.065
B	0.33		0.51	0.013		0.020
C	0.19		0.25	0.007		0.010
D	4.80		5.00	0.189		0.197
E	3.80		4.00	0.150		0.157
e		1.27			0.050	
H	5.80		6.20	0.228		0.244
h	0.25		0.50	0.010		0.020
L	0.40		1.27	0.016		0.050
k	g° (max.)					
ddd			0.1			0.04



DPAK MECHANICAL DATA

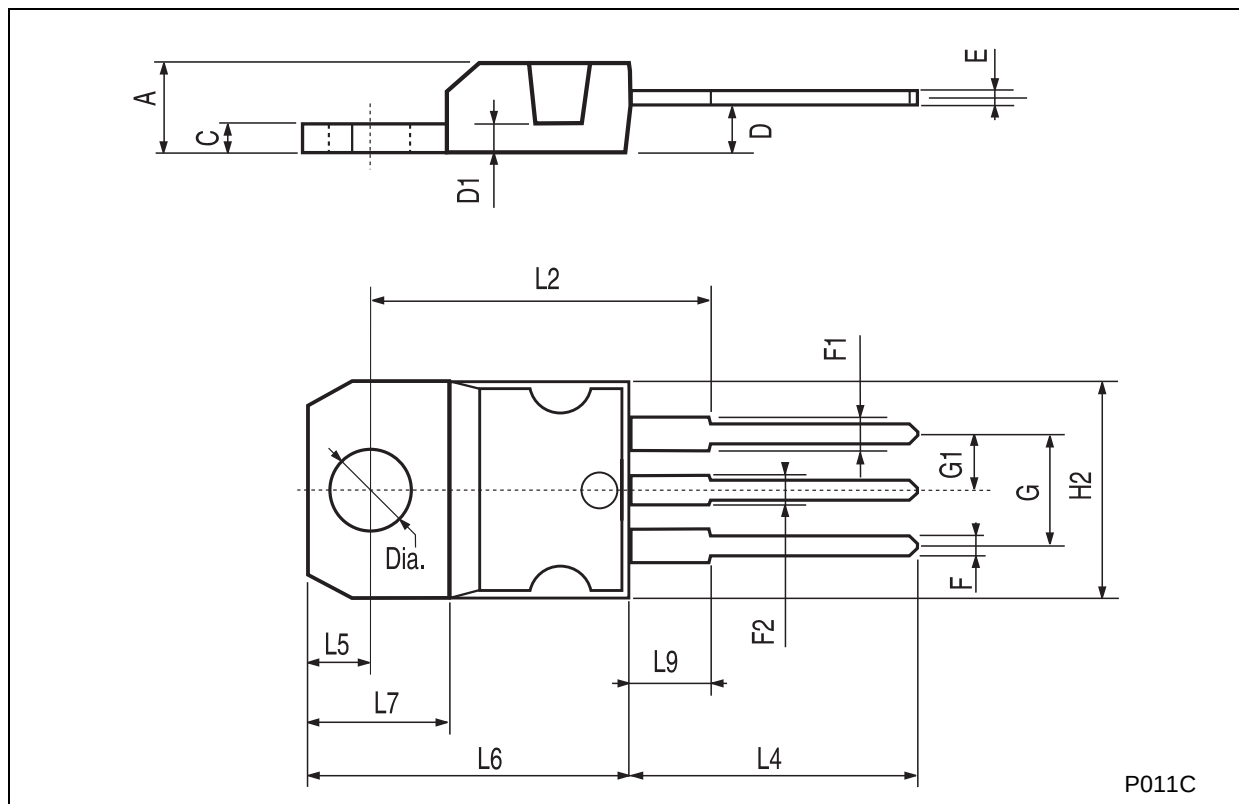
DIM.	mm.			inch		
	MIN.	TYP	MAX.	MIN.	TYP.	MAX.
A	2.2		2.4	0.086		0.094
A1	0.9		1.1	0.035		0.043
A2	0.03		0.23	0.001		0.009
B	0.64		0.9	0.025		0.035
b4	5.2		5.4	0.204		0.212
C	0.45		0.6	0.017		0.023
C2	0.48		0.6	0.019		0.023
D	6		6.2	0.236		0.244
D1		5.1			0.200	
E	6.4		6.6	0.252		0.260
E1		4.7			0.185	
e		2.28			0.090	
e1	4.4		4.6	0.173		0.181
H	9.35		10.1	0.368		0.397
L	1			0.039		
(L1)		2.8			0.110	
L2		0.8			0.031	
L4	0.6		1	0.023		0.039



0068772-F

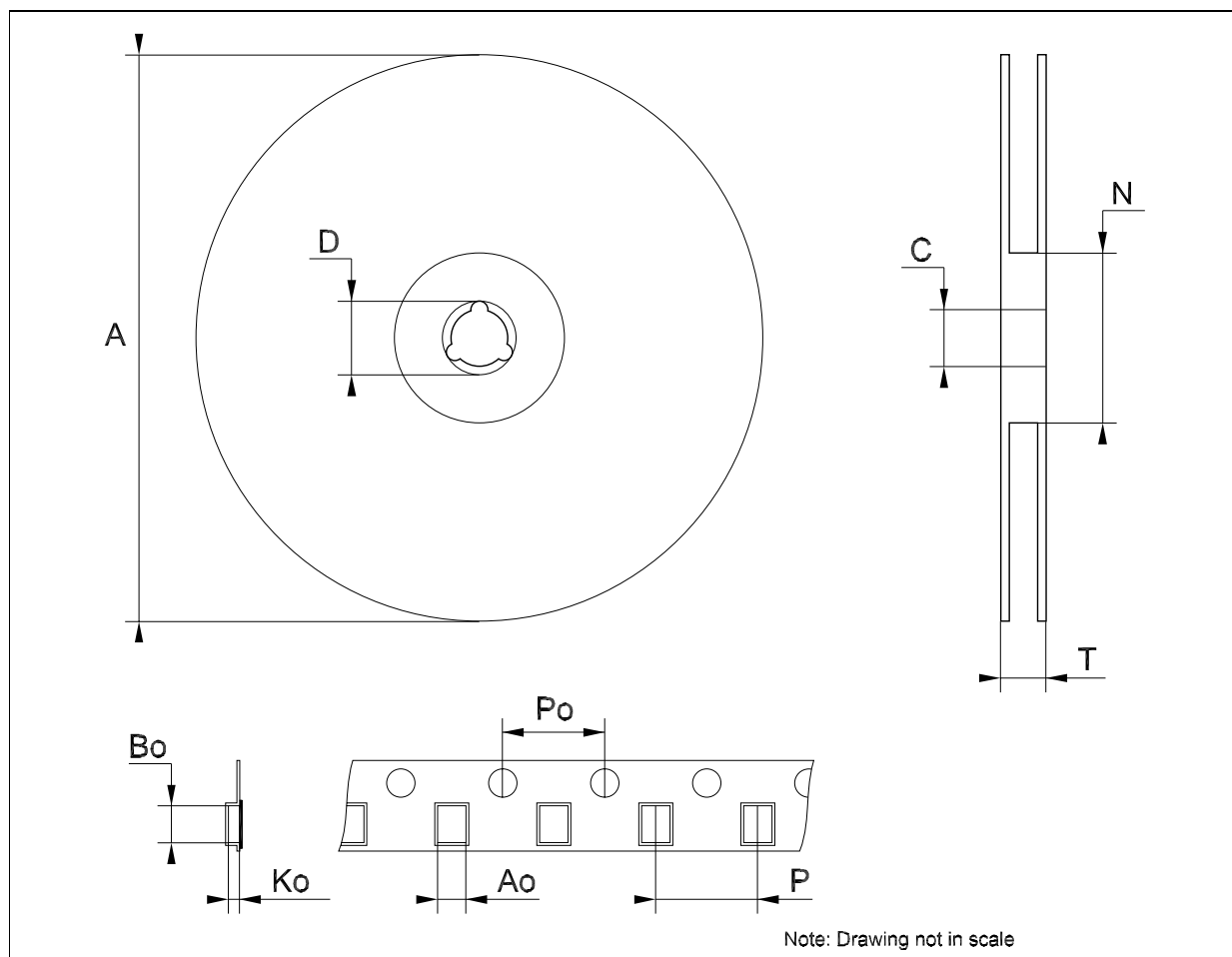
LD1117 SERIES**TO-220 MECHANICAL DATA**

DIM.	mm.			inch		
	MIN.	TYP	MAX.	MIN.	TYP.	MAX.
A	4.40		4.60	0.173		0.181
C	1.23		1.32	0.048		0.051
D	2.40		2.72	0.094		0.107
D1		1.27			0.050	
E	0.49		0.70	0.019		0.027
F	0.61		0.88	0.024		0.034
F1	1.14		1.70	0.044		0.067
F2	1.14		1.70	0.044		0.067
G	4.95		5.15	0.194		0.203
G1	2.4		2.7	0.094		0.106
H2	10.0		10.40	0.393		0.409
L2		16.4			0.645	
L4	13.0		14.0	0.511		0.551
L5	2.65		2.95	0.104		0.116
L6	15.25		15.75	0.600		0.620
L7	6.2		6.6	0.244		0.260
L9	3.5		3.93	0.137		0.154
DIA.	3.75		3.85	0.147		0.151



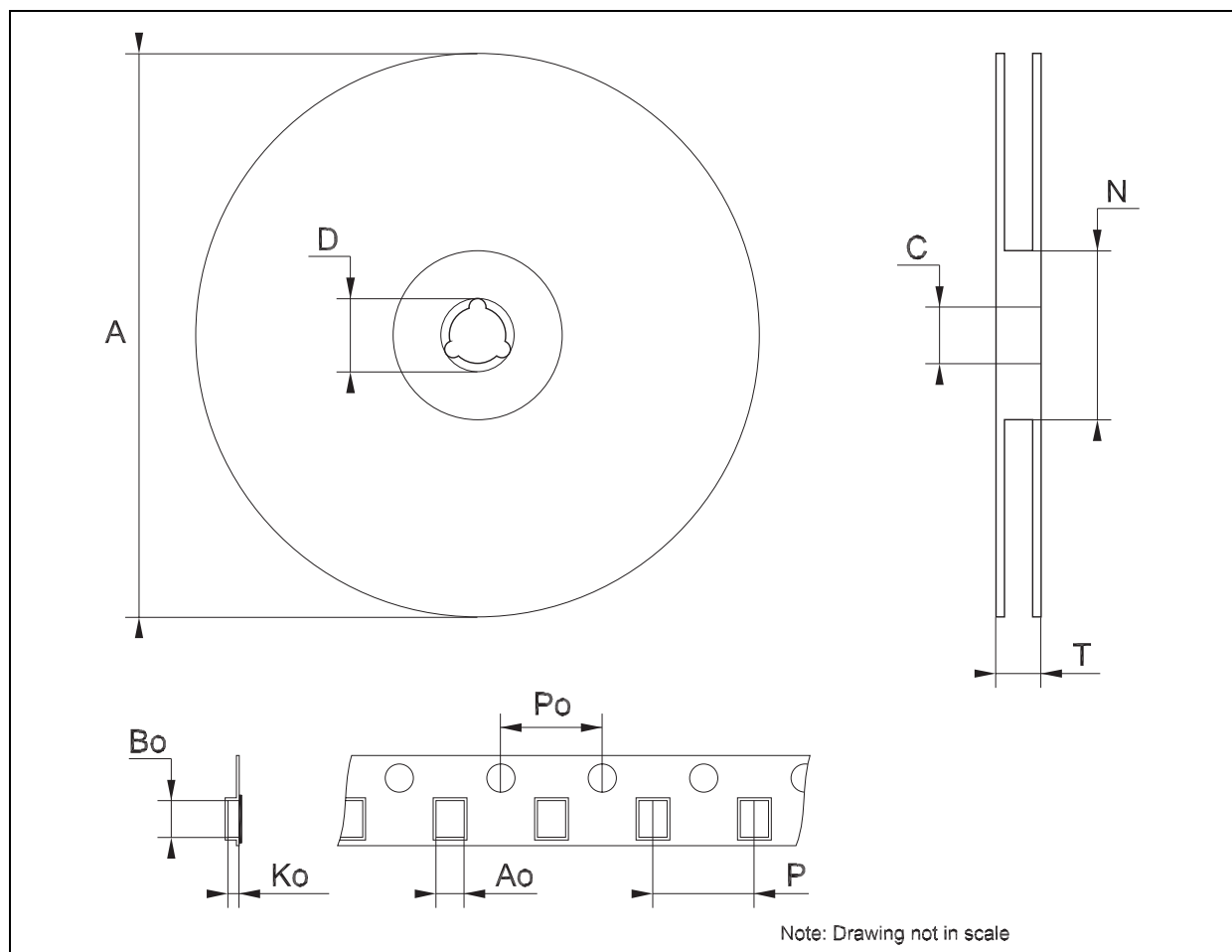
Tape & Reel SOT223 MECHANICAL DATA

DIM.	mm.			inch		
	MIN.	TYP	MAX.	MIN.	TYP.	MAX.
A			330			12.992
C	12.8	13.0	13.2	0.504	0.512	0.519
D	20.2			0.795		
N	60			2.362		
T			14.4			0.567
Ao	6.73	6.83	6.93	0.265	0.269	0.273
Bo	7.32	7.42	7.52	0.288	0.292	0.296
Ko	1.78		2	0.070		0.078
Po	3.9	4.0	4.1	0.153	0.157	0.161
P	7.9	8.0	8.1	0.311	0.315	0.319



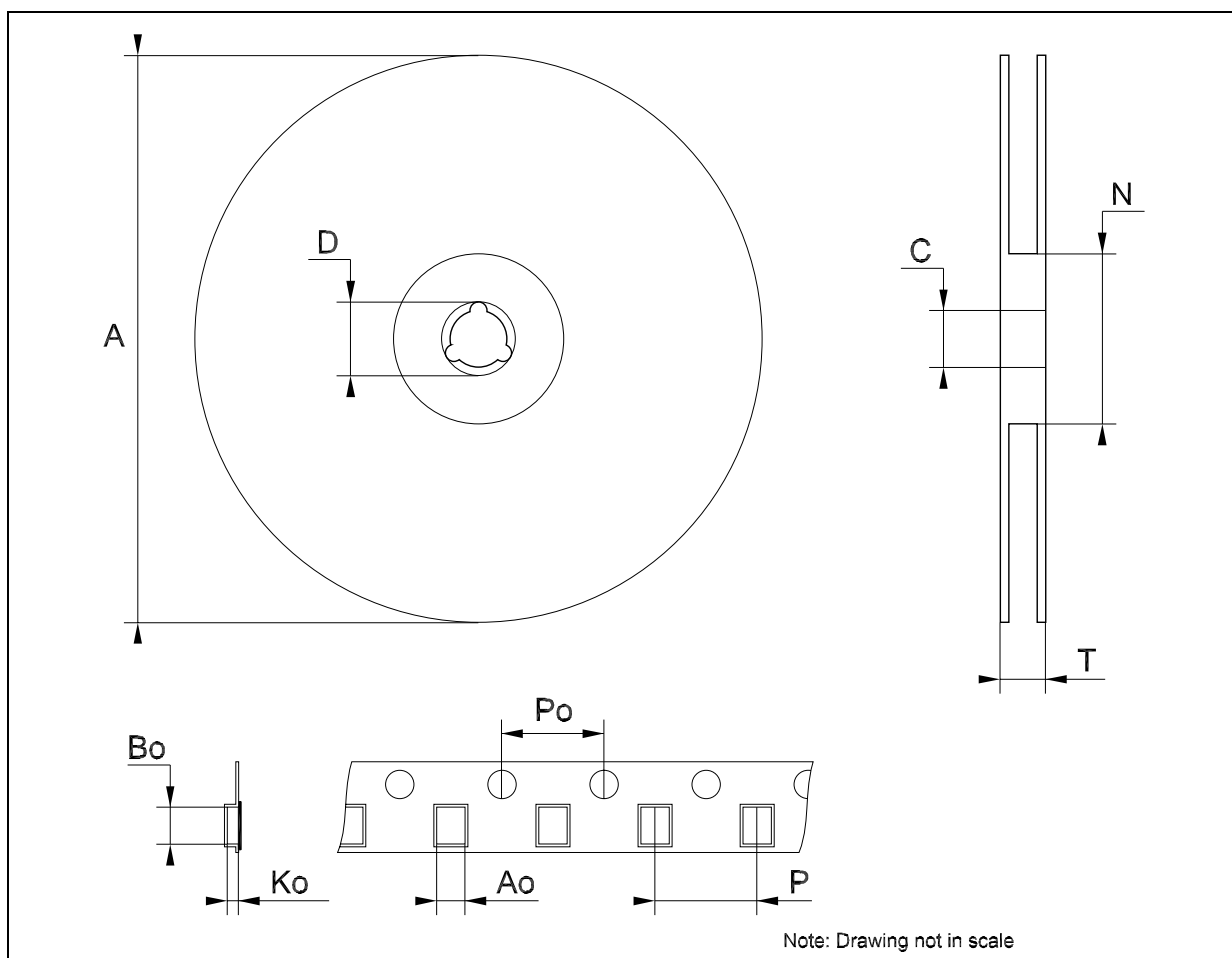
LD1117 SERIES**Tape & Reel SO-8 MECHANICAL DATA**

DIM.	mm.			inch		
	MIN.	TYP	MAX.	MIN.	TYP.	MAX.
A			330			12.992
C	12.8		13.2	0.504		0.519
D	20.2			0.795		
N	60			2.362		
T			22.4			0.882
Ao	8.1		8.5	0.319		0.335
Bo	5.5		5.9	0.216		0.232
Ko	2.1		2.3	0.082		0.090
Po	3.9		4.1	0.153		0.161
P	7.9		8.1	0.311		0.319



Tape & Reel DPAK-PPAK MECHANICAL DATA

DIM.	mm.			inch		
	MIN.	TYP	MAX.	MIN.	TYP.	MAX.
A			330			12.992
C	12.8	13.0	13.2	0.504	0.512	0.519
D	20.2			0.795		
N	60			2.362		
T			22.4			0.882
Ao	6.80	6.90	7.00	0.268	0.272	0.276
Bo	10.40	10.50	10.60	0.409	0.413	0.417
Ko	2.55	2.65	2.75	0.100	0.104	0.105
Po	3.9	4.0	4.1	0.153	0.157	0.161
P	7.9	8.0	8.1	0.311	0.315	0.319



LD1117 SERIES

Table 19: Revision History

Date	Revision	Description of Changes
22-Sep-2004	15.0	Add new Part Number #12C; Typing Error: Note on table 2.
25-Oct-2004	16.0	Add V_{ref} Reference Voltage on Table 12.
18-Jul-2005	17.0	The DPAK Mechanical Data has been updated.
25-Nov-2005	18.0	The TO220FM Package has been removed.
14-Dec-2005	19.0	The T_{op} on Table 2 has been updated.

Information furnished is believed to be accurate and reliable. However, STMicroelectronics assumes no responsibility for the consequences of use of such information nor for any infringement of patents or other rights of third parties which may result from its use. No license is granted by implication or otherwise under any patent or patent rights of STMicroelectronics. Specifications mentioned in this publication are subject to change without notice. This publication supersedes and replaces all information previously supplied. STMicroelectronics products are not authorized for use as critical components in life support devices or systems without express written approval of STMicroelectronics.

The ST logo is a registered trademark of STMicroelectronics
All other names are the property of their respective owners

© 2005 STMicroelectronics - All Rights Reserved

STMicroelectronics group of companies

Australia - Belgium - Brazil - Canada - China - Czech Republic - Finland - France - Germany - Hong Kong - India - Israel - Italy - Japan -
Malaysia - Malta - Morocco - Singapore - Spain - Sweden - Switzerland - United Kingdom - United States of America
www.st.com

